
PyModbus

Release 4.0.0dev6

Open Source volunteers

Oct 10, 2025

CONTENTS:

1	Pymodbus 4.0 upgrade procedure	3
2	API changes	5
2.1	API changes 3.12.0	5
2.2	API changes 3.11.0	5
2.3	API changes 3.10.0	5
2.4	API changes 3.9.0	5
2.5	API changes 3.8.0	6
2.6	API changes 3.7.0	6
2.7	API changes 3.6.0	6
2.8	API changes 3.5.0	6
2.9	API changes 3.4.0	7
2.10	API changes 3.3.0	7
2.11	API changes 3.2.0	7
2.12	API changes 3.1.0	8
2.13	API changes 3.0.0	8
3	Client	9
3.1	Client performance	9
3.2	Client protocols/framers	9
3.2.1	Serial (RS-485)	10
3.2.2	TCP	10
3.2.3	TLS	10
3.2.4	UDP	10
3.3	Client usage	10
3.3.1	Synchronous example	10
3.3.2	Asynchronous example	11
3.3.3	Retry logic for async clients	11
3.3.4	Development notes	11
3.4	Client device addressing	11
3.5	Client response handling	12
3.6	Client interface classes	12
3.6.1	Client common	12
3.6.2	Client serial	14
3.6.3	Client TCP	17
3.6.4	Client TLS	19
3.6.5	Client UDP	22
3.7	Modbus calls	24
4	Server (3.x)	37

5	Pymodbus REPL (Read Evaluate Print Loop)	45
5.1	Install as pymodbus optional dependency	45
5.2	Install directly from the github repo	45
6	Simulator (3.x)	47
6.1	Configuration	47
6.1.1	Json file layout	48
6.1.2	Server entries	48
6.1.3	Server configuration examples	49
6.1.4	Device entries	50
6.1.4.1	Setup section	52
6.1.4.2	Invalid section	54
6.1.4.3	Write section	54
6.1.4.4	Bits section	54
6.1.4.5	Uint16 section	55
6.1.4.6	Uint32 section	55
6.1.4.7	Float32 section	55
6.1.4.8	String section	55
6.1.4.9	Repeat section	56
6.1.5	Device configuration examples	56
6.1.6	Configuration used for test	60
6.2	Simulator datastore	64
6.3	Web frontend v3.x	64
6.3.1	pymodbus.simulator (v3.x)	64
6.4	Pymodbus simulator ReST API	67
6.4.1	Registers Endpoint	67
6.4.1.1	/restapi/registers	67
6.4.2	Calls Endpoint	69
6.4.2.1	/restapi/calls	69
6.4.3	Server Endpoint	73
6.4.4	Log Endpoint	73
7	Simulator	75
8	Server/Simulator	77
8.1	Data model configuration	78
8.1.1	Usage examples	78
8.1.2	Class definitions	79
8.1.3	Action data class examples	83
8.2	Simulator server	83
8.3	Web frontend	83
8.4	REST API	83
9	Examples	85
9.1	Ready to run examples:	85
9.1.1	Simple asynchronous client	85
9.1.2	Simple synchronous client	87
9.2	Advanced examples	89
9.2.1	Client asynchronous calls	89
9.2.2	Client asynchronous	90
9.2.3	Client calls	90
9.2.4	Custom message	91
9.2.5	Client synchronous	91
9.2.6	Server asynchronous	92
9.2.7	Server callback	92

9.2.8	Server tracer	93
9.2.9	Server synchronous	93
9.2.10	Server updating	93
9.2.11	Simulator example	94
9.2.12	Simulator datastore (shared storage) example	94
9.2.13	Message Parser	95
9.3	Examples contributions	95
9.3.1	Solar	95
9.3.2	Serial Forwarder	95
10	Authors	97
10.1	Pymodbus version 3 family	97
10.2	Pymodbus version 2 family	100
10.3	Pymodbus version 1 family	101
10.4	Pymodbus version 0 family	102
11	Changelog	103
11.1	Version 3.11.3	103
11.2	Version 3.11.2	103
11.3	Version 3.11.1	104
11.4	Version 3.11.0	104
11.5	Version 3.10.0	105
11.6	Version 3.9.2	105
11.7	Version 3.9.1	105
11.8	Version 3.9.0	105
11.9	Version 3.8.6	106
11.10	Version 3.8.5	106
11.11	Version 3.8.4	106
11.12	Version 3.8.3	107
11.13	Version 3.8.2	107
11.14	Version 3.8.1	107
11.15	Version 3.8.0	107
11.16	Version 3.7.4	109
11.17	Version 3.7.3	110
11.18	Version 3.7.2	111
11.19	Version 3.7.1	111
11.20	Version 3.7.0	111
11.21	Version 3.6.9	112
11.22	Version 3.6.8	113
11.23	Version 3.6.7	113
11.24	Version 3.6.6	114
11.25	Version 3.6.5	114
11.26	Version 3.6.4	116
11.27	Version 3.6.3	116
11.28	Version 3.6.2	117
11.29	Version 3.6.1	117
11.30	Version 3.6.0	117
11.31	Version 3.5.4	119
11.32	Version 3.5.3	119
11.33	Version 3.5.2	119
11.34	Version 3.5.1	119
11.35	Version 3.5.0	120
11.36	Version 3.4.1	120
11.37	Version 3.4.0	121

11.38	Version 3.3.2	122
11.39	Version 3.3.1	122
11.40	Version 3.3.0	123
11.41	Version 3.2.2	124
11.42	Version 3.2.1	124
11.43	Version 3.2.0	124
11.44	Version 3.1.3	125
11.45	Version 3.1.2	126
11.46	Version 3.1.1	126
11.47	Version 3.1.0	126
11.48	Version 3.0.2	127
11.49	Version 3.0.1	128
11.50	Version 3.0.0	128
11.51	Version 2.5.3	130
11.52	Version 2.5.2	130
11.53	Version 2.5.1	130
11.54	Version 2.5.0	131
11.55	Version 2.4.0	132
11.56	Version 2.3.0	132
11.57	Version 2.2.0	132
11.58	Version 2.1.0	133
11.59	Version 2.0.1	133
11.60	Version 2.0.0	133
11.61	Version 1.5.2	134
11.62	Version 1.5.1	134
11.63	Version 1.5.0	134
11.64	Version 1.4.0	134
11.65	Version 1.3.2	135
11.66	Version 1.3.1	135
11.67	Version 1.2.0	136
11.68	Version 1.1.0	136
11.69	Version 1.0.0	136
12	Pymodbus internals	137
12.1	NullModem	137
12.2	Datastore	137
12.2.1	Datastore classes	137
12.3	Framer	141
12.3.1	pymodbus.framer.FramerAscii module	141
12.3.2	pymodbus.framer.FramerRTU module	141
12.3.3	pymodbus.framer.FramerSocket module	143
12.3.4	pymodbus.framer.FramerTLS module	143
12.4	Constants	143
12.5	Extra functions	146
12.6	PDU classes	149
12.7	Architecture	175
13	PyModbus - A Python Modbus Stack	177
13.1	Pymodbus in a nutshell	177
13.1.1	Common features	178
13.1.2	Client Features	178
13.1.3	Server Features	178
13.1.4	Simulator Features	178
13.2	Use Cases	179

13.3	Install	179
13.3.1	Install with pip	179
13.3.2	Install with github	180
13.4	Example Code	181
13.5	Contributing	181
13.6	Development instructions	182
13.6.1	Internals	182
13.6.2	Generate documentation	182
13.7	License Information	182
Python Module Index		183
Index		185

Please select a topic in the left hand column.

PYMODBUS 4.0 UPGRADE PROCEDURE

Pymodbus 4.0 contains a number of incompatibilities with Pymodbus 3.x, however most of these are simple edits.

This will be amended, whenever there API changes are merged

API CHANGES

Versions (X.Y.Z) where Z > 0 e.g. 3.0.1 do NOT have API changes!

2.1 API changes 3.12.0

- when using `no_response_expected=`, the call returns `None`

2.2 API changes 3.11.0

- Revert wrong byte handling in v3.10.0 bit handling order is LSB-> MSB for each byte REMARK: word are ordered depending on big/little endian readCoils and other bit functions now return bit in logical order (NOT byte order)

Example: Hex bytes: 0x00 0x01 delivers `False * 8 True False * 7`

Hex bytes: 0x01 0x03 delivers `True False * 7 True True False * 6`

2.3 API changes 3.10.0

- `ModbusSlaveContext` replaced by `ModbusDeviceContext`
- payload removed (replaced by “`convert_to/from_registers`”)
- `slave=`, `slaves=` replaced by `device_id=`, `device_ids=`
- slave request names changed to device
- bit handling order is LSB (last byte) -> MSB (first byte) readCoils and other bit functions now return bit in logical order (NOT byte order)

Older versions had LSB -> MSB pr byte V3.10 have LSB -> MSB across bytes.

Example: Hex bytes: 0x00 0x01 Older versions would deliver `False * 8 True False * 7` V3.10 deliver `True False * 15`

Hex bytes: 0x01 0x03 Older versions would deliver `True False * 7 True True False * 6` V3.10 deliver `True True False * 6 True False * 7`

2.4 API changes 3.9.0

- Python 3.9 is reaching end of life, and no longer supported. Depending on the usage the code might still work
- `Start*Server`, `custom_functions` -> `custom_pdu` (handled by `Modbus<x>Server`)
- Bit handling (e.g. `read_coils`) was not handling the bits in the correct order

2.5 API changes 3.8.0

- ModbusSlaveContext, removed zero_mode parameter.
- Removed skip_encode parameter.
- renamed ModbusExceptions enums to legal constants.
- enforced client keyword only parameters (positional not allowed).
- added trace_packet/pdu/connect to client/server.
- removed on_connect_callback from client.
- removed response_manipulator, request_tracer from server.

2.6 API changes 3.7.0

- default slave changed to 1 from 0 (which is broadcast).
- broadcast_enable, retry_on_empty, no_resend_on_retry parameters removed.
- class method generate_ssl() added to TLS client (sync/async).
- removed certfile, keyfile, password from TLS client, please use generate_ssl()
- on_reconnect_callback() removed from clients (sync/async).
- on_connect_callback(true/false) added to async clients.
- binary framer no longer supported
- Framer.<type> renamed to FramerType.<type>
- PDU classes moved to pymodbus/pdu
- Simulator config custom actions kwargs -> parameters
- Non defined parameters (kwargs) no longer valid
- Drop support for Python 3.8 (its no longer tested, but will probably work)

2.7 API changes 3.6.0

- framer= is an enum: pymodbus.Framer, but still accept a framer class

2.8 API changes 3.5.0

- Remove handler parameter from ModbusUdpServer
- Remove loop parameter from ModbusSerialServer
- Remove handler and allow_reuse_port from repl default config
- Static classes from the constants module are now inheriting from enum.Enum and using UPPER_CASE naming scheme, this affects: - MoreData - DeviceInformation - ModbusPlusOperation - Endian - ModbusStatus
- Async clients now accepts no_resend_on_retry=True, to not resend the request when retrying.
- ModbusSerialServer now accepts request_tracer=.

2.9 API changes 3.4.0

- `Modbus<x>Client.connect()` returns `True/False` (connected or not)
- `Modbus<x>Server.handler=`, `allow_reuse_addr=`, `backlog=` are no longer accepted
- `ModbusTcpClient` / `AsyncModbusTcpClient` no longer support unix path
- `StartAsyncUnixServer` / `ModbusUnixServer` removed (never worked on Windows)
- `ModbusTlsServer.reqlicert=` is no longer accepted
- `ModbusSerialServer.auto_connect=` is no longer accepted
- `ModbusSimulatorServer.serve_forever(only_start=False)` added to allow return

2.10 API changes 3.3.0

- `ModbusTcpDiagClient` is removed due to lack of support
- Clients have an optional parameter: `on_reconnect_callback`, Function that will be called just before a reconnection attempt.
- general parameter `unit=` -> `slave=`
- move `SqlSlaveContext`, `RedisSlaveContext` to `examples/contrib` (due to lack of maintenance)
- `BinaryPayloadBuilder.to_string` was renamed to `BinaryPayloadBuilder.encode`
- `on_reconnect_callback` for async clients works slightly different
- `utilities/unpack_bitstring` now expects an argument named *data* not *string*

2.11 API changes 3.2.0

- helper to convert values in mixin: `convert_from_registers`, `convert_to_registers`
- `import pymodbus.version` -> `from pymodbus import __version__, __version_full__`
- `pymodbus.pymodbus_apply_logging_config(log_file_name="pymodbus.log")` to enable file `pymodbus_apply_logging_config`
- `pymodbus.pymodbus_apply_logging_config` have default `DEBUG`, if not called root settings will be used.
- `pymodbus/interfaces/IModbusDecoder` removed.
- `pymodbus/interfaces/IModbusFramer` removed.
- `pymodbus/interfaces/IModbusSlaveContext` -> `pymodbus/datastore/ModbusBaseSlaveContext`.
- `StartAsync<type>Server`, removed `defer_start` argument, return is `None`. instead of using `defer_start` instantiate the `Modbus<type>Server` directly.
- `ReturnSlaveNoReponseCountResponse` has been corrected to `ReturnSlaveNoResponseCountResponse`
- Option `-modbus-config` for REPL server renamed to `-modbus-config-path`
- `client.protocol.<something>` -> `client.<something>`
- `client.factory.<something>` -> `client.<something>`

2.12 API changes 3.1.0

- Added `--host` to `client_*` examples, to allow easier use.
- `unit=` in client calls are no longer converted to `slave=`, but raises a runtime exception.
- Added missing client calls (all standard request are not available as methods).
- `client.mask_write_register()` changed parameters.
- server classes no longer accept `reuse_port=` (the socket do not accept it)

2.13 API changes 3.0.0

Base for recording changes.

CLIENT

Pymodbus offers both a `synchronous client` and a `asynchronous client`. Both clients offer simple calls for each type of request, as well as a unified response, removing a lot of the complexities in the modbus protocol.

In addition to the “pure” client, pymodbus offers a set of utilities converting to/from registers to/from “normal” python values.

The client is NOT thread safe, meaning the application must ensure that calls are serialized. This is only a problem for synchronous applications that use multiple threads or for asynchronous applications that use multiple `asyncio.create_task`.

It is allowed to have multiple client objects that e.g. each communicate with a TCP based device.

3.1 Client performance

There are currently a big performance gap between the 2 clients.

This is due to a rather old implementation of the synchronous client, we are currently working to update the client code. Our aim is to achieve a similar data rate with both clients and at least double the data rate while keeping the stability. Table below is a test with 1000 calls each reading 10 registers.

client	asynchronous	synchronous
total time	0,33 sec	114,10 sec
ms/call	0,33 ms	114,10 ms
ms/register	0,03 ms	11,41 ms
calls/sec	3.030	8
registers/sec	30.300	87

3.2 Client protocols/framers

Pymodbus offers clients with transport different protocols and different framers

protocol	ASCII	RTU	RTU_OVER_TCP	SOCKET	TLS
SERIAL (RS-485)	Yes	Yes	No	No	No
TCP	Yes	No	Yes	Yes	No
TLS	No	No	No	No	Yes
UDP	Yes	No	Yes	Yes	No

3.2.1 Serial (RS-485)

Pymodbus do not connect to the device (server) but connects to a comm port or usb port on the local computer.

RS-485 is a half duplex protocol, meaning the servers do nothing until the client sends a request then the server being addressed responds. The client controls the traffic and as a consequence one RS-485 line can only have 1 client but upto 254 servers (physical devices).

RS-485 is a simple 2 wire cabling with a pullup resistor. It is important to note that many USB converters do not have a builtin resistor, this must be added manually. When experiencing many faulty packets and retries this is often the problem.

3.2.2 TCP

Pymodbus connects directly to the device using a standard socket and have a one-to-one connection with the device. In case of multiple TCP devices the application must instantiate multiple client objects one for each connection.

Tip

a TCP device often represent multiple physical devices (e.g Ethernet-RS485 converter), each of these devices can be addressed normally

3.2.3 TLS

A variant of **TCP** that uses encryption and certificates. **TLS** is mostly used when the devices are connected to the internet.

3.2.4 UDP

A broadcast variant of **TCP**. **UDP** allows addressing of many devices with a single request, however there are no control that a device have received the packet.

3.3 Client usage

Using pymodbus client to set/get information from a device (server) is done in a few simple steps.

3.3.1 Synchronous example

```
from pymodbus.client import ModbusTcpClient

client = ModbusTcpClient('MyDevice.lan')      # Create client object
client.connect()                               # connect to device
client.write_coil(1, True, device_id=1)        # set information in device
result = client.read_coils(2, 3, device_id=1)  # get information from device
print(result.bits[0])                          # use information
client.close()                                 # Disconnect device
```

The line `client.connect()` connects to the device (or comm port). If this cannot connect successfully within the timeout it throws an exception. After this initial connection, further calls to the same client (here, `client.write_coil(...)` and `client.read_coils(...)`) will check whether the client is still connected, and automatically reconnect if not.

3.3.2 Asynchronous example

```
from pymodbus.client import AsyncModbusTcpClient

client = AsyncModbusTcpClient('MyDevice.lan')      # Create client object
await client.connect()                             # connect to device, reconnect_
↳ automatically
await client.write_coil(1, True, device_id=1)       # set information in device
result = await client.read_coils(2, 3, device_id=1) # get information from device
print(result.bits[0])                             # use information
client.close()                                     # Disconnect device
```

The line `client = AsyncModbusTcpClient('MyDevice.lan')` only creates the object; it does not activate anything.

The line `await client.connect()` connects to the device (or comm port), if this cannot connect successfully within the timeout it throws an exception. If connected successfully reconnecting later is handled automatically

The line `await client.write_coil(1, True, device_id=1)` is an example of a write request, set address 1 to True on device 1.

The line `result = await client.read_coils(2, 3, device_id=1)` is an example of a read request, get the value of address 2, 3 and 4 (count = 3) from device 1.

The last line `client.close()` closes the connection and render the object inactive.

3.3.3 Retry logic for async clients

If no response is received to a request (call), it is retried (parameter `retries`) times, if not successful an exception response is returned, BUT the connection is not touched.

If 3 consecutive requests (calls) do not receive a response, the connection is terminated.

3.3.4 Development notes

Large parts of the implementation are shared between the different classes, to ensure high stability and efficient maintenance.

The synchronous clients are not thread safe nor is a single client intended to be used from multiple threads. Due to the nature of the modbus protocol, it makes little sense to have client calls split over different threads, however the application can do it with proper locking implemented.

The asynchronous client only runs in the thread where the asyncio loop is created, it does not provide mechanisms to prevent (semi)parallel calls, that must be prevented at application level.

3.4 Client device addressing

With **TCP**, **TLS** and **UDP**, the tcp/ip address of the physical device is defined when creating the object. Logical devices represented by the device is addressed with the `device_id=` parameter.

With **Serial**, the comm port is defined when creating the object. The physical devices are addressed with the `device_id=` parameter.

`device_id=0` is defined as broadcast in the modbus standard, but pymodbus treats it as a normal device. please note `device_id=0` can only be used to address devices that truly have id=0 ! Using `device_id=0` to address a single device with id not 0 is against the protocol.

If an application is expecting multiple responses to a broadcast request, it must call `client.execute` and deal with the responses.

If no response is expected to a request, the `no_response_expected=True` argument can be used in the normal API calls, this will cause the call to return immediately with `ExceptionResponse(0xff)`.

3.5 Client response handling

All simple request calls (mixin) return a unified result independent whether it's a read, write or diagnostic call.

The application should evaluate the result generically:

```
try:
    rr = await client.read_coils(1, 1, device_id=1)
except ModbusException as exc:
    _logger.error(f"ERROR: exception in pymodbus {exc}")
    raise exc
if rr.isError():
    _logger.error("ERROR: pymodbus returned an error!")
    raise ModbusException(txt)
```

`except ModbusException as exc:` happens generally when pymodbus experiences an internal error. There are a few situation where an unexpected response from a device can cause an exception.

`rr.isError()` is set whenever the device reports a problem.

And in case of read retrieve the data depending on type of request

- `rr.bits` is set for coils / input_register requests
- `rr.registers` is set for other requests

Remark if using `no_response_expected=True` `rr` will always be `None`.

3.6 Client interface classes

There are a client class for each type of communication and for asynchronous/synchronous

Serial	AsyncModbusSerialClient	ModbusSerialClient
TCP	AsyncModbusTcpClient	ModbusTcpClient
TLS	AsyncModbusTlsClient	ModbusTlsClient
UDP	AsyncModbusUdpClient	ModbusUdpClient

3.6.1 Client common

Some methods are common to all clients:

```
class pymodbus.client.base.ModbusBaseClient(framer: FramerType, retries: int, comm_params:
    CommParams, trace_packet: Callable[[bool, bytes], bytes]
    | None, trace_pdu: Callable[[bool, ModbusPDU],
    ModbusPDU] | None, trace_connect: Callable[[bool],
    None] | None)
```

Bases: [ModbusClientMixin](#)[Awaitable[ModbusPDU]]

ModbusBaseClient.

`ModbusBaseClient` is normally not referenced outside [pymodbus](#).

property connected: bool

Return state of connection.

async connect() → bool

Call transport connect.

register(*custom_response_class*: type[ModbusPDU]) → None

Register a custom response class with the decoder (call **sync**).

Parameters

custom_response_class – (optional) Modbus response class.

Raises

MessageRegisterException – Check exception text.

Use register() to add non-standard responses (like e.g. a login prompt) and have them interpreted automatically.

close() → None

Close connection.

set_max_no_responses(*max_count*: int) → None

Override default max no request responses.

Parameters

max_count – Max aborted requests before disconnecting.

The parameter retries defines how many times a request is retried before being aborted. Once aborted a counter is incremented, and when this counter is greater than max_count the connection is terminated.

 Tip

When a request is successful the count is reset.

```
class pymodbus.client.base.ModbusBaseSyncClient(framer: FramerType, retries: int, comm_params:
    CommParams, trace_packet: Callable[[bool, bytes],
    bytes] | None, trace_pdu: Callable[[bool,
    ModbusPDU], ModbusPDU] | None, trace_connect:
    Callable[[bool], None] | None)
```

Bases: [ModbusClientMixin](#)[ModbusPDU]

ModbusBaseClient.

ModbusBaseClient is normally not referenced outside [pymodbus](#).

register(*custom_response_class*: type[ModbusPDU]) → None

Register a custom response class with the decoder.

Parameters

custom_response_class – (optional) Modbus response class.

Raises

MessageRegisterException – Check exception text.

Use register() to add non-standard responses (like e.g. a login prompt) and have them interpreted automatically.

idle_time() → float

Time before initiating next transaction (call **sync**).

Applications can call message functions without checking `idle_time()`, this is done automatically.

set_max_no_responses(*max_count: int*) → None

Override default max no request responses.

Parameters

max_count – Max aborted requests before disconnecting.

The parameter retries defines how many times a request is retried before being aborted. Once aborted a counter is incremented, and when this counter is greater than `max_count` the connection is terminated.

Tip

When a request is successful the count is reset.

connect() → bool

Connect to other end, overwritten.

close()

Close connection, overwritten.

3.6.2 Client serial

```
class pymodbus.client.AsyncModbusSerialClient(port: str, *(Keyword-only parameters separator (PEP 3102)), framer: Framertype = Framertype.RTU,
baudrate: int = 19200, bytesize: int = 8, parity: str =
'N', stopbits: int = 1, handle_local_echo: bool = False,
name: str = 'comm', reconnect_delay: float = 0.1,
reconnect_delay_max: float = 300, timeout: float = 3,
retries: int = 3, trace_packet: Callable[[bool, bytes],
bytes] | None = None, trace_pdu: Callable[[bool,
ModbusPDU], ModbusPDU] | None = None,
trace_connect: Callable[[bool], None] | None = None))
```

Bases: [ModbusBaseClient](#)

AsyncModbusSerialClient.

Fixed parameters:

Parameters

port – Serial port used for communication.

Optional parameters:

Parameters

- **framer** – Framer name, default `Framertype.RTU`
- **baudrate** – Bits per second.
- **bytesize** – Number of bits per byte 7-8.
- **parity** – 'E'ven, 'O'dd or 'N'one
- **stopbits** – Number of stop bits 1, 1.5, 2.
- **handle_local_echo** – Discard local echo from dongle.

- **name** – Set communication name, used in logging
- **reconnect_delay** – Minimum delay when reconnecting, in seconds (use decimals for milliseconds).
- **reconnect_delay_max** – Maximum delay when reconnecting, in seconds (use decimals for milliseconds).
- **timeout** – Timeout for connecting and receiving data, in seconds (use decimals for milliseconds).
- **retries** – Max number of retries per request.
- **trace_packet** – Called with bytestream received/to be sent
- **trace_pdu** – Called with PDU received/to be sent
- **trace_connect** – Called when connected/disconnected

 Tip

The trace methods allow to modify the datastream/pdu !

 Tip

reconnect_delay doubles automatically with each unsuccessful connect, from **reconnect_delay** to **reconnect_delay_max**. Set *reconnect_delay=0* to avoid automatic reconnection.

Example:

```
from pymodbus.client import AsyncModbusSerialClient

async def run():
    client = AsyncModbusSerialClient("dev/serial0")

    await client.connect()
    ...
    client.close()
```

Please refer to *Pymodbus internals* for advanced usage.

```
class pymodbus.client.ModbusSerialClient(port: str, *, framer: FramerType = FramerType.RTU,
                                         baudrate: int = 19200, bytesize: int = 8, parity: str = 'N',
                                         stopbits: int = 1, handle_local_echo: bool = False, name: str
                                         = 'comm', reconnect_delay: float = 0.1, reconnect_delay_max:
                                         float = 300, timeout: float = 3, retries: int = 3, trace_packet:
                                         Callable[[bool, bytes], bytes] | None = None, trace_pdu:
                                         Callable[[bool, ModbusPDU], ModbusPDU] | None = None,
                                         trace_connect: Callable[[bool], None] | None = None)
```

Bases: *ModbusBaseSyncClient*

ModbusSerialClient.

Fixed parameters:

Parameters

port – Serial port used for communication.

Optional parameters:

Parameters

- **framer** – Framer name, default FramerType.RTU
- **baudrate** – Bits per second.
- **bytesize** – Number of bits per byte 7-8.
- **parity** – ‘E’ven, ‘O’dd or ‘N’one
- **stopbits** – Number of stop bits 0-2.
- **handle_local_echo** – Discard local echo from dongle.
- **name** – Set communication name, used in logging
- **reconnect_delay** – Not used in the sync client
- **reconnect_delay_max** – Not used in the sync client
- **timeout** – Timeout for connecting and receiving data, in seconds.
- **retries** – Max number of retries per request.
- **trace_packet** – Called with bytestream received/to be sent
- **trace_pdu** – Called with PDU received/to be sent
- **trace_connect** – Called when connected/disconnected

Tip

The trace methods allow to modify the datastream/pdu !

Example:

```
from pymodbus.client import ModbusSerialClient

def run():
    client = ModbusSerialClient("dev/serial0")

    client.connect()
    ...
    client.close()
```

Please refer to *Pymodbus internals* for advanced usage.

property connected: bool

Check if socket exists.

connect() → bool

Connect to the modbus serial server.

close()

Close the underlying socket connection.

send(request: bytes, addr: tuple | None = None) → int

Send data on the underlying socket.

recv(size: int | None) → bytes

Read data from the underlying descriptor.

is_socket_open() → bool

Check if socket is open.

3.6.3 Client TCP

```
class pymodbus.client.AsyncModbusTcpClient(host: str, *, framer: FramerType = FramerType.SOCKET,
                                           port: int = 502, name: str = 'comm', source_address:
                                           tuple[str, int] | None = None, reconnect_delay: float = 0.1,
                                           reconnect_delay_max: float = 300, timeout: float = 3,
                                           retries: int = 3, trace_packet: Callable[[bool, bytes], bytes]
                                           | None = None, trace_pdu: Callable[[bool, ModbusPDU],
                                           ModbusPDU] | None = None, trace_connect:
                                           Callable[[bool], None] | None = None)
```

Bases: [*ModbusBaseClient*](#)

AsyncModbusTcpClient.

Fixed parameters:

Parameters

host – Host IP address or host name

Optional parameters:

Parameters

- **framer** – Framer name, default **FramerType**.SOCKET
- **port** – Port used for communication
- **name** – Set communication name, used in logging
- **source_address** – source address of client
- **reconnect_delay** – Minimum delay when reconnecting, in seconds (use decimals for milliseconds).
- **reconnect_delay_max** – Maximum delay when reconnecting, in seconds (use decimals for milliseconds).
- **timeout** – Timeout for connecting and receiving data, in seconds (use decimals for milliseconds).
- **retries** – Max number of retries per request.
- **trace_packet** – Called with bytestream received/to be sent
- **trace_pdu** – Called with PDU received/to be sent
- **trace_connect** – Called when connected/disconnected

 Tip

The trace methods allow to modify the datastream/pdu !

 Tip

reconnect_delay doubles automatically with each unsuccessful connect, from **reconnect_delay** to **reconnect_delay_max**. Set *reconnect_delay=0* to avoid automatic reconnection.

Example:

```
from pymodbus.client import AsyncModbusTcpClient

async def run():
    client = AsyncModbusTcpClient("localhost")

    await client.connect()
    ...
    client.close()
```

Please refer to *PyModbus internals* for advanced usage.

```
class pymodbus.client.ModbusTcpClient(host: str, *, framer: Framertype = Framertype.SOCKET, port: int
    = 502, name: str = 'comm', source_address: tuple[str, int] | None
    = None, reconnect_delay: float = 0.1, reconnect_delay_max: float
    = 300, timeout: float = 3, retries: int = 3, trace_packet:
    Callable[[bool, bytes], bytes] | None = None, trace_pdu:
    Callable[[bool, ModbusPDU], ModbusPDU] | None = None,
    trace_connect: Callable[[bool], None] | None = None)
```

Bases: *ModbusBaseSyncClient*

ModbusTcpClient.

Fixed parameters:

Parameters

host – Host IP address or host name

Optional parameters:

Parameters

- **framer** – Framer name, default Framertype.SOCKET
- **port** – Port used for communication
- **name** – Set communication name, used in logging
- **source_address** – source address of client
- **reconnect_delay** – Not used in the sync client
- **reconnect_delay_max** – Not used in the sync client
- **timeout** – Timeout for connecting and receiving data, in seconds.
- **retries** – Max number of retries per request.
- **trace_packet** – Called with bytestream received/to be sent
- **trace_pdu** – Called with PDU received/to be sent
- **trace_connect** – Called when connected/disconnected

 Tip

The trace methods allow to modify the datastream/pdu !

Example:

```
from pymodbus.client import ModbusTcpClient

async def run():
    client = ModbusTcpClient("localhost")

    client.connect()
    ...
    client.close()
```

Please refer to *Pymodbus internals* for advanced usage.

property connected: bool

Check if socket exists.

connect()

Connect to the modbus tcp server.

close()

Close the underlying socket connection.

send(request, addr: tuple | None = None)

Send data on the underlying socket.

recv(size: int | None) → bytes

Read data from the underlying descriptor.

is_socket_open() → bool

Check if socket is open.

3.6.4 Client TLS

```
class pymodbus.client.AsyncModbusTlsClient(host: str, *, sslctx: ~ssl.SSLContext = <ssl.SSLContext
object>, framer: ~pymodbus.framer.base.FramerType =
FramerType.TLS, port: int = 802, name: str = 'comm',
source_address: tuple[str, int] | None = None,
reconnect_delay: float = 0.1, reconnect_delay_max: float =
300, timeout: float = 3, retries: int = 3, trace_packet:
~collections.abc.Callable[[bool, bytes], bytes] | None =
None, trace_pdu: ~collections.abc.Callable[[bool,
~pymodbus.pdu.pdu.ModbusPDU],
~pymodbus.pdu.pdu.ModbusPDU] | None = None,
trace_connect: ~collections.abc.Callable[[bool], None] |
None = None)
```

Bases: *AsyncModbusTcpClient*

AsyncModbusTlsClient.

Fixed parameters:

Parameters

host – Host IP address or host name

Optional parameters:

Parameters

- **sslctx** – SSLContext to use for TLS
- **framer** – Framer name, default FramerType.TLS
- **port** – Port used for communication
- **name** – Set communication name, used in logging
- **source_address** – Source address of client
- **reconnect_delay** – Minimum delay when reconnecting, in seconds (use decimals for milliseconds).
- **reconnect_delay_max** – Maximum delay when reconnecting, in seconds (use decimals for milliseconds).
- **timeout** – Timeout for connecting and receiving data, in seconds (use decimals for milliseconds).
- **retries** – Max number of retries per request.
- **trace_packet** – Called with bytestream received/to be sent
- **trace_pdu** – Called with PDU received/to be sent
- **trace_connect** – Called when connected/disconnected

 Tip

The trace methods allow to modify the datastream/pdu !

 Tip

reconnect_delay doubles automatically with each unsuccessful connect, from **reconnect_delay** to **reconnect_delay_max**. Set *reconnect_delay=0* to avoid automatic reconnection.

Example:

```
from pymodbus.client import AsyncModbusTlsClient

async def run():
    client = AsyncModbusTlsClient("localhost")

    await client.connect()
    ...
    client.close()
```

Please refer to *Pymodbus internals* for advanced usage.

classmethod generate_ssl(*certfile: str | None = None, keyfile: str | None = None, password: str | None = None*) → SSLContext

Generate sslctx from cert/key/password.

Parameters

- **certfile** – Cert file path for TLS server request
- **keyfile** – Key file path for TLS server request
- **password** – Password for for decrypting private key file

Remark: - MODBUS/TCP Security Protocol Specification demands TLSv2 at least - verify_mode is set to ssl.NONE

```
class pymodbus.client.ModbusTlsClient(host: str, *, sslctx: ~ssl.SSLContext = <ssl.SSLContext object>,
                                     framer: ~pymodbus.framer.base.FramerType = FramerType.TLS,
                                     port: int = 802, name: str = 'comm', source_address: tuple[str, int]
                                     | None = None, reconnect_delay: float = 0.1,
                                     reconnect_delay_max: float = 300, timeout: float = 3, retries: int
                                     = 3, trace_packet: ~collections.abc.Callable[[bool, bytes], bytes] |
                                     None = None, trace_pdu: ~collections.abc.Callable[[bool,
                                     ~pymodbus.pdu.pdu.ModbusPDU],
                                     ~pymodbus.pdu.pdu.ModbusPDU] | None = None, trace_connect:
                                     ~collections.abc.Callable[[bool], None] | None = None)
```

Bases: [ModbusTcpClient](#)

ModbusTlsClient.

Fixed parameters:

Parameters

host – Host IP address or host name

Optional parameters:

Parameters

- **sslctx** – SSLContext to use for TLS
- **framer** – Framer name, default FramerType.TLS
- **port** – Port used for communication
- **name** – Set communication name, used in logging
- **source_address** – Source address of client
- **reconnect_delay** – Not used in the sync client
- **reconnect_delay_max** – Not used in the sync client
- **timeout** – Timeout for connecting and receiving data, in seconds.
- **retries** – Max number of retries per request.
- **trace_packet** – Called with bytestream received/to be sent
- **trace_pdu** – Called with PDU received/to be sent
- **trace_connect** – Called when connected/disconnected

 Tip

The trace methods allow to modify the datastream/pdu !

Example:

```

from pymodbus.client import ModbusTlsClient

async def run():
    client = ModbusTlsClient("localhost")

    client.connect()
    ...
    client.close()

```

Please refer to *Pymodbus internals* for advanced usage.

classmethod generate_ssl(*certfile: str | None = None, keyfile: str | None = None, password: str | None = None*) → SSLContext

Generate sslctx from cert/key/password.

Parameters

- **certfile** – Cert file path for TLS server request
- **keyfile** – Key file path for TLS server request
- **password** – Password for for decrypting private key file

Remark: - MODBUS/TCP Security Protocol Specification demands TLSv2 at least - verify_mode is set to ssl.NONE

property connected: bool

Connect internal.

connect()

Connect to the modbus tls server.

3.6.5 Client UDP

class pymodbus.client.**AsyncModbusUdpClient**(*host: str, *, framer: Framertype = Framertype.SOCKET, port: int = 502, name: str = 'comm', source_address: tuple[str, int] | None = None, reconnect_delay: float = 0.1, reconnect_delay_max: float = 300, timeout: float = 3, retries: int = 3, trace_packet: Callable[[bool, bytes], bytes] | None = None, trace_pdu: Callable[[bool, ModbusPDU], ModbusPDU] | None = None, trace_connect: Callable[[bool], None] | None = None*)

Bases: *ModbusBaseClient*

AsyncModbusUdpClient.

Fixed parameters:

Parameters

host – Host IP address or host name

Optional parameters:

Parameters

- **framer** – Framer name, default Framertype.SOCKET
- **port** – Port used for communication.
- **name** – Set communication name, used in logging

- **source_address** – source address of client,
- **reconnect_delay** – Minimum delay when reconnecting, in seconds (use decimals for milliseconds).
- **reconnect_delay_max** – Maximum delay when reconnecting, in seconds (use decimals for milliseconds).
- **timeout** – Timeout for connecting and receiving data, in seconds (use decimals for milliseconds).
- **retries** – Max number of retries per request.
- **trace_packet** – Called with bytestream received/to be sent
- **trace_pdu** – Called with PDU received/to be sent
- **trace_connect** – Called when connected/disconnected

 Tip

The trace methods allow to modify the datastream/pdu !

 Tip

reconnect_delay doubles automatically with each unsuccessful connect, from **reconnect_delay** to **reconnect_delay_max**. Set *reconnect_delay=0* to avoid automatic reconnection.

Example:

```
from pymodbus.client import AsyncModbusUdpClient

async def run():
    client = AsyncModbusUdpClient("localhost")

    await client.connect()
    ...
    client.close()
```

Please refer to *Pymodbus internals* for advanced usage.

```
class pymodbus.client.ModbusUdpClient(host: str, *, framer: FramerType = FramerType.SOCKET, port: int
    = 502, name: str = 'comm', source_address: tuple[str, int] | None
    = None, reconnect_delay: float = 0.1, reconnect_delay_max: float
    = 300, timeout: float = 3, retries: int = 3, trace_packet:
    Callable[[bool, bytes], bytes] | None = None, trace_pdu:
    Callable[[bool, ModbusPDU], ModbusPDU] | None = None,
    trace_connect: Callable[[bool], None] | None = None)
```

Bases: *ModbusBaseSyncClient*

ModbusUdpClient.

Fixed parameters:

Parameters

host – Host IP address or host name

Optional parameters:

Parameters

- **framer** – Framer name, default FramerType.SOCKET
- **port** – Port used for communication.
- **name** – Set communication name, used in logging
- **source_address** – source address of client,
- **reconnect_delay** – Not used in the sync client
- **reconnect_delay_max** – Not used in the sync client
- **timeout** – Timeout for connecting and receiving data, in seconds.
- **retries** – Max number of retries per request.
- **trace_packet** – Called with bytestream received/to be sent
- **trace_pdu** – Called with PDU received/to be sent
- **trace_connect** – Called when connected/disconnected

Tip

The trace methods allow to modify the datastream/pdu !

Example:

```
from pymodbus.client import ModbusUdpClient

async def run():
    client = ModbusUdpClient("localhost")

    client.connect()
    ...
    client.close()
```

Please refer to *Pymodbus internals* for advanced usage.

property connected: bool

Connect internal.

3.7 Modbus calls

Pymodbus makes all standard modbus requests/responses available as simple calls.

Using `Modbus<transport>Client.register()` custom messages can be added to pymodbus, and handled automatically.

class `pymodbus.client.mixin.ModbusClientMixin`

Bases: `Generic[T]`

ModbusClientMixin.

This is an interface class to facilitate the sending requests/receiving responses like `read_coils`. `execute()` allows to make a call with non-standard or user defined function codes (remember to add a PDU in the transport class to interpret the request/response).

Simple modbus message call:

```
response = client.read_coils(1, 10)
# or
response = await client.read_coils(1, 10)
```

Advanced modbus message call:

```
request = ReadCoilsRequest(1,10)
response = client.execute(False, request)
# or
request = ReadCoilsRequest(1,10)
response = await client.execute(False, request)
```

Tip

All methods can be used directly (synchronous) or with await <method> (asynchronous) depending on the client used.

abstractmethod execute(*no_response_expected: bool, request: ModbusPDU*) → T

Execute request.

read_coils(*address: int, *, count: int = 1, device_id: int = 1, no_response_expected: bool = False*) → T

Read coils (code 0x01).

Parameters

- **address** – Start address to read from
- **count** – (optional) Number of coils to read
- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

reads from 1 to 2000 contiguous in a remote device.

Coils are addressed as 0-N (Note some device manuals uses 1-N, assuming 1==0).

read_discrete_inputs(*address: int, *, count: int = 1, device_id: int = 1, no_response_expected: bool = False*) → T

Read discrete inputs (code 0x02).

Parameters

- **address** – Start address to read from
- **count** – (optional) Number of coils to read
- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

read from 1 to 2000(0x7d0) discrete inputs (bits) in a remote device.

Discrete Inputs are addressed as 0-N (Note some device manuals uses 1-N, assuming 1==0).

read_holding_registers(*address: int, *, count: int = 1, device_id: int = 1, no_response_expected: bool = False*) → T

Read holding registers (code 0x03).

Parameters

- **address** – Start address to read from
- **count** – (optional) Number of registers to read
- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

This function is used to read the contents of a contiguous block of holding registers in a remote device. The Request specifies the starting register address and the number of registers.

Registers are addressed starting at zero. Therefore devices that specify 1-16 are addressed as 0-15.

read_input_registers(*address: int, *, count: int = 1, device_id: int = 1, no_response_expected: bool = False*) → T

Read input registers (code 0x04).

Parameters

- **address** – Start address to read from
- **count** – (optional) Number of registers to read
- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

This function is used to read from 1 to approx. 125 contiguous input registers in a remote device. The Request specifies the starting register address and the number of registers.

Registers are addressed starting at zero. Therefore devices that specify 1-16 are addressed as 0-15.

write_coil(*address: int, value: bool, *, device_id: int = 1, no_response_expected: bool = False*) → T

Write single coil (code 0x05).

Parameters

- **address** – Address to write to
- **value** – Boolean to write
- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

write ON/OFF to a single coil in a remote device.

Coils are addressed as 0-N (Note some device manuals uses 1-N, assuming 1==0).

write_register(*address: int, value: int, *, device_id: int = 1, no_response_expected: bool = False*) → T

Write register (code 0x06).

Parameters

- **address** – Address to write to
- **value** – Value to write
- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

This function is used to write a single holding register in a remote device.

The Request specifies the address of the register to be written.

Registers are addressed starting at zero. Therefore register numbered 1 is addressed as 0.

read_exception_status(**, device_id: int = 1, no_response_expected: bool = False*) → T

Read Exception Status (code 0x07).

Parameters

- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

This function is used to read the contents of eight Exception Status outputs in a remote device.

The function provides a simple method for accessing this information, because the Exception Output references are known (no output reference is needed in the function).

diag_query_data(*msg: bytes, *, device_id: int = 1, no_response_expected: bool = False*) → T

Diagnose query data (code 0x08 sub 0x00).

Parameters

- **msg** – Message to be returned
- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

The data passed in the request data field is to be returned (looped back) in the response. The entire response message should be identical to the request.

diag_restart_communication(*toggle: bool, *, device_id: int = 1, no_response_expected: bool = False*) → T

Diagnose restart communication (code 0x08 sub 0x01).

Parameters

- **toggle** – True if toggled.
- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises***ModbusException*** –

The remote device serial line port must be initialized and restarted, and all of its communications event counters are cleared. If the port is currently in Listen Only Mode, no response is returned. This function is the only one that brings the port out of Listen Only Mode. If the port is not currently in Listen Only Mode, a normal response is returned. This occurs before the restart is `update_datastored`.

diag_read_diagnostic_register(*, *device_id*: int = 1, *no_response_expected*: bool = False) → T

Diagnose read diagnostic register (code 0x08 sub 0x02).

Parameters

- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises***ModbusException*** –

The contents of the remote device's 16-bit diagnostic register are returned in the response.

diag_change_ascii_input_delimiter(*, *delimiter*: int = 10, *device_id*: int = 1, *no_response_expected*: bool = False) → T

Diagnose change ASCII input delimiter (code 0x08 sub 0x03).

Parameters

- **delimiter** – char to replace LF
- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises***ModbusException*** –

The character passed in the request becomes the end of message delimiter for future messages (replacing the default LF character). This function is useful in cases of a Line Feed is not required at the end of ASCII messages.

diag_force_listen_only(*, *device_id*: int = 1, *no_response_expected*: bool = False) → T

Diagnose force listen only (code 0x08 sub 0x04).

Parameters

- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises***ModbusException*** –

Forces the addressed remote device to its Listen Only Mode for MODBUS communications.

This isolates it from the other devices on the network, allowing them to continue communicating without interruption from the addressed remote device. No response is returned.

diag_clear_counters(*, *device_id*: int = 1, *no_response_expected*: bool = False) → T

Diagnose clear counters (code 0x08 sub 0x0A).

Parameters

- **device_id** – (optional) Modbus device ID

- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

Clear ll counters and the diagnostic register. Also, counters are cleared upon power-up

diag_read_bus_message_count(**, device_id: int = 1, no_response_expected: bool = False*) → T

Diagnose read bus message count (code 0x08 sub 0x0B).

Parameters

- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

The response data field returns the quantity of messages that the remote device has detected on the communications systems since its last restart, clear counters operation, or power-up

diag_read_bus_comm_error_count(**, device_id: int = 1, no_response_expected: bool = False*) → T

Diagnose read Bus Communication Error Count (code 0x08 sub 0x0C).

Parameters

- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

The response data field returns the quantity of CRC errors encountered by the remote device since its last restart, clear counter operation, or power-up

diag_read_bus_exception_error_count(**, device_id: int = 1, no_response_expected: bool = False*) → T

Diagnose read Bus Exception Error Count (code 0x08 sub 0x0D).

Parameters

- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

The response data field returns the quantity of modbus exception responses returned by the remote device since its last restart, clear counters operation, or power-up

diag_read_device_message_count(**, device_id: int = 1, no_response_expected: bool = False*) → T

Diagnose read device Message Count (code 0x08 sub 0x0E).

Parameters

- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

The response data field returns the quantity of messages addressed to the remote device, that the remote device has processed since its last restart, clear counters operation, or power-up

diag_read_device_no_response_count(*, *device_id: int = 1, no_response_expected: bool = False*) → T

Diagnose read device No Response Count (code 0x08 sub 0x0F).

Parameters

- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

The response data field returns the quantity of messages addressed to the remote device, that the remote device has processed since its last restart, clear counters operation, or power-up.

diag_read_device_nak_count(*, *device_id: int = 1, no_response_expected: bool = False*) → T

Diagnose read device NAK Count (code 0x08 sub 0x10).

Parameters

- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

The response data field returns the quantity of messages addressed to the remote device for which it returned a Negative ACKNOWLEDGE (NAK) exception response, since its last restart, clear counters operation, or power-up. Exception responses are described and listed in section 7 .

diag_read_device_busy_count(*, *device_id: int = 1, no_response_expected: bool = False*) → T

Diagnose read device Busy Count (code 0x08 sub 0x11).

Parameters

- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

The response data field returns the quantity of messages addressed to the remote device for which it returned device Busy exception response, since its last restart, clear counters operation, or power-up.

diag_read_bus_char_overrun_count(*, *device_id: int = 1, no_response_expected: bool = False*) → T

Diagnose read Bus Character Overrun Count (code 0x08 sub 0x12).

Parameters

- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

The response data field returns the quantity of messages addressed to the remote device that it could not handle due to a character overrun condition, since its last restart, clear counters operation, or power-up. A character overrun is caused by data characters arriving at the port faster than they can be stored, or by the loss of a character due to a hardware malfunction.

diag_read_iop_overnrun_count(*, *device_id*: int = 1, *no_response_expected*: bool = False) → T

Diagnose read Iop overrun count (code 0x08 sub 0x13).

Parameters

- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

An IOP overrun is caused by data characters arriving at the port faster than they can be stored, or by the loss of a character due to a hardware malfunction. This function is specific to the 884.

diag_clear_overnrun_counter(*, *device_id*: int = 1, *no_response_expected*: bool = False) → T

Diagnose Clear Overrun Counter and Flag (code 0x08 sub 0x14).

Parameters

- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

An error flag should be cleared, but nothing else in the specification mentions is, so it is ignored.

diag_getclear_modbus_response(*, *data*: int = 0, *device_id*: int = 1, *no_response_expected*: bool = False) → T

Diagnose Get/Clear modbus plus (code 0x08 sub 0x15).

Parameters

- **data** – “Get Statistics” or “Clear Statistics”
- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

In addition to the Function code (08) and Subfunction code (00 15 hex) in the query, a two-byte Operation field is used to specify either a “Get Statistics” or a “Clear Statistics” operation. The two operations are exclusive - the “Get” operation cannot clear the statistics, and the “Clear” operation does not return statistics prior to clearing them. Statistics are also cleared on power-up of the device,

diag_get_comm_event_counter(*, *device_id*: int = 1, *no_response_expected*: bool = False) → T

Diagnose get event counter (code 0x0B).

Parameters

- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

This function is used to get a status word and an event count from the remote device.

By fetching the current count before and after a series of messages, a client can determine whether the messages were handled normally by the remote device.

The device's event counter is incremented once for each successful message completion. It is not incremented for exception responses, poll commands, or fetch event counter commands.

The event counter can be reset by means of the Diagnostics function Restart Communications or Clear Counters and Diagnostic Register.

diag_get_comm_event_log(**, device_id: int = 1, no_response_expected: bool = False*) → T

Diagnose get event counter (code 0x0C).

Parameters

- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

This function is used to get a status word.

Event count, message count, and a field of event bytes from the remote device.

The status word and event counts are identical to that returned by the Get Communications Event Counter function.

The message counter contains the quantity of messages processed by the remote device since its last restart, clear counters operation, or power-up. This count is identical to that returned by the Diagnostic function Return Bus Message Count.

The event bytes field contains 0-64 bytes, with each byte corresponding to the status of one MODBUS send or receive operation for the remote device. The remote device enters the events into the field in chronological order. Byte 0 is the most recent event. Each new byte flushes the oldest byte from the field.

write_coils(*address: int, values: list[bool], *, device_id: int = 1, no_response_expected: bool = False*) → T

Write coils (code 0x0F).

Parameters

- **address** – Start address to write to
- **values** – List of booleans to write, or a single boolean to write
- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

write ON/OFF to multiple coils in a remote device.

Coils are addressed as 0-N (Note some device manuals uses 1-N, assuming 1==0).

write_registers(*address: int, values: list[int], *, device_id: int = 1, no_response_expected: bool = False*) → T

Write registers (code 0x10).

Parameters

- **address** – Start address to write to
- **values** – List of values to write
- **device_id** – (optional) Modbus device ID

- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

This function is used to write a block of contiguous registers (1 to approx. 120 registers) in a remote device.

report_device_id(*, *device_id: int = 1, no_response_expected: bool = False*) → T

Report device ID (code 0x11).

Parameters

- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

This function is used to read the description of the type, the current status and other information specific to a remote device.

read_file_record(*records: list[FileRecord], *, device_id: int = 1, no_response_expected: bool = False*) → T

Read file record (code 0x14).

Parameters

- **records** – List of FileRecord (Reference type, File number, Record Number)
- **device_id** – device id
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

This function is used to perform a file record read. All request data lengths are provided in terms of number of bytes and all record lengths are provided in terms of registers.

A file is an organization of records. Each file contains 10000 records, addressed 0000 to 9999 decimal or 0x0000 to 0x270f. For example, record 12 is addressed as 12. The function can read multiple groups of references. The groups can be separating (non-contiguous), but the references within each group must be sequential. Each group is defined in a separate “sub-request” field that contains seven bytes:

The reference **type**: 1 byte
 The file number: 2 bytes
 The starting record number within the file: 2 bytes
 The length of the record to be read: 2 bytes

The quantity of registers to be read, combined with all other fields in the expected response, must not exceed the allowable length of the MODBUS PDU: 235 bytes.

write_file_record(*records: list[FileRecord], *, device_id: int = 1, no_response_expected: bool = False*) → T

Write file record (code 0x15).

Parameters

- **records** – List of File_record (Reference type, File number, Record Number, Record Length, Record Data)
- **device_id** – (optional) Device id

- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

This function is used to perform a file record write. All request data lengths are provided in terms of number of bytes and all record lengths are provided in terms of the number of 16 bit words.

mask_write_register(*, *address: int = 0, and_mask: int = 65535, or_mask: int = 0, device_id: int = 1, no_response_expected: bool = False*) → T

Mask write register (code 0x16).

Parameters

- **address** – The mask pointer address (0x0000 to 0xffff)
- **and_mask** – The and bitmask to apply to the register address
- **or_mask** – The or bitmask to apply to the register address
- **device_id** – (optional) device id
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

This function is used to modify the contents of a specified holding register using a combination of an AND mask, an OR mask, and the register's current contents.

The function can be used to set or clear individual bits in the register.

readwrite_registers(*, *read_address: int = 0, read_count: int = 0, write_address: int = 0, address: int | None = None, values: list[int] | None = None, device_id: int = 1, no_response_expected: bool = False*) → T

Read/Write registers (code 0x17).

Parameters

- **read_address** – The address to start reading from
- **read_count** – The number of registers to read from address
- **write_address** – The address to start writing to
- **address** – (optional) use as read/write address
- **values** – List of values to write, or a single value to write
- **device_id** – (optional) Modbus device ID
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises

ModbusException –

This function performs a combination of one read operation and one write operation in a single MODBUS transaction. The write operation is performed before the read.

Holding registers are addressed starting at zero. Therefore holding registers 1-16 are addressed in the PDU as 0-15.

read_fifo_queue(*, *address: int = 0, device_id: int = 1, no_response_expected: bool = False*) → T

Read FIFO queue (code 0x18).

Parameters

- **address** – The address to start reading from
- **device_id** – (optional) device id
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises*ModbusException* –

This function allows to read the contents of a First-In-First-Out (FIFO) queue of register in a remote device. The function returns a count of the registers in the queue, followed by the queued data. Up to 32 registers can be read: the count, plus up to 31 queued data registers.

The queue count register is returned first, followed by the queued data registers. The function reads the queue contents, but does not clear them.

read_device_information(**, read_code: int | None = None, object_id: int = 0, device_id: int = 1, no_response_expected: bool = False*) → T

Read FIFO queue (code 0x2B sub 0x0E).

Parameters

- **read_code** – The device information read code
- **object_id** – The object to read from
- **device_id** – (optional) Device id
- **no_response_expected** – (optional) The client will not expect a response to the request

Raises*ModbusException* –

This function allows reading the identification and additional information relative to the physical and functional description of a remote device, only.

The Read Device Identification interface is modeled as an address space composed of a set of addressable data elements. The data elements are called objects and an object Id identifies them.

class DATATYPE(*value*)

Bases: Enum

Datatype enum (name and internal data), used for convert_* calls.

classmethod convert_from_registers(*registers: list[int], data_type: DATATYPE, word_order: Literal['big', 'little'] = 'big', string_encoding: str = 'utf-8'*) → int | float | str | list[bool] | list[int] | list[float]

Convert registers to int/float/str.

Parameters

- **registers** – list of registers received from e.g. read_holding_registers()
- **data_type** – data type to convert to
- **word_order** – “big”/“little” order of words/registers
- **string_encoding** – The encoding with which to decode the bytearray, only used when data_type=DATATYPE.STRING

Returns

scalar or array of “data_type”

Raises

- **TypeError** – when data_type is not DATATYPE

- ***ModbusException*** – when size of registers is not a multiple of data_type
- ***ParameterException*** – when the specified string encoding is not supported

classmethod **convert_to_registers**(value: int | float | str | list[bool] | list[int] | list[float], data_type: DATATYPE, word_order: Literal['big', 'little'] = 'big', string_encoding: str = 'utf-8') → list[int]

Convert int/float/str to registers (16/32/64 bit).

Parameters

- **value** – value to be converted
- **data_type** – data type to convert from
- **word_order** – “big”/”little” order of words/registers
- **string_encoding** – The encoding with which to encode the bytearray, only used when data_type=DATATYPE.STRING

Returns

List of registers, can be used directly in e.g. write_registers()

Raises

- **TypeError** – when there is a mismatch between data_type and value or data_type is not DATATYPE
- ***ParameterException*** – when the specified string encoding is not supported

SERVER (3.X)

Pymodbus offers servers with transport protocols for

- *Serial* (RS-485) typically using a dongle
- *TCP*
- *TLS*
- *UDP*
- possibility to add a custom transport protocol

communication in 2 versions:

- `synchronous server`,
- `asynchronous server` using `asyncio`.

Remark All servers are implemented with `asyncio`, and the synchronous servers are just an interface layer allowing synchronous applications to use the server as if it was synchronous.

Warning The current framer implementation does not support running the server on a shared rs485 line (multipoint).
Server classes.

```
class pymodbus.server.ModbusBaseServer(params: CommParams, context: ModbusServerContext | None,  
                                       ignore_missing_devices: bool, broadcast_enable: bool, identity:  
                                       ModbusDeviceIdentification | None, framer: Framertype,  
                                       trace_packet: Callable[[bool, bytes], bytes] | None, trace_pdu:  
                                       Callable[[bool, ModbusPDU], ModbusPDU] | None,  
                                       trace_connect: Callable[[bool], None] | None, custom_pdu:  
                                       list[type[ModbusPDU]] | None)
```

Bases: `ModbusProtocol`

Common functionality for all server classes.

active_server: `ModbusBaseServer` | `None` = `None`

callback_connected() → `None`

Call when connection is successful.

callback_data(data: bytes, addr: tuple | `None` = `None`) → `int`

Handle received data.

callback_disconnected(exc: Exception | `None`) → `None`

Call when connection is lost.

callback_new_connection()

Handle incoming connect.

async serve_forever(*, background: bool = False)

Start endless loop.

async shutdown()

Close server.

```
class pymodbus.server.ModbusSerialServer(context: ModbusServerContext, *, framer: FramerType =
    FramerType.RTU, ignore_missing_devices: bool = False,
    identity: ModbusDeviceIdentification | None = None,
    broadcast_enable: bool = False, trace_packet: Callable[[bool,
    bytes], bytes] | None = None, trace_pdu: Callable[[bool,
    ModbusPDU], ModbusPDU] | None = None, trace_connect:
    Callable[[bool], None] | None = None, custom_pdu:
    list[type[ModbusPDU]] | None = None, **kwargs)
```

Bases: [ModbusBaseServer](#)

A modbus threaded serial socket server.

Tip

Remember to call `serve_forever` to start server.

```
class pymodbus.server.ModbusSimulatorServer(modbus_server: str = 'server', modbus_device: str =
    'device', http_host: str = '0.0.0.0', http_port: int = 8080,
    log_file: str = 'server.log', json_file: str = 'setup.json',
    custom_actions_module: str | None = None)
```

Bases: `object`

ModbusSimulatorServer.

Parameters

- **modbus_server** – Server name in json file (default: “server”)
- **modbus_device** – Device name in json file (default: “client”)
- **http_host** – TCP host for HTTP (default: “localhost”)
- **http_port** – TCP port for HTTP (default: 8080)
- **json_file** – setup file (default: “setup.json”)
- **custom_actions_module** – python module with custom actions (default: none)

if either `http_port` or `http_host` is none, HTTP will not be started. This class starts a http server, that serves a couple of endpoints:

- “<addr>/” static files
- “<addr>/api/log” log handling, HTML with GET, REST-API with post
- “<addr>/api/registers” register handling, HTML with GET, REST-API with post
- “<addr>/api/calls” call (function code / message) handling, HTML with GET, REST-API with post
- “<addr>/api/server” server handling, HTML with GET, REST-API with post

Example:

```

from pymodbus.server import ModbusSimulatorServer

async def run():
    simulator = ModbusSimulatorServer(
        modbus_server="my server",
        modbus_device="my device",
        http_host="localhost",
        http_port=8080)
    await simulator.run_forever(only_start=True)
    ...
    await simulator.stop()

```

action_add(*params*, *range_start*, *range_stop*)

Build list of registers matching filter.

action_clear(*_params*, *_range_start*, *_range_stop*)

Clear register filter.

action_monitor(*params*, *range_start*, *range_stop*)

Start monitoring calls.

action_reset(*_params*, *_range_start*, *_range_stop*)

Reset call simulation.

action_set(*params*, *_range_start*, *_range_stop*)

Set register value.

action_simulate(*params*, *_range_start*, *_range_stop*)

Simulate responses.

action_stop(*_params*, *_range_start*, *_range_stop*)

Stop call monitoring.

build_html_calls(*params: dict*, *html: str*) → str

Build html calls page.

build_html_log(*_params*, *html*)

Build html log page.

build_html_registers(*params*, *html*)

Build html registers page.

build_html_server(*_params*, *html*)

Build html server page.

build_json_calls(*params: dict*) → dict

Build json calls response.

build_json_log(*params*)

Build json log page.

build_json_registers(*params*)

Build json registers response.

build_json_server(*params*)

Build html server page.

async handle_html(request)

Handle html.

async handle_html_static(request)

Handle static html.

async handle_json(request)

Handle api registers.

helper_handle_submit(params, submit_actions)

Build html register submit.

async run_forever(only_start=False)

Start modbus and http servers.

async start_modbus_server(app)

Start Modbus server as asyncio task.

async stop()

Stop modbus and http servers.

async stop_modbus_server(app)

Stop modbus server.

class pymodbus.server.ModbusTcpServer(context: [ModbusServerContext](#), *, framer=FramerType.SOCKET, identity: [ModbusDeviceIdentification](#) | None = None, address: tuple[str, int] = ("", 502), ignore_missing_devices: bool = False, broadcast_enable: bool = False, trace_packet: Callable[[bool, bytes], bytes] | None = None, trace_pdu: Callable[[bool, ModbusPDU], ModbusPDU] | None = None, trace_connect: Callable[[bool], None] | None = None, custom_pdu: list[type[ModbusPDU]] | None = None)

Bases: [ModbusBaseServer](#)

A modbus threaded tcp socket server.

Tip

Remember to call `serve_forever` to start server.

class pymodbus.server.ModbusTlsServer(context: [ModbusServerContext](#), *, framer=FramerType.TLS, identity: [ModbusDeviceIdentification](#) | None = None, address: tuple[str, int] = ("", 502), sslctx=None, certfile=None, keyfile=None, password=None, ignore_missing_devices=False, broadcast_enable=False, trace_packet: Callable[[bool, bytes], bytes] | None = None, trace_pdu: Callable[[bool, ModbusPDU], ModbusPDU] | None = None, trace_connect: Callable[[bool], None] | None = None, custom_pdu: list[type[ModbusPDU]] | None = None)

Bases: [ModbusTcpServer](#)

A modbus threaded tls socket server.

 Tip

Remember to call `serve_forever` to start server.

```
class pymodbus.server.ModbusUdpServer(context: ModbusServerContext, *, framer=FramerType.SOCKET,
                                     identity: ModbusDeviceIdentification | None = None, address:
                                     tuple[str, int] = ("", 502), ignore_missing_devices: bool = False,
                                     broadcast_enable: bool = False, trace_packet: Callable[[bool,
                                     bytes], bytes] | None = None, trace_pdu: Callable[[bool,
                                     ModbusPDU], ModbusPDU] | None = None, trace_connect:
                                     Callable[[bool], None] | None = None, custom_pdu:
                                     list[type[ModbusPDU]] | None = None)
```

Bases: [ModbusBaseServer](#)

A modbus threaded udp socket server.

 Tip

Remember to call `serve_forever` to start server.

```
async pymodbus.server.ServerAsyncStop() → None
```

Terminate server.

```
pymodbus.server.ServerStop() → None
```

Terminate server.

```
async pymodbus.server.StartAsyncSerialServer(context: ModbusServerContext, **kwargs) → None
```

Start and run a serial modbus server.

Parameters

- **context** – Datastore object
- **kwargs** – for parameter explanation see [ModbusSerialServer](#)

 Tip

Only handles a single server !

Use [ModbusSerialServer](#) to allow multiple servers in one app.

```
async pymodbus.server.StartAsyncTcpServer(context: ModbusServerContext, **kwargs) → None
```

Start and run a tcp modbus server.

Parameters

- **context** – Datastore object
- **kwargs** – for parameter explanation see [ModbusTcpServer](#)

 Tip

Only handles a single server !

Use ModbusTcpServer to allow multiple servers in one app.

async pymodbus.server.**StartAsyncTlsServer**(context: [ModbusServerContext](#), **kwargs) → None

Start and run a tls modbus server.

Parameters

- **context** – Datastore object
- **kwargs** – for parameter explanation see ModbusTlsServer

 Tip

Only handles a single server !

Use ModbusTlsServer to allow multiple servers in one app.

async pymodbus.server.**StartAsyncUdpServer**(context: [ModbusServerContext](#), **kwargs) → None

Start and run a udp modbus server.

Parameters

- **context** – Datastore object
- **kwargs** – for parameter explanation see ModbusUdpServer

 Tip

Only handles a single server !

Use ModbusUdpServer to allow multiple servers in one app.

pymodbus.server.**StartSerialServer**(context: [ModbusServerContext](#), **kwargs) → None

Start and run a modbus serial server.

Parameters

- **context** – Datastore object
- **kwargs** – for parameter explanation see ModbusSerialServer

 Tip

Only handles a single server !

Use ModbusSerialServer to allow multiple servers in one app.

pymodbus.server.**StartTcpServer**(context: [ModbusServerContext](#), **kwargs) → None

Start and run a modbus TCP server.

Parameters

- **context** – Datastore object
- **kwargs** – for parameter explanation see ModbusTcpServer

 Tip

Only handles a single server !

Use ModbusTcpServer to allow multiple servers in one app.

`pymodbus.server.StartTlsServer(context: ModbusServerContext, **kwargs) → None`

Start and run a modbus TLS server.

Parameters

- **context** – Datastore object
- **kwargs** – for parameter explanation see ModbusTlsServer

 Tip

Only handles a single server !

Use ModbusTlsServer to allow multiple servers in one app.

`pymodbus.server.StartUdpServer(context: ModbusServerContext, **kwargs) → None`

Start and run a modbus UDP server.

Parameters

- **context** – Datastore object
- **kwargs** – for parameter explanation see ModbusUdpServer

 Tip

Only handles a single server !

Use ModbusUdpServer to allow multiple servers in one app.

`pymodbus.server.get_simulator_commandline(cmdline=None)`

Get command line arguments.

PYMODBUS REPL (READ EVALUATE PRINT LOOP)

Pymodbus provides a simple UI to manipulate server/client, this is handled by a separate repo [pymodbus-repl](#)

5.1 Install as pymodbus optional dependency

```
$ pip install ".[repl]"
```

5.2 Install directly from the github repo

```
$ pip install "git+https://github.com/pymodbus-dev/repl"
```


SIMULATOR (3.X)

 Warning

Beginning with v3.9.0 and ending with v4.0.0 this simulator will be replaced by a new version.

The simulator is a full fledged modbus simulator, which is constantly being evolved with user ideas / amendments.

The purpose of the simulator is to provide support for client application test harnesses with end-to-end testing simulating real life modbus devices.

The datastore simulator allows the user to (all automated)

- simulate a modbus device by adding a simple configuration,
- test how a client handles modbus exceptions,
- test a client apps correct use of the simulated device.

The web interface allows the user to (online / manual)

- test how a client handles modbus errors,
- test how a client handles communication errors like divided messages,
- run your test server in the cloud,
- monitor requests/responses,
- inject modbus errors like malicious a response,
- see/Change values online.

The REST API allow the test process to be automated

- spin up a test server with unix domain sockets in your test harness,
- set expected responses with a simple REST API command,
- check the result with another simple REST API command,
- test your client app in a true end-to-end fashion.

6.1 Configuration

Configuring the pymodbus simulator is done with a json file, or if only using the datastore simulator a python dict (same structure as the device part of the json file).

6.1.1 Json file layout

The json file consist of 2 main entries “server_list” (see *Server entries*) and “device_list” (see *Device entries*) each containing a list of servers/devices

```
{
  "server_list": {
    "<name>": { ... },
    ...
  },
  "device_list": {
    "<name>": { ... },
    ...
  }
}
```

You can define as many server and devices as you like, when starting *pymodbus.simulator (v3.x)* you select one server and one device to simulate.

A entry in “device_list” correspond to the dict you can use as parameter to *datastore_simulator* is you want to construct your own simulator.

6.1.2 Server entries

The entries for a tcp server with minimal parameters look like:

```
{
  "server_list": {
    "server": {
      "comm": "tcp",
      "host": "0.0.0.0",
      "port": 5020,
      "framer": "socket",
    }
  }
  "device_list": {
    ...
  }
}
```

The example uses “comm”: “tcp”, so the entries are arguments to *pymodbus.server.ModbusTcpServer*, where detailed information are available.

The entry “comm” allows the following values:

- “serial”, to use *pymodbus.server.ModbusSerialServer*,
- “tcp”, to use *pymodbus.server.ModbusTcpServer*,
- “tls”, to use *pymodbus.server.ModbusTlsServer*,
- “udp”; to use *pymodbus.server.ModbusUdpServer*.

The entry “framer” allows the following values:

- “ascii” to use *pymodbus.framer.FramerAscii*,
- “rtu” to use *pymodbus.framer.FramerRTU*,
- “socket” to use *pymodbus.framer.FramerSocket*.

- “tls” to use `pymodbus.framer.FramerTLS`,

Optional entry “device_id” will limit server to only accept a single id. If not set, the server will accept all device id.

Warning

not all “framer” types can be used with all “comm” types.

e.g. “framer”: “tls” only works with “comm”: “tls”!

6.1.3 Server configuration examples

```
{
  "server_list": {
    "server": {
      "comm": "tcp",
      "host": "0.0.0.0",
      "port": 5020,
      "ignore_missing_devices": false,
      "framer": "socket",
      "identity": {
        "VendorName": "pymodbus",
        "ProductCode": "PM",
        "VendorUrl": "https://github.com/pymodbus-dev/pymodbus",
        "ProductName": "pymodbus Server",
        "ModelName": "pymodbus Server",
        "MajorMinorRevision": "3.1.0"
      }
    },
    "server_try_serial": {
      "comm": "serial",
      "port": "/dev/tty0",
      "stopbits": 1,
      "bytesize": 8,
      "parity": "N",
      "baudrate": 9600,
      "timeout": 3,
      "reconnect_delay": 2,
      "framer": "rtu",
      "identity": {
        "VendorName": "pymodbus",
        "ProductCode": "PM",
        "VendorUrl": "https://github.com/pymodbus-dev/pymodbus",
        "ProductName": "pymodbus Server",
        "ModelName": "pymodbus Server",
        "MajorMinorRevision": "3.1.0"
      }
    },
    "server_try_tls": {
      "comm": "tls",
      "host": "0.0.0.0",
      "port": 5020,
      "certfile": "certificates/pymodbus.crt",
```

(continues on next page)

(continued from previous page)

```

        "keyfile": "certificates/pymodbus.key",
        "ignore_missing_devices": false,
        "framer": "tls",
        "identity": {
            "VendorName": "pymodbus",
            "ProductCode": "PM",
            "VendorUrl": "https://github.com/pymodbus-dev/pymodbus",
            "ProductName": "pymodbus Server",
            "ModelName": "pymodbus Server",
            "MajorMinorRevision": "3.1.0"
        }
    },
    "server_test_try_udp": {
        "comm": "udp",
        "host": "0.0.0.0",
        "port": 5020,
        "ignore_missing_devices": false,
        "framer": "socket",
        "identity": {
            "VendorName": "pymodbus",
            "ProductCode": "PM",
            "VendorUrl": "https://github.com/pymodbus-dev/pymodbus",
            "ProductName": "pymodbus Server",
            "ModelName": "pymodbus Server",
            "MajorMinorRevision": "3.1.0"
        }
    }
}

```

6.1.4 Device entries

Each device is configured in a number of sections, described in detail below

- “setup”, defines the overall structure of the device, like e.g. number of registers,
- “invalid”, defines invalid registers and causes a modbus exception when reading and/or writing,
- “write”, defines registers which allow read/write, other registers causes a modbus exception when writing,
- “bits”, defines registers which contain bits (discrete input and coils),
- “uint16”, defines registers which contain a 16 bit unsigned integer,
- “uint32”, defines sets of registers (2) which contain a 32 bit unsigned integer,
- “float32”, defines sets of registers (2) which contain a 32 bit float,
- “string”, defines sets of registers which contain a string,
- “repeat”, is a special command to copy configuration if a device contains X bay controllers, configure one and use repeat for X-1.

The datastore simulator manages the registers in a big list, which can be manipulated with

- actions (functions that are called with each access)
- manually via the WEB interface

- automated via the REST API interface
- the client (writing values)

It is important to understand that the modbus protocol does not know or care how the physical memory/registers are organized, but it has a huge impact on the client!

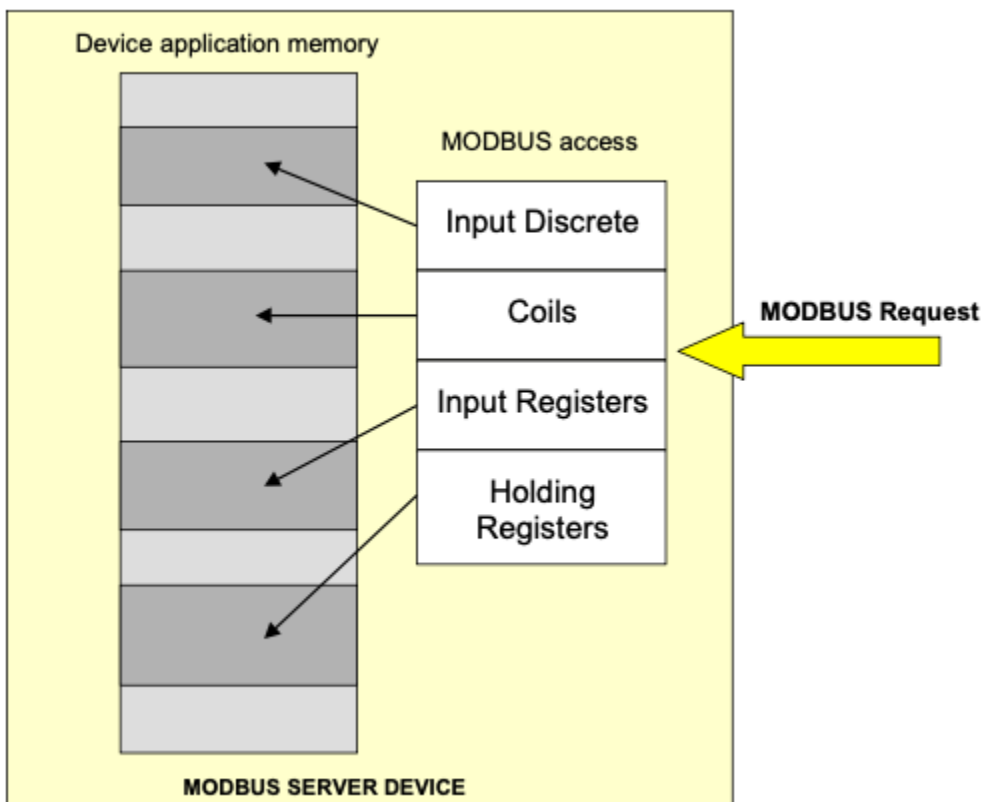
Communication with a modbus device is based on registers which each contain 16 bits (2 bytes). The requests are grouped in 4 groups

- Input Discrete
- Coils
- Input registers
- Holding registers

The 4 blocks are mapped into physical memory, but the modbus protocol makes no assumption or demand on how this is done.

The history of modbus devices have shown 2 forms of mapping.

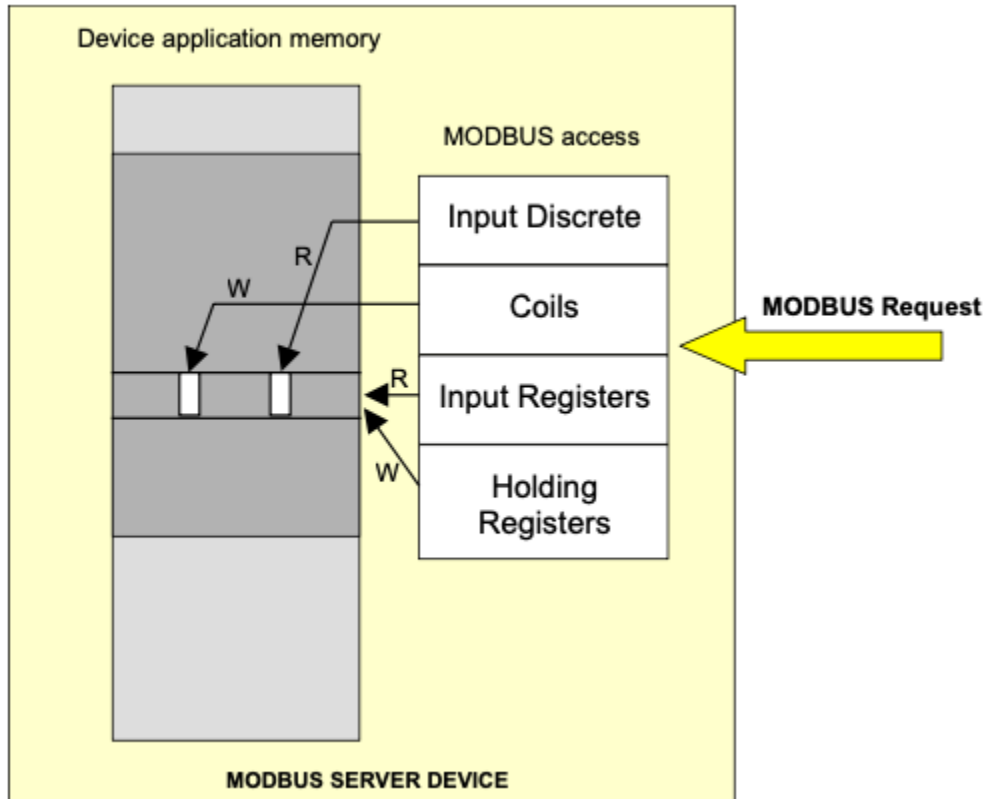
The first form is also the original form. It originates from a time where the devices did not contain memory, but the request was mapped directly to a physical sensor:



When reading holding register 1 (block 4) you get a different register as when reading input register 1 (block 1). Each block references a different physical register memory, in other words the size of the needed memory is the sum of the block sizes.

The second form uses 1 shared block, most modern devices use this form for 2 main reasons:

- the modbus protocol implementation do not connect directly to the sensors but to a shared memory controlled by a small microprocessor.
- designers can group related information independent of type (e.g. a bay controller with register 1 as coil, register 2 as input and register 3 as holding)



When reading holding register 1 the same physical register is accessed as when reading input register 1. Each block references the same physical register memory, in other words the size of the needed memory is the size of the largest block.

The datastore simulator supports both types.

6.1.4.1 Setup section

Example “setup” configuration:

```
"setup": {
  "co size": 10,
  "di size": 20,
  "hr size": 15,
  "ir size": 25,
  "shared blocks": true,
  "type exception": true,
  "defaults": {
    "value": {
      "bits": 0,
      "uint16": 0,
      "uint32": 0,

```

(continues on next page)

(continued from previous page)

```

        "float32": 0.0,
        "string": " "
    },
    "action": {
        "bits": null,
        "uint16": "register",
        "uint32": "register",
        "float32": "register",
        "string": null
    }
}

```

“co size”, “di size”, “hr size”, “ir size”:

Define the size of each block. If using shared block the register list size will be the size of the biggest block (25 registers) If not using shared block the register list size will be the sum of the 4 block sizes (70 registers).

“shared blocks”

Defines if the blocks are independent or shared (true)

 Tip

if shared is set to false, please remember to adjust the addresses, depending on in which group they are.

assuming all sizes are set to 10, the addresses for configuration are as follows:

- coils have addresses 0-9,
- discrete_inputs have addresses 10-19,
- input_registers have addresses 20-29,
- holding_registers have addresses 30-39

when configuring the the datatypes (when calling each block start with 0).

This is needed because the datatypes can be in different blocks.

“type exception”

Defines is the server returns a modbus exception if a read/write request violates the specified type. E.g. Read holding register 10 with count 1, but the 10,11 are defined as UINT32 and thus can only be read with multiples of 2.

This feature is designed to control that a client access the device in the manner it was designed.

“defaults”

Defines how to defines registers not configured or or only partial configured.

“value” defines the default value for each type.

“action” defines the default action for each type. Actions are functions that are called whenever the register is accessed and thus allows automatic manipulation.

The datastore simulator have a number of builtin actions, and allows custom actions to be added:

- **“random”**, change the value with every access,

- “**increment**”, increment the value by 1 with every access,
- “**timestamp**”, uses 6 registers and build a timestamp,
- “**reset**”, causes a reboot of the simulator,
- “**uptime**”, sets the number of seconds the server have been running.

The “**random**” and “**increment**” actions may optionally minimum and/or maximum. In case of “**increment**”, the counter is reset to the minimum value, if the maximum is crossed.

```
{ "addr": 9, "value": 7, "action": "random", "parameters": { "minval": 0, "maxval": 12 } },  
{ "addr": 10, "value": 100, "action": "increment", "parameters": { "minval": 50 } }
```

6.1.4.2 Invalid section

Example “invalid” configuration:

```
"invalid": [  
    5,  
    [10, 15]  
],
```

Defines invalid registers which cannot be read or written. When accessed the response in a modbus exception **invalid address**. In the example registers 5, 10, 11, 12, 13, 14, 15 will produce an exception response.

Registers can be singulars (first entry) or arrays (second entry)

6.1.4.3 Write section

Example “write” configuration:

```
"write": [  
    4,  
    [5, 6]  
],
```

Defines registers which can be written to. When writing to registers not defined here the response is a modbus exception **invalid address**.

Registers can be singulars (first entry) or arrays (second entry)

6.1.4.4 Bits section

Example “bits” configuration:

```
"bits": [  
    5,  
    [6, 7],  
    { "addr": 8, "value": 7 },  
    { "addr": 9, "value": 7, "action": "random" },  
    { "addr": [11, 12], "value": 7, "action": "random" }  
],
```

defines registers which contain bits (discrete input and coils),

Registers can be singulars (first entry) or arrays (second entry), furthermore a value and/or a action can be defined, the value and/or action is inserted into each register defined in “addr”.

6.1.4.5 Uint16 section

Example “uint16” configuration:

```
"uint16": [
    5,
    [6, 7],
    {"addr": 8, "value": 30123},
    {"addr": 9, "value": 712, "action": "increment"},
    {"addr": [11, 12], "value": 517, "action": "random"}
],
```

defines registers which contain a 16 bit unsigned integer,

Registers can be singulars (first entry) or arrays (second entry), furthermore a value and/or a action can be defined, the value and/or action is inserted into each register defined in “addr”.

6.1.4.6 Uint32 section

Example “uint32” configuration:

```
"uint32": [
    [6, 7],
    {"addr": [8, 9], "value": 300123},
    {"addr": [10, 13], "value": 400712, "action": "increment"},
    {"addr": [14, 15], "value": 500517, "action": "random"}
],
```

defines sets of registers (2) which contain a 32 bit unsigned integer,

Registers can only be arrays in multiples of 2, furthermore a value and/or a action can be defined, the value and/or action is converted (high/low value) and inserted into each register set defined in “addr”.

6.1.4.7 Float32 section

Example “float32” configuration:

```
"float32": [
    [6, 7],
    {"addr": [8, 9], "value": 3123.17},
    {"addr": [10, 13], "value": 712.5, "action": "increment"},
    {"addr": [14, 15], "value": 517.0, "action": "random"}
],
```

defines sets of registers (2) which contain a 32 bit float,

Registers can only be arrays in multiples of 2, furthermore a value and/or a action can be defined, the value and/or action is converted (high/low value) and inserted into each register set defined in “addr”.

Remark remember to set "value": <float value> like 512.0 (float) not 512 (integer).

6.1.4.8 String section

Example “string” configuration:

```
"string": [
    7,
```

(continues on next page)

(continued from previous page)

```
[8, 9],
{"addr": [16, 20], "value": "A_B_C_D_E_"}
],
```

defines sets of registers which contain a string,

Registers can be singulars (first entry) or arrays (second entry). Important each string must be defined individually.

- Entry 1 is a string of 2 chars,
- Entry 2 is a string of 4 chars,
- Entry 3 is a string of 10 chars with the value 'A_B_C_D_E_'.

6.1.4.9 Repeat section

Example "repeat" configuration:

```
"repeat": [
  {"addr": [0, 2], "to": [10, 11]},
  {"addr": [0, 2], "to": [10, 15]},
]
```

is a special command to copy configuration if a device contains X bay controllers, configure one and use repeat for X-1.

First entry copies registers 0-2 to 10-11, resulting in 10 == 0, 11 == 1, 12 unchanged.

Second entry copies registers 0-2 to 10-15, resulting in 10 == 0, 11 == 1, 12 == 2, 13 == 0, 14 == 1, 15 == 2, 16 unchanged.

6.1.5 Device configuration examples

```
{
  "server_list": {
    ...
  },
  "device_list": {
    "device": {
      "setup": {
        "co size": 63000,
        "di size": 63000,
        "hr size": 63000,
        "ir size": 63000,
        "shared blocks": true,
        "type exception": true,
        "defaults": {
          "value": {
            "bits": 0,
            "uint16": 0,
            "uint32": 0,
            "float32": 0.0,
            "string": " "
          }
        },
        "action": {
          "bits": null,

```

(continues on next page)

(continued from previous page)

```

        "uint16": "register",
        "uint32": "register",
        "float32": "register",
        "string": null
    }
}
},
"invalid": [
    1
],
"write": [
    5
],
"bits": [
    {"addr": 2, "value": 7}
],
"uint16": [
    {"addr": 3, "value": 17001},
    2100
],
"uint32": [
    {"addr": 4, "value": 617001},
    [3037, 3038]
],
"float32": [
    {"addr": 6, "value": 404.17},
    [4100, 4101]
],
"string": [
    5047,
    {"addr": [16, 20], "value": "A_B_C_D_E_"}
],
"repeat": [
]
},
"device_try": {
    "setup": {
        "co size": 63000,
        "di size": 63000,
        "hr size": 63000,
        "ir size": 63000,
        "shared blocks": true,
        "type exception": true,
        "defaults": {
            "value": {
                "bits": 0,
                "uint16": 0,
                "uint32": 0,
                "float32": 0.0,
                "string": " "
            }
        },
        "action": {

```

(continues on next page)

(continued from previous page)

```

        "bits": null,
        "uint16": "register",
        "uint32": "register",
        "float32": "register",
        "string": null
    }
}
},
"invalid": [
    [0, 5],
    77
],
"write": [
    10,
    [61, 76]
],
"bits": [
    10,
    1009,
    [1116, 1119],
    {"addr": 1144, "value": 1},
    {"addr": [1148, 1149], "value": 32117},
    {"addr": [1208, 1306], "action": "random"}
],
"uint16": [
    11,
    2027,
    [2126, 2129],
    {"addr": 2164, "value": 1},
    {"addr": [2168, 2169], "value": 32117},
    {"addr": [2208, 2306], "action": null}
],
"uint32": [
    12,
    3037,
    [3136, 3139],
    {"addr": 3174, "value": 1},
    {"addr": [3188, 3189], "value": 32514},
    {"addr": [3308, 3406], "action": null},
    {"addr": [3688, 3878], "value": 115, "action": "increment"}
],
"float32": [
    14,
    4047,
    [4146, 4149],
    {"addr": 4184, "value": 1},
    {"addr": [4198, 4191], "value": 32514.1},
    {"addr": [4308, 4406], "action": null},
    {"addr": [4688, 4878], "value": 115.7, "action": "increment"}
],
"string": [
    {"addr": [16, 20], "value": "A_B_C_D_E_"},

```

(continues on next page)

(continued from previous page)

```

        5047,
        [5146, 5149],
        {"addr": [529, 544], "value": "Brand name, 32 bytes.....X"}
    ],
    "repeat": [
        {"addr": [0, 999], "to": [10000, 10999]},
        {"addr": [10, 1999], "to": [11000, 11999]}
    ]
}
},
"device_minimum": {
    "setup": {
        "co size": 10,
        "di size": 10,
        "hr size": 10,
        "ir size": 10,
        "shared blocks": true,
        "type exception": false,
        "defaults": {
            "value": {
                "bits": 0,
                "uint16": 0,
                "uint32": 0,
                "float32": 0.0,
                "string": " "
            },
            "action": {
                "bits": null,
                "uint16": null,
                "uint32": null,
                "float32": null,
                "string": null
            }
        }
    },
    "invalid": [],
    "write": [],
    "bits": [],
    "uint16": [
        [0, 9]
    ],
    "uint32": [],
    "float32": [],
    "string": [],
    "repeat": []
}
}
}

```

6.1.6 Configuration used for test

```
{
  "server_list": {
    "server": {
      "comm": "tcp",
      "host": "0.0.0.0",
      "port": 5020,
      "ignore_missing_devices": false,
      "framer": "socket",
      "identity": {
        "VendorName": "pymodbus",
        "ProductCode": "PM",
        "VendorUrl": "https://github.com/pymodbus-dev/pymodbus/",
        "ProductName": "pymodbus Server",
        "ModelName": "pymodbus Server",
        "MajorMinorRevision": "3.1.0"
      }
    },
    "server_try_serial": {
      "comm": "serial",
      "port": "/dev/tty0",
      "stopbits": 1,
      "bytesize": 8,
      "parity": "N",
      "baudrate": 9600,
      "timeout": 3,
      "reconnect_delay": 2,
      "framer": "rtu",
      "identity": {
        "VendorName": "pymodbus",
        "ProductCode": "PM",
        "VendorUrl": "https://github.com/pymodbus-dev/pymodbus/",
        "ProductName": "pymodbus Server",
        "ModelName": "pymodbus Server",
        "MajorMinorRevision": "3.1.0"
      }
    },
    "server_try_tls": {
      "comm": "tls",
      "host": "0.0.0.0",
      "port": 5020,
      "certfile": "certificates/pymodbus.crt",
      "keyfile": "certificates/pymodbus.key",
      "ignore_missing_devices": false,
      "framer": "tls",
      "identity": {
        "VendorName": "pymodbus",
        "ProductCode": "PM",
        "VendorUrl": "https://github.com/pymodbus-dev/pymodbus/",
        "ProductName": "pymodbus Server",
        "ModelName": "pymodbus Server",
        "MajorMinorRevision": "3.1.0"
      }
    }
  }
}
```

(continues on next page)

(continued from previous page)

```

    }
  },
  "server_test_try_udp": {
    "comm": "udp",
    "host": "0.0.0.0",
    "port": 5020,
    "ignore_missing_devices": false,
    "framer": "socket",
    "identity": {
      "VendorName": "pymodbus",
      "ProductCode": "PM",
      "VendorUrl": "https://github.com/pymodbus-dev/pymodbus/",
      "ProductName": "pymodbus Server",
      "ModelName": "pymodbus Server",
      "MajorMinorRevision": "3.1.0"
    }
  }
},
"device_list": {
  "device": {
    "setup": {
      "co size": 63000,
      "di size": 63000,
      "hr size": 63000,
      "ir size": 63000,
      "shared blocks": true,
      "type exception": true,
      "defaults": {
        "value": {
          "bits": 0,
          "uint16": 0,
          "uint32": 0,
          "float32": 0.0,
          "string": " "
        },
        "action": {
          "bits": null,
          "uint16": "increment",
          "uint32": "increment",
          "float32": "increment",
          "string": null
        }
      }
    },
    "invalid": [
      1
    ],
    "write": [
      3
    ],
    "bits": [
      {"addr": 2, "value": 7}
    ]
  }
}

```

(continues on next page)

(continued from previous page)

```

    ],
    "uint16": [
        {"addr": 3, "value": 17001, "action": null},
        2100
    ],
    "uint32": [
        {"addr": [4, 5], "value": 617001, "action": null},
        [3037, 3038]
    ],
    "float32": [
        {"addr": [6, 7], "value": 404.17},
        [4100, 4101]
    ],
    "string": [
        5047,
        {"addr": [16, 20], "value": "A_B_C_D_E_"}
    ],
    "repeat": [
    ],
},
"device_try": {
    "setup": {
        "co size": 63000,
        "di size": 63000,
        "hr size": 63000,
        "ir size": 63000,
        "shared blocks": true,
        "type exception": true,
        "defaults": {
            "value": {
                "bits": 0,
                "uint16": 0,
                "uint32": 0,
                "float32": 0.0,
                "string": " "
            },
            "action": {
                "bits": null,
                "uint16": null,
                "uint32": null,
                "float32": null,
                "string": null
            }
        }
    },
    "invalid": [
        [0, 5],
        77
    ],
    "write": [
        10
    ],

```

(continues on next page)

(continued from previous page)

```

    "bits": [
        10,
        1009,
        [1116, 1119],
        {"addr": 1144, "value": 1},
        {"addr": [1148, 1149], "value": 32117},
        {"addr": [1208, 1306], "action": "random"}
    ],
    "uint16": [
        11,
        2027,
        [2126, 2129],
        {"addr": 2164, "value": 1},
        {"addr": [2168, 2169], "value": 32117},
        {"addr": [2208, 2304], "action": "increment"},
        {"addr": 2305,
            "value": 50,
            "action": "increment",
            "parameters": {"minval": 45, "maxval": 155}
        },
        {"addr": 2306,
            "value": 50,
            "action": "random",
            "parameters": {"minval": 45, "maxval": 55}
        }
    ],
    "uint32": [
        [12, 13],
        [3037, 3038],
        [3136, 3139],
        {"addr": [3174, 3175], "value": 1},
        {"addr": [3188, 3189], "value": 32514},
        {"addr": [3308, 3407], "action": null},
        {"addr": [3688, 3875], "value": 115, "action": "increment"},
        {"addr": [3876, 3877],
            "value": 50000,
            "action": "increment",
            "parameters": {"minval": 45000, "maxval": 55000}
        },
        {"addr": [3878, 3879],
            "value": 50000,
            "action": "random",
            "parameters": {"minval": 45000, "maxval": 55000}
        }
    ],
    "float32": [
        [14, 15],
        [4047, 4048],
        [4146, 4149],
        {"addr": [4184, 4185], "value": 1},
        {"addr": [4188, 4191], "value": 32514.2},
        {"addr": [4308, 4407], "action": null},

```

(continues on next page)

(continued from previous page)

```

        {"addr": [4688, 4875], "value": 115.7, "action": "increment"},
        {"addr": [4876, 4877],
         "value": 50000.0,
         "action": "increment",
         "parameters": {"minval": 45000.0, "maxval": 55000.0}
        },
        {"addr": [4878, 48779],
         "value": 50000.0,
         "action": "random",
         "parameters": {"minval": 45000.0, "maxval": 55000.0}
        }
    ],
    "string": [
        {"addr": [16, 20], "value": "A_B_C_D_E_"},
        {"addr": [529, 544], "value": "Brand name, 32 bytes.....X"}
    ],
    "repeat": [
    ]
}
}
}

```

6.2 Simulator datastore

The simulator datastore is an advanced datastore. The simulator allows to simulate the registers of a real life modbus device by adding a simple dict (definition see *Device entries*).

The simulator datastore allows to add actions (functions) to a register, and thus allows a low level automation.

Documentation [pymodbus.datastore.ModbusSimulatorContext](#)

6.3 Web frontend v3.x

TO BE DOCUMENTED.

6.3.1 pymodbus.simulator (v3.x)

The easiest way to run the simulator with web is to use “pymodbus.simulator” from the commandline.

TO BE DOCUMENTED. HTTP server for modbus simulator.

class pymodbus.server.simulator.http_server.**CallTracer**(*call: bool = False, fc: int = -1, address: int = -1, count: int = -1, data: bytes = b"*)

Bases: object

Define call/response traces.

class pymodbus.server.simulator.http_server.**CallTypeMonitor**(*active: bool = False, trace_response: bool = False, range_start: int = -1, range_stop: int = -1, function: int = -1, hex: bool = False, decode: bool = False*)

Bases: object

Define Request/Response monitor.

```
class pymodbus.server.simulator.http_server.CallTypeResponse(active: int = -1, split: int = 0, delay:
    int = 0, junk_len: int = 10,
    error_response: int = 0,
    change_rate: int = 0, clear_after: int
    = 1)
```

Bases: object

Define Response manipulation.

```
class pymodbus.server.simulator.http_server.ModbusSimulatorServer(modbus_server: str = 'server',
    modbus_device: str = 'device',
    http_host: str = '0.0.0.0',
    http_port: int = 8080,
    log_file: str = 'server.log',
    json_file: str = 'setup.json',
    custom_actions_module: str |
    None = None)
```

Bases: object

ModbusSimulatorServer.

Parameters

- **modbus_server** – Server name in json file (default: “server”)
- **modbus_device** – Device name in json file (default: “client”)
- **http_host** – TCP host for HTTP (default: “localhost”)
- **http_port** – TCP port for HTTP (default: 8080)
- **json_file** – setup file (default: “setup.json”)
- **custom_actions_module** – python module with custom actions (default: none)

if either http_port or http_host is none, HTTP will not be started. This class starts a http server, that serves a couple of endpoints:

- “<addr>/” static files
- “<addr>/api/log” log handling, HTML with GET, REST-API with post
- “<addr>/api/registers” register handling, HTML with GET, REST-API with post
- “<addr>/api/calls” call (function code / message) handling, HTML with GET, REST-API with post
- “<addr>/api/server” server handling, HTML with GET, REST-API with post

Example:

```
from pymodbus.server import ModbusSimulatorServer

async def run():
    simulator = ModbusSimulatorServer(
        modbus_server="my server",
        modbus_device="my device",
        http_host="localhost",
        http_port=8080)
    await simulator.run_forever(only_start=True)
```

(continues on next page)

(continued from previous page)

```
...
await simulator.stop()
```

async start_modbus_server(*app*)
Start Modbus server as asyncio task.

async stop_modbus_server(*app*)
Stop modbus server.

async run_forever(*only_start=False*)
Start modbus and http servers.

async stop()
Stop modbus and http servers.

async handle_html_static(*request*)
Handle static html.

async handle_html(*request*)
Handle html.

async handle_json(*request*)
Handle api registers.

build_html_registers(*params, html*)
Build html registers page.

build_html_calls(*params: dict, html: str*) → str
Build html calls page.

build_html_log(*_params, html*)
Build html log page.

build_html_server(*_params, html*)
Build html server page.

build_json_registers(*params*)
Build json registers response.

build_json_calls(*params: dict*) → dict
Build json calls response.

build_json_log(*params*)
Build json log page.

build_json_server(*params*)
Build html server page.

helper_handle_submit(*params, submit_actions*)
Build html register submit.

action_clear(*_params, _range_start, _range_stop*)
Clear register filter.

action_stop(*_params, _range_start, _range_stop*)
Stop call monitoring.

action_reset(*_params*, *_range_start*, *_range_stop*)

Reset call simulation.

action_add(*params*, *range_start*, *range_stop*)

Build list of registers matching filter.

action_monitor(*params*, *range_start*, *range_stop*)

Start monitoring calls.

action_set(*params*, *_range_start*, *_range_stop*)

Set register value.

action_simulate(*params*, *_range_start*, *_range_stop*)

Simulate responses.

6.4 Pymodbus simulator ReST API

This is still a Work In Progress. There may be large changes to the API in the future.

The API is a simple copy of having most of the same features as in the Web UI.

The API provides the following endpoints:

- /restapi/registers
- /restapi/calls
- /restapi/server
- /restapi/log

6.4.1 Registers Endpoint

6.4.1.1 /restapi/registers

The registers endpoint is used to read and write registers.

Request Parameters

- **submit (string, required):**
The action to perform. Must be one of *Register*, *Set*.
- **range_start (integer, optional):**
The starting register to read from. Defaults to 0.
- **range_end (integer, optional):**
The ending register to read from. Defaults to *range_start*.

Response Parameters

Returns a json object with the following keys:

- **result (string):**
The result of the action. Either *ok* or *error*.
- **error (string, conditional):**
The error message if the result is *error*.
- **register_rows (list):**
A list of objects containing the data of the registers.
- **footer (string):**
A cleartext status of the action. HTML leftover.

- **register_types (list):**
A static list of register types. HTML leftover.
- **register_actions (list):**
A static list of register actions. HTML leftover.

Example Request and Response

Request Example:

```
{
  "range_start": 16,
  "range_end": 16,
  "submit": "Register"
}
```

Response Example:

```
{
  "result": "ok",
  "footer": "Operation completed successfully",
  "register_types": {
    "bits": 1,
    "uint16": 2,
    "uint32": 3,
    "float32": 4,
    "string": 5,
    "next": 6,
    "invalid": 0
  },
  "register_actions": {
    "null": 0,
    "increment": 1,
    "random": 2,
    "reset": 3,
    "timestamp": 4,
    "uptime": 5
  },
  "register_rows": [
    {
      "index": "16",
      "type": "uint16",
      "access": "True",
      "action": "none",
      "value": "3124",
      "count_read": "0",
      "count_write": "0"
    }
  ]
}
```

6.4.2 Calls Endpoint

The calls endpoint is used to handle ModBus response manipulation.

6.4.2.1 /restapi/calls

The calls endpoint is used to simulate different conditions for ModBus responses.

Request Parameters

- **submit (string, required):**
The action to perform. Must be one of *Simulate*, *Reset*.

The following must be present if *submit* is *Simulate*:

- **response_clear_after (integer, required):**
The number of packet to clear simulation after.
- **response_cr (string, required):**
Must be present but can be any value. Turns on change rate simulation (WIP).
- **response_cr_pct (integer, required):**
The percentage of change rate, how many percent of packets should be changed.
- **response_split (string, required):**
Must be present but can be any value. Turns on split response simulation (WIP).
- **split_delay (integer, required):**
The delay in seconds to wait before sending the second part of the split response.
- **response_delay (integer, required):**
The delay in seconds to wait before sending the response.
- **response_error (integer, required):**
The error code to send in the response. The valid values can be one from the response *function_error* list.

When *submit* is *Reset*, no other parameters are required. It resets all simulation options to their defaults (off).

Example Request and Response

Request:

```
{
  "submit": "Simulate"
  "response_clear_after": 0,
  "response_cr": "",
  "response_cr_pct": 0,
  "response_split": "",
  "split_delay": 1
  "response_delay": 0,
  "response_error": 0,
  "response_junk_data_len": 0,
  "response_type": 0,
}
```

Response:

Unfortunately, the endpoint response contains extra clutter due to not being finalized.

```

{
  "simulation_action": "ACTIVE",
  "range_start": null,
  "range_stop": null,
  "function_codes": [
    {
      "value": 3,
      "text": "read_holding_registers",
      "selected": false
    },
    {
      "value": 2,
      "text": "read_discrete_input",
      "selected": false
    },
    {
      "value": 4,
      "text": "read_input_registers",
      "selected": false
    },
    {
      "value": 1,
      "text": "read_coils",
      "selected": false
    },
    {
      "value": 15,
      "text": "write_coils",
      "selected": false
    },
    {
      "value": 16,
      "text": "write_registers",
      "selected": false
    },
    {
      "value": 6,
      "text": "write_register",
      "selected": false
    },
    {
      "value": 5,
      "text": "write_coil",
      "selected": false
    },
    {
      "value": 23,
      "text": "read_write_multiple_registers",
      "selected": false
    },
    {
      "value": 8,
      "text": "diagnostic_status",

```

(continues on next page)

(continued from previous page)

```

        "selected": false
    },
    {
        "value": 7,
        "text": "read_exception_status",
        "selected": false
    },
    {
        "value": 11,
        "text": "get_event_counter",
        "selected": false
    },
    {
        "value": 12,
        "text": "get_event_log",
        "selected": false
    },
    {
        "value": 17,
        "text": "report_device_id",
        "selected": false
    },
    {
        "value": 20,
        "text": "read_file_record",
        "selected": false
    },
    {
        "value": 21,
        "text": "write_file_record",
        "selected": false
    },
    {
        "value": 22,
        "text": "mask_write_register",
        "selected": false
    },
    {
        "value": 24,
        "text": "read_fifo_queue",
        "selected": false
    },
    {
        "value": 43,
        "text": "read_device_information",
        "selected": false
    }
    ],
    "function_show_hex_checked": false,
    "function_show_decoded_checked": false,
    "function_response_normal_checked": true,
    "function_response_error_checked": false,

```

(continues on next page)

(continued from previous page)

```
"function_response_empty_checked": false,
"function_response_junk_checked": false,
"function_response_split_checked": true,
"function_response_split_delay": 1,
"function_response_cr_checked": false,
"function_response_cr_pct": 0,
"function_response_delay": 0,
"function_response_junk": 0,
"function_error": [
    {
        "value": 1,
        "text": "ILLEGAL_FUNCTION",
        "selected": false
    },
    {
        "value": 2,
        "text": "ILLEGAL_ADDRESS",
        "selected": false
    },
    {
        "value": 3,
        "text": "ILLEGAL_VALUE",
        "selected": false
    },
    {
        "value": 4,
        "text": "DEVICE_FAILURE",
        "selected": false
    },
    {
        "value": 5,
        "text": "ACKNOWLEDGE",
        "selected": false
    },
    {
        "value": 6,
        "text": "DEVICE_BUSY",
        "selected": false
    },
    {
        "value": 7,
        "text": "MEMORY_PARITY_ERROR",
        "selected": false
    },
    {
        "value": 10,
        "text": "GATEWAY_PATH_UNAVIABLE",
        "selected": false
    },
    {
        "value": 11,
        "text": "GATEWAY_NO_RESPONSE",
```

(continues on next page)

(continued from previous page)

```
        "selected": false
    },
    ],
    "function_response_clear_after": 1,
    "call_rows": [],
    "foot": "not active",
    "result": "ok"
}
```

6.4.3 Server Endpoint

The server endpoint has not yet been implemented.

6.4.4 Log Endpoint

The log endpoint has not yet been implemented.

SIMULATOR

WORK IN PROGRESS, do NOT use

The simulator is a full fledged modbus server/simulator.

The purpose of the simulator is to provide support for client application test harnesses with end-to-end testing simulating real life modbus devices.

The simulator allows the user to (all automated):

- simulate a modbus device by adding a simple configuration,
- simulate a multipoint line, but adding multiple device configurations,
- simulate devices that are not conforming to the protocol,
- simulate communication problems (data loss etc),
- test how a client handles modbus response and exceptions,
- test a client apps correct use of the simulated device.

For details please see:

- *Data model configuration*
- *Simulator server*

The web interface (activated optionally) allows the user to:

- introduce modbus errors (like e.g. wrong length),
- introduce communication errors (like splitting a message),
- monitor requests/responses,
- see/Change values online.
- inject modbus errors like malicious a response,
- run your test server in the cloud,

For details please see:

- *Web frontend*

The REST API allow the test process to be automated

- spin up a test server in your test harness,
- set expected responses with a simple REST API command,
- check the result with a simple REST API command,
- test your client app in a true end-to-end fashion.

The web server uses the REST API internally, which helps to ensure that it actually works.

For details please see:

- [*REST API*](#)

SERVER/SIMULATOR

WORK IN PROGRESS, do NOT use

The simulator is a full fledged modbus server/simulator.

The purpose of the simulator is to provide support for client application test harnesses with end-to-end testing simulating real life modbus devices.

The simulator allows the user to (all automated):

- simulate a modbus device by adding a simple configuration,
- simulate a multipoint line, but adding multiple device configurations,
- simulate devices that are not conforming to the protocol,
- simulate communication problems (data loss etc),
- test how a client handles modbus response and exceptions,
- test a client apps correct use of the simulated device.

The web interface (activated optionally) allows the user to:

- introduce modbus errors (like e.g. wrong length),
- introduce communication errors (like splitting a message),
- monitor requests/responses,
- see/Change values online.
- inject modbus errors like malicious a response,
- run your test server in the cloud,

The REST API allow the test process to be automated

- spin up a test server in your test harness,
- set expected responses with a simple REST API command,
- check the result with a simple REST API command,
- test your client app in a true end-to-end fashion.

The web server uses the REST API internally, which helps to ensure that it actually works.

8.1 Data model configuration

The simulator data model represent the registers and parameters of the simulated devices. The data model is defined using `SimData` and `SimDevice` before starting the server and cannot be changed without restarting the server.

`SimData` defines a group of continuous identical registers. This is the basis of the model, multiple `SimData` are used to mirror the physical device.

`SimDevice` defines device parameters and a list of `SimData`. The list of `SimData` can be added as shared registers or as 4 separate blocks as defined in modbus. `SimDevice` are used to simulate a single device, while a list of `SimDevice` simulates a multipoint line (rs485 line) or a serial forwarder.

A server consist of communication parameters and a list of `SimDevice`

8.1.1 Usage examples

```
#!/usr/bin/env python3
"""Pymodbus server datamodel examples.

This file shows examples of how to configure the datamodel for the server/simulator.

There are different examples showing the flexibility of the datamodel.
"""

from pymodbus.constants import DataType
from pymodbus.simulator import SimData, SimDevice

def define_datamodel():
    """Define register groups.

    Coils and direct inputs are modeled as bits representing a relay in the device.
    There are no real difference between coils and direct inputs, but historically
    they have been divided. Please be aware the coils and direct inputs are addressed
    differently
    in shared vs non-shared models.
    - In a non-shared model the address is the bit directly.
      It can be thought of as if 1 register == 1 bit.
    - In a shared model the address is the register containing the bits.
      1 register == 16bit, so a single bit CANNOT be addressed directly.

    Holding registers and input registers are modeled as int/float/string representing a
    sensor in the device.
    There are no real difference between holding registers and input registers, but
    historically they have
    been divided.
    Please be aware that 1 sensor might be modeled as several register because it needs
    more than
    16 bit for accuracy (e.g. a INT32).
    """
    # SimData can be instantiated with positional or optional parameters:
    assert SimData(
        5, 10, 17, DataType.REGISTERS
    ) == SimData(
```

(continues on next page)

(continued from previous page)

```

        address=5, values=17, count=10, datatype=DataType.REGISTERS
    )

    # Define a group of coils/direct inputs non-shared (address=15..31 each 1 bit)
    #block1 = SimData(address=15, count=16, values=True, datatype=DataType.BITS)
    # Define a group of coils/direct inputs shared (address=15..31 each 16 bit)
    #block2 = SimData(address=15, count=16, values=0xFFFF, datatype=DataType.BITS)

    # Define a group of holding/input registers (remark NO difference between shared and
    ↪non-shared)
    #block3 = SimData(10, 1, 123.4, datatype=DataType.FLOAT32)
    #block4 = SimData(17, count=5, values=123, datatype=DataType.INT64)
    block5 = SimData(27, 1, "Hello ", datatype=DataType.STRING)

    block_def = SimData(0, count=1000, datatype=DataType.REGISTERS, default=True)

    # SimDevice can be instantiated with positional or optional parameters:
    assert SimDevice(
        5,
        [block_def, block5],
    ) == SimDevice(
        id=5, type_check=False, registers=[block_def, block5]
    )

    # SimDevice can define either a shared or a non-shared register model
    SimDevice(id=1, type_check=False, registers=[block_def, block5])
    #SimDevice(2, False,
    #    block_coil=[block1],
    #    block_direct=[block1],
    #    block_holding=[block2],
    #    block_input=[block3, block4])
    # Remark: it is legal to reuse SimData, the object is only used for configuration,
    # not for runtime.

    # id=0 in a SimDevice act as a "catch all". Requests to an unknown id is executed in
    ↪this SimDevice.
    #SimDevice(0, block_shared=[block2])

def main():
    """Combine setup and run."""
    define_datamodel()

if __name__ == "__main__":
    main()

```

8.1.2 Class definitions

class pymodbus.constants.**DataType**(value)

Register types, used to define of a group of registers.

This is the types pymodbus recognizes, actually the modbus standard do NOT define e.g. INT32, but since nearly every device contain e.g. values of type INT32, it is available in pymodbus, with automatic conversions to/from

registers.

INT16 = 1

1 integer == 1 register

UINT16 = 2

1 positive integer == 1 register

INT32 = 3

1 integer == 2 registers

UINT32 = 4

1 positive integer == 2 registers

INT64 = 5

1 integer == 4 registers

UINT64 = 6

1 positive integer == 4 register

FLOAT32 = 7

1 float == 2 registers

FLOAT64 = 8

1 float == 4 registers

STRING = 9

1 string == (len(string) / 2) registers

BITS = 10

16 bits == 1 register

REGISTERS = 11

Registers == 2 bytes (identical to UINT16)

```
class pymodbus.simulator.SimData(address: int, count: int = 1, values: int | float | str | bytes | list[int | float |
    str | bytes | bool] = 0, datatype: DataType = DataType.REGISTERS,
    action: Callable[[int, int, list[int]], Awaitable[list[int] |
    ExceptionResponse]] | None = None, readonly: bool = False, invalid:
    bool = False, default: bool = False, register_count: int = -1, type_size:
    int = -1)
```

Bases: object

Configure a group of continuous identical values/registers.

Examples:

```
SimData(
    address=100,
    count=5,
    value=12345678
    datatype=DataType.INT32
)
SimData(
    address=100,
    value=[1, 2, 3, 4, 5]
    datatype=DataType.INT32
)
```


Each SimData defines 5 INT32 in total 10 registers (address 100-109)

```
SimData(
    address=100,
    count=17,
    value=True
    datatype=DataType.BITS
)
SimData(
    address=100,
    value=[0xffff, 1]
    datatype=DataType.BITS
)
```

Each SimData defines 17 BITS (coils), with value True.

In block mode (CO and DI) addresses are 100-116 (each 1 bit)

In shared mode BITS are stored in registers (16bit is 1 register), the address refer to the register, addresses are 100-101 (with register 101 being padded with 15 bits set to False)

```
SimData(
    address=0,
    count=1000,
    value=0x1234
    datatype=DataType.REGISTERS
)
```

Defines a range of addresses 0..999 each with the value 0x1234.

address: int

Address of first register, starting with 0 (identical to the requests)

count: int = 1

Count of datatype e.g.

- count=3 datatype=DataType.REGISTERS is 3 registers.
- count=3 datatype=DataType.INT32 is 6 registers.
- count=1 (default), value="ABCD" is 2 registers

Cannot be used if value is a list or datatype is DataType.STRING

values: int | float | str | bytes | list[int | float | str | bytes | bool] = 0

Value/Values of datatype, will automatically be converted to registers, according to datatype.

datatype: *DataType* = 11

Used to check access and convert value to/from registers.

action: Callable[[int, int, list[int]], Awaitable[list[int] | *ExceptionResponse*]] | None = None

 Tip

use functools.partial to add extra parameters if needed.

readonly: `bool = False`

Mark register(s) as readonly.

invalid: `bool = False`

Mark register(s) as invalid. **remark** only to be used with `address=` and `count=`

default: `bool = False`

Use as default for undefined registers Define legal register range as:

`address= <= legal addresses <= address= + count=`

remark only to be used with `address=` and `count=`

register_count: `int = -1`

The following are internal variables

type_size: `int = -1`

```
class pymodbus.simulator.SimDevice(id: int, registers: list[SimData], offset_address: tuple[int, int, int, int]
                                   = (-1, -1, -1, -1), type_check: bool = False)
```

Bases: `object`

Configure a device with parameters and registers.

Registers are always defined as one block.

Some old devices uses 4 distinct blocks instead of 1 block, to support these devices, define 1 large block consisting of the 4 blocks and use the `offset_*=` parameters.

When using distinct blocks, coils and direct inputs are addressed differently, each register represent 1 coil/relay.

Device with shared registers:

```
SimDevice(
    id=1,
    registers=[SimData(...)]
)
```

Device with non-shared registers:

```
SimDevice(
    id=1,
    registers=[SimData(...)],
    non_shared_mode=True,
    offset_coil=0,
    offset_direct=10,
    offset_holding=20,
    offset_input=30,
)
```

Meaning registers:

- 0-9 are coils
- 10-19 are relays
- 20-29 are holding registers
- 30-.. are input registers

A server can contain either a single *SimDevice* or list of *SimDevice* to simulate a multipoint line.

Warning

each block is sorted by address !!

id: `int`

Address/id of device

Default 0 means accept all devices, except those specifically defined.

registers: `list[SimData]`

List of registers.

offset_address: `tuple[int, int, int, int] = (-1, -1, -1, -1)`

Use this for old devices with 4 blocks.

Tip

content is (coil, direct, holding, input)

type_check: `bool = False`

Enforce type checking, if True access are controlled to be conform with datatypes.

Type violations like e.g. reading INT32 as INT16 are returned as *ExceptionResponses*, as well as being logged.

8.1.3 Action data class examples

8.2 Simulator server

Note

This is a v4.0.0 functionality currently not available, please see the 3x simulator server.

8.3 Web frontend

Note

This is a v4.0.0 functionality currently not available, please see the 3x simulator server.

8.4 REST API

Note

This is a v4.0.0 functionality currently not available, please see the 3x simulator server.

EXAMPLES

Examples are divided in 2 parts:

The first part are some simple client examples which can be copied and run directly. These examples show the basic functionality of the library.

The second part are more advanced examples, but in order to not duplicate code, this requires you to download the examples directory and run the examples in the directory.

9.1 Ready to run examples:

These examples are very basic examples, showing how a client can communicate with a server.

You need to modify the code to adapt it to your situation.

9.1.1 Simple asynchronous client

Source: `examples/simple_async_client.py`

```
#!/usr/bin/env python3
"""Pymodbus asynchronous client example.

An example of a single threaded synchronous client.

usage: simple_async_client.py

All options must be adapted in the code
The corresponding server must be started before e.g. as:
    python3 server_sync.py
"""
import asyncio

import pymodbus.client as ModbusClient
from pymodbus import (
    Framertype,
    ModbusException,
    pymodbus_apply_logging_config,
)

async def run_async_simple_client(comm, host, port, framer=Framertype.SOCKET):
    """Run async client."""
```

(continues on next page)

(continued from previous page)

```

# activate debugging
pymodbus_apply_logging_config("DEBUG")

print("get client")
client: ModbusClient.ModbusBaseClient
if comm == "tcp":
    client = ModbusClient.AsyncModbusTcpClient(
        host,
        port=port,
        framer=framer,
        # timeout=10,
        # retries=3,
        # source_address=("localhost", 0),
    )
elif comm == "udp":
    client = ModbusClient.AsyncModbusUdpClient(
        host,
        port=port,
        framer=framer,
        # timeout=10,
        # retries=3,
        # source_address=None,
    )
elif comm == "serial": # pragma: no cover
    client = ModbusClient.AsyncModbusSerialClient(
        port,
        framer=framer,
        # timeout=10,
        # retries=3,
        baudrate=9600,
        bytesize=8,
        parity="N",
        stopbits=1,
        # handle_local_echo=False,
    )
else: # pragma: no cover
    print(f"Unknown client {comm} selected")
    return

print("connect to server")
await client.connect()
# test client is connected
assert client.connected

print("get and verify data")
try:
    # See all calls in client_calls.py
    rr = await client.read_coils(1, count=1, device_id=1)
except ModbusException as exc: # pragma: no cover
    print(f"Received ModbusException({exc}) from library")
    client.close()
    return

```

(continues on next page)

(continued from previous page)

```

if rr.isError(): # pragma: no cover
    print(f"Received exception from device ({rr})")
    # THIS IS NOT A PYTHON EXCEPTION, but a valid modbus message
    client.close()
    return
try:
    # See all calls in client_calls.py
    rr = await client.read_holding_registers(10, count=2, device_id=1)
except ModbusException as exc: # pragma: no cover
    print(f"Received ModbusException({exc}) from library")
    client.close()
    return
if rr.isError(): # pragma: no cover
    print(f"Received exception from device ({rr})")
    # THIS IS NOT A PYTHON EXCEPTION, but a valid modbus message
    client.close()
    return
value_int32 = client.convert_from_registers(rr.registers, data_type=client.DATATYPE.
→ INT32)
print(f"Got int32: {value_int32}")
print("close connection")
client.close()

if __name__ == "__main__":
    asyncio.run(
        run_async_simple_client("tcp", "127.0.0.1", 5020), debug=True
    )

```

9.1.2 Simple synchronous client

Source: examples/simple_sync_client.py

```

#!/usr/bin/env python3
"""Pymodbus synchronous client example.

An example of a single threaded synchronous client.

usage: simple_sync_client.py

All options must be adapted in the code
The corresponding server must be started before e.g. as:
    python3 server_sync.py
"""

# ----- #
# import the various client implementations
# ----- #
import pymodbus.client as ModbusClient
from pymodbus import (
    Framertype,
    ModbusException,

```

(continues on next page)

(continued from previous page)

```

    pymodbus_apply_logging_config,
)

def run_sync_simple_client(comm, host, port, framer=FramerType.SOCKET):
    """Run sync client."""
    # activate debugging
    pymodbus_apply_logging_config("DEBUG")

    print("get client")
    client: ModbusClient.ModbusBaseSyncClient
    if comm == "tcp":
        client = ModbusClient.ModbusTcpClient(
            host,
            port=port,
            framer=framer,
            # timeout=10,
            # retries=3,
            # source_address=("localhost", 0),
        )
    elif comm == "udp":
        client = ModbusClient.ModbusUdpClient(
            host,
            port=port,
            framer=framer,
            # timeout=10,
            # retries=3,
            # source_address=None,
        )
    elif comm == "serial": # pragma: no cover
        client = ModbusClient.ModbusSerialClient(
            port,
            framer=framer,
            # timeout=10,
            # retries=3,
            baudrate=9600,
            bytesize=8,
            parity="N",
            stopbits=1,
            # handle_local_echo=False,
        )
    else: # pragma: no cover
        print(f"Unknown client {comm} selected")
        return

    print("connect to server")
    client.connect()

    print("get and verify data")
    try:
        rr = client.read_coils(1, count=1, device_id=1)
    except ModbusException as exc: # pragma: no cover

```

(continues on next page)

(continued from previous page)

```

    print(f"Received ModbusException({exc}) from library")
    client.close()
    return
if rr.isError(): # pragma: no cover
    print(f"Received exception from device ({rr})")
    # THIS IS NOT A PYTHON EXCEPTION, but a valid modbus message
    client.close()
    return
try:
    # See all calls in client_calls.py
    rr = client.read_holding_registers(10, count=2, device_id=1)
except ModbusException as exc: # pragma: no cover
    print(f"Received ModbusException({exc}) from library")
    client.close()
    return
if rr.isError(): # pragma: no cover
    print(f"Received exception from device ({rr})")
    # THIS IS NOT A PYTHON EXCEPTION, but a valid modbus message
    client.close()
    return
value_int32 = client.convert_from_registers(rr.registers, data_type=client.DATATYPE.
→INT32)
print(f"Got int32: {value_int32}")

print("close connection")
client.close()

if __name__ == "__main__":
    run_sync_simple_client("tcp", "127.0.0.1", "5020")

```

9.2 Advanced examples

These examples are considered essential usage examples, and are guaranteed to work, because they are tested automatically with each dev branch commit using CI.

Tip

The examples need to be run from within the examples directory, unless you modify them. Most examples use `helper.py` and `client_*.py` or `server_*.py`. This is done to avoid maintaining the same code in multiple files.

- `examples.zip`
- `examples.tgz`

9.2.1 Client asynchronous calls

Source: `examples/client_async_calls.py`

Pymodbus Client modbus async all calls example.

Please see method `async_template_call` for a template on how to make modbus calls and check for different error conditions.

The handle* functions each handle a set of modbus calls with the same register type (e.g. coils).

All available modbus calls are present.

If you are performing a request that is not available in the client mixin, you have to perform the request like this instead:

```
from pymodbus.pdu.diag_message import ClearCountersRequest
from pymodbus.pdu.diag_message import ClearCountersResponse

request = ClearCountersRequest()
response = client.execute(request)
if isinstance(response, ClearCountersResponse):
    ... do something with the response
```

This example uses client_async.py and client_sync.py to handle connection, and have the same options.

The corresponding server must be started before e.g. as:

```
./server_async.py
```

9.2.2 Client asynchronous

Source: [examples/client_async.py](#)

Pymodbus asynchronous client example.

usage:

```
client_async.py [-h] [-c {tcp,udp,serial,tls}]
                [-f {ascii,rtu,socket,tls}]
                [-l {critical,error,warning,info,debug}] [-p PORT]
                [--baudrate BAUDRATE] [--host HOST]

-h, --help
    show this help message and exit
-c, -comm {tcp,udp,serial,tls}
    set communication, default is tcp
-f, --framer {ascii,rtu,socket,tls}
    set framer, default depends on --comm
-l, --log {critical,error,warning,info,debug}
    set log level, default is info
-p, --port PORT
    set port
--baudrate BAUDRATE
    set serial device baud rate
--host HOST
    set host, default is 127.0.0.1
```

The corresponding server must be started before e.g. as:

```
python3 server_sync.py
```

9.2.3 Client calls

Source: [examples/client_calls.py](#)

Pymodbus Client modbus all calls example.

Please see method **template_call** for a template on how to make modbus calls and check for different error conditions.

The handle* functions each handle a set of modbus calls with the same register type (e.g. coils).

All available modbus calls are present.

If you are performing a request that is not available in the client mixin, you have to perform the request like this instead:

```
from pymodbus.pdu.diag_message import ClearCountersRequest
from pymodbus.pdu.diag_message import ClearCountersResponse

request = ClearCountersRequest()
response = client.execute(request)
if isinstance(response, ClearCountersResponse):
    ... do something with the response
```

This example uses client_async.py and client_sync.py to handle connection, and have the same options.

The corresponding server must be started before e.g. as:

```
./server_async.py
```

9.2.4 Custom message

Source: [examples/custom_msg.py](#)

Pymodbus Synchronous Client Examples.

The following is an example of how to use the synchronous modbus client implementation from pymodbus:

```
with ModbusClient("127.0.0.1") as client:
    result = client.read_coils(1,10)
    print result
```

9.2.5 Client synchronous

Source: [examples/client_sync.py](#)

Pymodbus Synchronous Client Example.

An example of a single threaded synchronous client.

usage:

```
client_sync.py [-h] [-c {tcp,udp,serial,tls}]
               [-f {ascii,rtu,socket,tls}]
               [-l {critical,error,warning,info,debug}] [-p PORT]
               [--baudrate BAUDRATE] [--host HOST]

-h, --help
    show this help message and exit
-c, --comm {tcp,udp,serial,tls}
    set communication, default is tcp
-f, --framer {ascii,rtu,socket,tls}
    set framer, default depends on --comm
-l, --log {critical,error,warning,info,debug}
    set log level, default is info
-p, --port PORT
    set port
--baudrate BAUDRATE
```

(continues on next page)

(continued from previous page)

```
set serial device baud rate
--host HOST
set host, default is 127.0.0.1
```

The corresponding server must be started before e.g. as:

```
python3 server_sync.py
```

9.2.6 Server asynchronous

Source: [examples/server_async.py](#)

Pymodbus asynchronous Server Example.

An example of a multi threaded asynchronous server.

usage:

```
server_async.py [-h] [--comm {tcp,udp,serial,tls}]
                [--framer {ascii,rtu,socket,tls}]
                [--log {critical,error,warning,info,debug}]
                [--port PORT] [--store {sequential,sparse,factory,none}]
                [--device_ids DEVICE_IDS]

-h, --help
    show this help message and exit
-c, --comm {tcp,udp,serial,tls}
    set communication, default is tcp
-f, --framer {ascii,rtu,socket,tls}
    set framer, default depends on --comm
-l, --log {critical,error,warning,info,debug}
    set log level, default is info
-p, --port PORT
    set port
    set serial device baud rate
--store {sequential,sparse,factory,none}
    set datastore type
--device_ids DEVICE_IDS
    set list of devices to respond to
```

The corresponding client can be started as:

```
python3 client_sync.py
```

9.2.7 Server callback

Source: [examples/server_callback.py](#)

Pymodbus Server With Callbacks.

This is an example of adding callbacks to a running modbus server when a value is written to it.

9.2.8 Server tracer

Source: [examples/server_hook.py](#)

Pymodbus Server With request/response manipulator.

This is an example of using the builtin request/response tracer to manipulate the messages to/from the modbus server

9.2.9 Server synchronous

Source: [examples/server_sync.py](#)

Pymodbus Synchronous Server Example.

An example of a single threaded synchronous server.

usage:

```
server_sync.py [-h] [--comm {tcp,udp,serial,tls}]
               [--framer {ascii,rtu,socket,tls}]
               [--log {critical,error,warning,info,debug}]
               [--port PORT] [--store {sequential,sparse,factory,none}]
               [--device_ids DEVICE_IDS]

-h, --help
    show this help message and exit
-c, --comm {tcp,udp,serial,tls}
    set communication, default is tcp
-f, --framer {ascii,rtu,socket,tls}
    set framer, default depends on --comm
-l, --log {critical,error,warning,info,debug}
    set log level, default is info
-p, --port PORT
    set port
    set serial device baud rate
--store {sequential,sparse,factory,none}
    set datastore type
--device_ids DEVICE_IDS
    set list of devices to respond to
```

The corresponding client can be started as:

```
python3 client_sync.py
```

REMARK It is recommended to use the async server! The sync server is just a thin cover on top of the async server and is in some aspects a lot slower.

9.2.10 Server updating

Source: [examples/server_updating.py](#)

Pymodbus asynchronous Server with updating task Example.

An example of an asynchronous server and a task that runs continuously alongside the server and updates values.

usage:

```
server_updating.py [-h] [--comm {tcp,udp,serial,tls}]
                  [--framer {ascii,rtu,socket,tls}]
                  [--log {critical,error,warning,info,debug}]
```

(continues on next page)

(continued from previous page)

```

        [--port PORT] [--store {sequential,sparse,factory,none}]
        [--device_ids DEVICE_IDS]

-h, --help
    show this help message and exit
-c, --comm {tcp,udp,serial,tls}
    set communication, default is tcp
-f, --framer {ascii,rtu,socket,tls}
    set framer, default depends on --comm
-l, --log {critical,error,warning,info,debug}
    set log level, default is info
-p, --port PORT
    set port
    set serial device baud rate
--store {sequential,sparse,factory,none}
    set datastore type
--device_ids DEVICE_IDS
    set number of devices to respond to

```

The corresponding client can be started as:

```
python3 client_sync.py
```

9.2.11 Simulator example

Source: [examples/simulator.py](#)

Pymodbus simulator server/client Example.

An example of how to use the simulator (server) with a client.

for usage see documentation of simulator

Tip

pymodbus.simulator starts the server directly from the commandline

9.2.12 Simulator datastore (shared storage) example

Source: [examples/datastore_simulator_share.py](#)

Pymodbus datastore simulator Example.

An example of using simulator datastore with json interface.

Detailed description of the device definition can be found at:

<https://pymodbus.readthedocs.io/en/latest/source/library/simulator/config.html#device-entries>

usage:

```

datastore_simulator_share.py [-h]
                             [--log {critical,error,warning,info,debug}]
                             [--port PORT]
                             [--test_client]

```

(continues on next page)

(continued from previous page)

```
-h, --help
    show this help message and exit
-l, --log {critical,error,warning,info,debug}
    set log level
-p, --port PORT
    set port to use
--test_client
    starts a client to test the configuration
```

The corresponding client can be started as:

```
python3 client_sync.py
```

Tip

This is NOT the pymodbus simulator, that is started as `pymodbus.simulator`.

9.2.13 Message Parser

Source: [examples/message_parser.py](#)

Modbus Message Parser.

The following is an example of how to parse modbus messages using the supplied framers.

9.3 Examples contributions

These examples are supplied by users of pymodbus. The pymodbus team thanks for sharing the examples.

9.3.1 Solar

Source: [examples/contrib/solar.py](#)

Pymodbus Synchronous Client Example.

Modified to test long term connection.

Modified to actually work with Huawei SUN2000 inverters, that better support async Modbus operations so errors will occur Configure HOST (the IP address of the inverter as a string), PORT and CYCLES to fit your needs

9.3.2 Serial Forwarder

Source: [examples/contrib/serial_forwarder.py](#)

Pymodbus SerialRTU2TCP Forwarder

```
usage : python3 serial_forwarder.py -log DEBUG -port "/dev/ttyUSB0" -baudrate 9600 -server_ip "192.168.1.27"
-server_port 5020 -device_ids 1 2 3
```


AUTHORS

All these versions would not be possible without volunteers!

This is a complete list for each major version.

A big “thank you” to everybody who helped out.

10.1 Pymodbus version 3 family

Thanks to

- aaru-astranis
- ahcm-dev
- AKJ7
- Alex
- Alex Ruddick
- Alexander Lanin
- Alexandre CUER
- alexis-care
- Alois Hockenschlohe
- Andy Walker
- Arjan
- André Srinivasan
- andrew-harness
- banana-sun
- Blaise Thompson
- Breina
- brherger
- CapraTheBest
- cgernert
- corollaries
- Chandler Riehm

- Chris Hung
- Christian Krause
- Christian Pfisterer
- daanwtb
- Daniel Rauber
- dhoomakethu
- doelki
- DominicDataP
- Dominique Martinet
- Dries
- duc996
- efdx
- embedded-bed
- Erlend E. Aasland
- Esco441-91
- Farzad Panahi
- Fredo70
- Gao Fang
- Ghostkeeper
- Hangyu Fan
- Hayden Roche
- igorbga
- Iktek
- Ilkka Ollakka
- Jakob Ruhe
- Jakob Schlyter
- James Braza
- James Cameron
- James Hilliard
- jan iversen
- Jerome Velociter
- Joe Burmeister
- John Miko
- Jonathan Reichelt Gjertsen
- JorisW
- julian

- Julian Lunz
- Justin Standring
- Kenny Johansson
- Kürşat Aktaş
- laund
- Logan Gunthorpe
- Luke Hoggatt
- Mark Deneen
- Marko Luther
- Martyy
- Máté Szabó
- Matthias Straka
- Matthias Urlichs
- Maxime LEURENT
- Michel F
- Mickaël Schoentgen
- Pavel Kostromitinov
- peufeu2
- Philip Couling
- Philip Jones
- Robin Trabert
- Qi Li
- Sebastian Machuca
- Sefa Keleş
- Steffen Beyer
- sumguytho
- Szewcson
- Thijs W
- Totally a booplicate
- WouterTuinstra
- wriswith
- Yash Jani
- Yohrog
- yyokusa
- zaid bin saeed

10.2 Pymodbus version 2 family

Thanks to

- alecjohanson
- Alexey Andreyev
- Andrea Canidio
- Carlos Gomez
- Cougar
- Christian Sandberg
- dhoomakethu
- dices
- Dmitri Zimine
- Emil Vanherp
- er888kh
- Eric Duminil
- Erlend Egeberg Aasland
- hackerboygn
- Jian-Hong Pan
- Jose J Rodriguez
- Justin Searle
- Karl Palsson
- Kim Hansen
- Kristoffer Sjöberg
- Kyle Altendorf
- Lars Kruse
- Malte Kliemann
- Memet Bilgin
- Michael Corcoran
- Mike
- sanjay
- Sekenre
- Siarhei Farbotka
- Steffen Vogel
- tcplomp
- Thor Michael Støre
- Tim Gates
- Ville Skyttä

- Wild Stray
- Yegor Yefremov

10.3 Pymodbus version 1 family

Thanks to

- Antoine Pitrou
- Bart de Waal
- bashwork
- bje-
- Claudio Catterina
- Chintalagiri Shashank
- dhoomakethu
- dragoshenron
- Elvis Stansvik
- Eren Inan Canpolat
- Everley
- Fabio Bonelli
- fleimgruber
- francozappa
- Galen Collins
- Gordon Broom
- Hamilton Kibbe
- Hynek Petrak
- idahogray
- Ingo van Lil
- Jack
- jbiswas
- jon mills
- Josh Kelley
- Karl Palsson
- Matheus Frata
- Patrick Fuller
- Perry Kundert
- Philippe Gauthier
- Rahul Raghunath
- sanjay

- schubduese42
- semyont
- Semyon Teplitsky
- Stuart Longland
- Yegor Yefremov

10.4 Pymodbus version 0 family

Thanks to

- Albert Brandl
- Galen Collins

Import to github was based on code from:

- S.W.A.C. GmbH, Germany.
- S.W.A.C. Bohemia s.r.o., Czech Republic.
- Hynek Petrak
- Galen Collins

CHANGELOG

All these version would not be possible without a lot of work from volunteers!

We, the maintainers, are grateful for each pull requests small or big, that helps make pymodbus a better product.

Authors: contains a complete list of volunteers have contributed to each major version.

11.1 Version 3.11.3

- Coverage 100% (using no cover, when needed). (#2783)
- Create pypi alias for home-assistant. (#2782)
- Bump utilities in pyproject.toml. (#2780)
- Fix pymodbus.simulator. (#2773)

11.2 Version 3.11.2

- Clarify documentation on reconnect_delay (#2769)
- Solve CI complaints. (#2766)
- Coverage not allowed below 99.5%. (#2765)
- Test coverage global 100%. (#2764)
- Test coverage simulator 100%. (#2763)
- Test coverage server 100%. (#2760)
- Fix python3.14 deprecation. (#2759)
- Test coverage datastore 100%. (#2757)
- Context test failed due to function code overwritten. (#2758)
- Test coverage transaction 100%. (#2756)
- Test coverage pdu 100%. (#2755)
- Framer test 100%. (#2754)
- llow sub_function_code is custom PDU. (#2753)
- Generate pdu table direct. (#2752)
- Clean pdu lookup in simulator. (#2751)
- diag sub_function_code is 2 bytes. (#2750)

- Requesthandler ignore missing devices logging (#2749)
- Simplify pdu lookup. (#2745)
- Missing coma in string representation of ModbusPDU (#2748)
- Correct “install uv”. (#2744)
- Suppress aiohttp missing. (#2743)
- Remove garbage bytes in serial comm. (#2741)
- Test now included python 3.14.
- Stricter types with pyright (#2731)

11.3 Version 3.11.1

- Auto debug in case of an error. (#2738)
- Remove duplicate log lines. (#2736)
- Remove unused callback in ServerRequestHandler (#2737)
- test on Python 3.14 (#2735)
- Validate address in all datastores. (#2733)
- Use asyncio.Event to deterministically ensure simulator start (#2734)
- Ignore lockfile (#2730)
- Link api_changes/changelog to README.
- Add note about semver.org.
- Datastore, add typing to set/get. (#2729)
- Move exception codes to constants. (#2728)
- Move ExceptionResponse to proper file. (#2727)
- make base frame signature match subclasses (#2726)
- Switch from venv+pip to uv (#2723)
- Cleanup CI configuration (#2724)
- Simplify code flow for broadcast requests (#2720)
- Fix serial_forwarder.py from examples/contrib (#2715)
- Remove discord. (#2714)

11.4 Version 3.11.0

- Correct bit handling (each byte is LSB->MSB). (#2707)
- read_input_registers docstring change count to regs (#2704)
- Add dev_id/tid check in clients (#2711)

11.5 Version 3.10.0

- Raise runtimeerror if listen() fails. (#2697)
- Correct values parameter in setValues. (#2696)
- Correct return from getValues. (#2695)
- Add request fc to exceptionResponse. (#2694)
- DummyProtocol is not async (#2686)
- Handle “little” for multiple values in to_registers (#2678)
- Remove unused const. (#2676)
- Add retries to ModbusPDU class (#2672)
- Don’t invoke *trace_connect* callback twice (#2670)
- ensure unpacking of proper length during decoding (#2664) (#2665)
- README clean-up (#2659)
- Bump coverage to 95,5% (#2658)
- Simplify response rejection. (#2657)
- Bump coverage to 93%. (#2656)
- Solve ModbusDeviceContext bug. (#2653)
- Bit handling LSB -> MSB across bytes. (#2634)
- Change slave to device_id and slave= to device_id=. (#2600)
- Remove payload. (#2524)

11.6 Version 3.9.2

- Reactivate simulator validate. (#2643)
- Don’t bool-test explicit datastores (#2638)
- Test and hard delayed response test. (#2636)
- Update simulator doc. (#2635)
- SimData update
- Officially working towards 4.0.0

11.7 Version 3.9.1

- Correct byte order in bits. (#2631)

11.8 Version 3.9.0

- Correct bit handling internally and in API. (#2627)
- default argument ModbusSequentialDataBlock (#2622)
- Fix exception error message for decoding response (#2618)

- Expose exception_code to API. (#2615)
- Simplify ruff config (#2611)
- Documentation dont fixed. (#2605)
- sum() can operate on an Iterator directly (#2610)
- SimData update. (#2601)
- Start<x>Server custom_functions -> custom_pdu.
- Update pyproject.toml to remove python 3.9.
- Remove validate() from datastores. (#2595)
- Python 3.9 is EOL, not supported actively. (#2596)
- correct handle_local_echo for sync client. (#2593)
- devcontainer, automatic install. (#2583)
- Don't set_result on completed futures. (#2582)
- Flush recv_buffer before each transaction write. (#2581)
- Add missing trace. (#2578)
- Update github actions. (#2579)

11.9 Version 3.8.6

- Allow id=0 and check if response.id == request.id. (#2572)

11.10 Version 3.8.5

- New simulator is WIP, not to be used. (#2568)
- dev_id=0 no response expected (returns ExceptionResponse(0xff)). (#2567)
- New simulator datastore. (#2535)

11.11 Version 3.8.4

- Parameterize string encoding in convert_to_registers and convert_from_registers (#2558)
- Fix client modbus function calls in remote by adding count as keyword argument (#2563)
- Fix exception text in ModbusPDU.validateAddress (#2551)
- Typo arround *no_response_expected* (#2550)
- Trace new connection in server. (#2549)
- Add trace to server.
- Update misleading DATATYPE text. (#2547)
- Fix pylint.
- Clarify server usage.
- Solve instable transaction testing. (#2538)

11.12 Version 3.8.3

- Remove deprecate from payload. (#2532)
- Add background parameter to servers. (#2529)
- Split async_io.py and simplify server start/stop. (#2528)
- Update custom_msg example to include server. (#2527)
- Move repl doc to repl repo. (#2522)
- Add API to set max until disconnect. (#2521)

11.13 Version 3.8.2

- Asyncio future removed from sync client. (#2514)

11.14 Version 3.8.1

- Convert endianness (#2506)
- Fix sync serial client, loop. (#2510)
- Correct future. (#2507)
- Correct #2501 (#2504)
- Raise exception on no response in async client. (#2502)
- re-instantiate Future on reconnect (#2501)
- Remove all trailing zeroes during string decoding (#2493)
- Fix too many sync client log messages. (#2491)

11.15 Version 3.8.0

- slave_id -> dev_id (internally). (#2486)
- Pin python 3.13.0 and update ruff. (#2487)
- Add documentation link to README. (#2483)
- Add datatype bits to convert_to/from_registers. (#2480)
- Add trace API to server. (#2479)
- Add trace API for client. (#2478)
- Integrate TransactionManager in server. (#2475)
- Rename test/sub. (#2473)
- Check server closes file descriptors. (#2472)
- Update http_server.py (#2471)
- Restrict write_registers etc to list[int]. (#2469)
- Write_registers/pdu typing again. (#2468)
- Remove ModbusExceptions enum. (#2467)

- Add special ssl socket handling of “no data”. (#2466)
- Add tip that values= will be modified to list[int]. (#2465)
- client 100% test coverage (#2396)
- Extend TransactionManager to handle sync. (#2457)
- Add convert_from to simple examples. (#2458)
- New async transaction manager. (#2453)
- Deprecate BinaryPayloadDecoder / BinaryPayloadBuilder. (#2456)
- Correct close for server transport. (#2455)
- RTU frame problem, when received split. (#2452)
- pdu, 100% coverage. (#2450)
- Refactor PDU, add strong typing to base classes. (#2438)
- Enforce keyword only parameters. (#2448)
- Fix read_device_information with sync client. (#2441)
- Simplify syncTransactionManager. (#2443)
- Import examples direct. (#2442)
- rename ModbusExceptions enums to legal constants. (#2436)
- Add typing to examples. (#2435)
- Refactor PDU diag. (#2421)
- Fix client lock, Parallel API calls are not permitted. (#2434)
- Ensure accept_no_response_limit > retries. (#2433)
- Check client and frametype. (#2426)
- Add MDAP to TLS frame. (#2425)
- Clean/Finalize testing for bit functions. (#2420)
- Simplify pdu bit, remove skip_encode. (#2417)
- remove zero_mode parameter. (#2354)
- Prepare refactor messages. (#2416)
- Fixed handle local echo in serialserver (#2415)
- Correct minor framer/pdu errors. (#2407)
- Rtu decode frames without byte count. (#2412)
- Improve type of parameter values of write_registers (#2411)
- PDU lookupClass work with sub function code. (#2410)
- Correct wait_next_api link in README. (#2406)

11.16 Version 3.7.4

- Clean PDU init. (#2399)
- Wrong close, when transaction do not match. (#2401)
- Remove unmaintained (not working) example contributions. (#2400)
- All pdu (incl. function code) tests to pdu directory. (#2397)
- Add *no_response_expected* argument to requests (#2385)
- Resubmit: Don't close/reopen tcp connection on single modbus message timeout (#2350)
- 100% test coverage for PDU. (#2394)
- Type DecodePDU. (#2392)
- Update to use DecodePDU. (#2391)
- Client/Server decoder renamed and moved to pdu. (#2390)
- Move client/server decoder to pdu. (#2388)
- Introducing PyModbus Guru on Gurubase.io (#2387)
- Remove IllegalFunctionRequest. (#2384)
- remove ModbusResponse. (#2383)
- Add typing to pdu base classes. (#2380)
- Updated roadmap.
- remove databuffer from framer. (#2379)
- Improve retries for sync client. (#2377)
- Move process test to framer tests (#2376)
- Framer do not check ids (#2375)
- Remove callback from framer. (#2374)
- Auto fill device ids for clients. (#2372)
- Reenable multidrop tests. (#2370)
- write_register/s accept bytes or int. (#2369)
- roadmap corrections.
- Added roadmap (not written in stone). (#2367)
- Update README to show python 3.13.
- Test on Python 3.13 (#2366)
- Use @abstractmethod (#2365)
- Corrected smaller documentation bugs. (#2364)
- README as landing page in readthedocs. (#2363)

11.17 Version 3.7.3

- 100% test coverage of framers (#2359)
- Framer, final touches. (#2360)
- Readme file renamed (#2357)
- Remove old framers (#2358)
- frameProcessIncomingPacket removed (#2355)
- Cleanup framers (reduce old_framers) (#2342)
- Run CI on PR targeted at wait_next_api.
- Sync client, allow unknown recv msg size. (#2353)
- integrate old rtu framer in new framer (#2344)
- Update README.rst (#2351)
- Client.close should not allow reconnect= (#2347)
- Remove async client.idle_time(). (#2349)
- Client doc, add common methods (base). (#2348)
- Reset receive buffer with send(). (#2343)
- Remove unused protocol_id from pdu (#2340)
- CI run on demand on non-protected branches. (#2339)
- Server listener and client connections have is_server set. (#2338)
- Reopen listener in server if disconnected. (#2337)
- Regroup test. (#2335)
- Improve docs around sync clients and reconnection (#2321)
- transport 100% test coverage (again) (#2333)
- Update actions to new node.js. (#2332)
- Bump 3rd party (#2331)
- Documentation on_connect_callback (#2324)
- Fixes the unexpected implementation of the ModbusSerialClient.connected property (#2327)
- Forward error responses instead of timing out. (#2329)
- Add *stacklevel=2* to logging functions (#2330)
- Fix encoding & decoding of ReadFileRecordResponse (#2319)
- Improvements for example/contib/solar (#2318)
- Update solar.py (#2316)
- Remove double conversion in int (#2315)
- Complete pull request #2310 (#2312)
- fixed type hints for write_register and write_registers (#2309)
- Remove _header from framers. (#2305)

11.18 Version 3.7.2

- Correct README
- Rename branch wait3.8.0 to wait_next_API

11.19 Version 3.7.1

- Better error message, when pyserial is missing.
- Slave=0 will return first response, used to identify device address. (#2298)
- Feature/add simulator api skeleton (#2274)
- Correct max. read size for registers. (#2295)
- Ruff complains, due to upgrade. (#2296)
- Properly process 'slaves' argument (#2292)
- Update repl requirement to >= 2.0.4 (#2291)
- Fix aiohttp < 3.9.0 (#2289)
- Simplify framer test setup (#2290)
- Clean up ModbusControlBlock (#2288)
- example docstrings diag_message -> pdu.diag_message (#2286)
- Explain version schema (#2284)
- Add more testing for WriteRegisters. (#2280)
- Proof for issue 2273. (#2277)
- Update simulator tests. (#2276)

11.20 Version 3.7.0

- Remove unneeded client parameters. (#2272)
- simulator: Fix context single parameter (#2264)
- buildPacket can be used for Request and Response (#2262)
- More descriptive decoder exceptions (#2260)
- Cleanup ReadWriteMultipleRegistersResponse and testing (#2261)
- Feature/simulator addressing (#2258)
- Framer optimization (apart from RTU). (#2146)
- Use mock.patch.object to avoid protected access errors. (#2251)
- Fix some mypy type checking errors in test_transaction.py (#2250)
- Update check for windows platform (#2247)
- Logging 100% coverage. (#2248)
- CI, Block draft PRs to use CPU minutes. (#2245, #2246)
- Remove kwargs client. (#2243, #2244, #2257)

- remove kwargs PDU messagees. (#2240)
- Remove message_generator example (not part of API). (#2239)
- Update dev dependencies (#2241)
- Fix ruff check in CI (#2242)
- Remove kwargs. (#2236, #2237)
- Simulator config, kwargs -> parameters. (#2235)
- Refactor transaction handling to better separate async and sync code. (#2232)
- Simplify some BinaryPayload pack operations (#2224)
- Fix writing to serial (rs485) on windows os. (#2191)
- Remember to remove serial writer. (#2209)
- Transaction_id for serial == 0. (#2208)
- Solve pylint error.
- Sync TLS needs time before reading frame (#2186)
- Update transaction.py (#2174)
- PDU classes -> pymodbus/pdu. (#2160)
- Speed up no data detection. (#2150)
- RTU decode hunt part. (#2138)
- Dislodge client classes from modbusProtocol. (#2137)
- Merge new message layer and old framer directory. (#2135)
- Coverage == 91%. (#2132)
- Remove binary_framer. (#2130)
- on_reconnect_callback -> on_connect_callback. (#2122)
- Remove certfile,keyfile,password from TLS client. (#2121)
- Drop support for python 3.8 (#2112)

11.21 Version 3.6.9

- Remove python 3.8 from CI
- Log comm retries. (#2220)
- Solve serial unrequested frame. (#2219)
- test convert registers with 1234... (#2217)
- Fix writing to serial (rs485) on windows os. (#2191)
- Remember to remove serial writer. (#2209)
- Update client.rst (#2199)
- Fix usage file names (#2194)
- Show error if example is run without support files. (#2189)
- Solve pylint error.

- Describe zero_mode in ModbusSlaveContext.__init__ (#2187)
- Datastore will not return ExceptionResponse. (#2175)
- call async datastore from modbus server (#2144)
- Transaction id overrun.
- Add minimal devcontainer. (#2172)
- Sphinx: do not turn warnings into errors.
- Fix usage of AsyncModbusTcpClient in client docs page (#2169)
- Bump actions CI. (#2166)
- Request/Response: change execute to be async method (#2142)
- datastore: add async_setValues/getValues methods (#2165)
- fixed kwargs not being expanded for actions on bit registers, adjusted tests to catch this issue (#2161)
- Clean datastore setValues. (#2145)
- modbus_server: call execute in a way that those can be either coroutines or normal methods (#2139)
- Streamline message class. (#2133)
- Fix decode for wrong mdap len.
- SOCKET/TLS framer using message decode(). (#2129)
- ASCII framer using message decode() (#2128)
- Add generate_ssl() to TLS client as helper. (#2120)
- add _legacy_decoder to message rtu (#2119)

11.22 Version 3.6.8

- Allow socket exception response with wrong length

11.23 Version 3.6.7

- Add lock to async requests, correct logging and length calc. (FIX, not on dev)
- test_simulator: use unused_tcp_port fixture (#2141)
- streamline imports in Factory.py (#2140)
- Secure testing is done with pymodbus in PR. (#2136)
- Fix link to github in README (#2134)
- Wildcard exception catch from pyserial. (#2125)
- Problem with stale CI. (#2117)
- Add connection exception to list of exceptions catpured in retries (#2113)
- Move on_reconnect to client level (#2111)
- Bump github stale. (#2110)
- update package_test_tool (add 4 test scenarios) (#2107)
- Bump dependencies. (#2108)

- Cancel send if no connection. (#2103)

11.24 Version 3.6.6

- Solve transport close() as not inherited method. (#2098)
- enable *mypy --check-untyped-defs* (#2096)
- Add get_expected_response_length to transaction.
- Remove control encode in framersRemove control encode in framers. (#2095)
- Bump codeql in CI to v3. (#2093)
- Improve server types (#2092)
- Remove pointless try/except (#2091)
- Improve transport types (#2090)
- Use explicit ValueError when called with incorrect function code (#2089)
- update message tests (incorporate all old tests). (#2088)
- Improve simulator type hints (#2084)
- Cleanup dead resetFrame code (#2082)
- integrate message.encode() into framer.buildPacket. (#2062)
- Repair client close() (intern= is needed for ModbusProtocol). (#2080)
- Updated Message_Parser example (#2079)
- Fix #2069 use released repl from pypi (#2077)
- Fix field encoding of Read File Record Response (#2075)
- Improve simulator types (#2076)
- Bump actions. (#2071)

11.25 Version 3.6.5

- Update framers to ease message integration (only decode/encode) (#2064)
- Add negative acknowledge to modbus exceptions (#2065)
- add Message Socket/TLS and amend tests. (#2061)
- Improve factory types (#2060)
- ASCII. (#2054)
- Improve datastore documentation (#2056)
- Improve types for messages (#2058)
- Improve payload types (#2057)
- Reorganize datastore inheritance (#2055)
- Added new message (framer) raw + 100%coverage. (#2053)
- message classes, first step (#1932)
- Use AbstractMethod in transport. (#2051)

- A datastore for each slave. (#2050)
- Only run coverage in ubuntu / python 3.12 (#2049)
- Replace lambda with functools.partial in transport. (#2047)
- Move self.loop in transport to init() (#2046)
- Fix decoder bug (#2045)
- Add support for server testing in package_test_tool. (#2044)
- DictTransactionManager -> ModbusTransactionManager (#2042)
- eliminate redundant server_close() (#2041)
- Remove reactive server (REPL server). (#2038)
- Improve types for client (#2032)
- Improve HTTP server type hints (#2035)
- eliminate asyncio.sleep() and replace time.sleep() with a timeout (#2034)
- Use “new” inter_byte_timeout and is_open for pyserial (#2031)
- Add more type hints to datastore (#2028)
- Add more framer tests, solve a couple of framer problems. (#2024)
- Rework slow tests (use NULL_MODEM) (#1995)
- Allow slave=0 in serial communication. (#2023)
- Client package test tool. (#2022)
- Add REPL documentation back with links to REPL repo (#2017)
- Move repl to a seperate repo (#2009)
- solve more mypy issues with client (#2013)
- solve more mypy issues with datastore (#2010)
- Remove useless. (#2011)
- streamline transport tests. (#2004)
- Improve types for REPL (#2007)
- Specify more types in base framer (#2005)
- Move htmlcov -> build/cov (#2003)
- Avoid pylint complain about lambda. (#1999)
- Improve client types (#1997)
- Fix setblocking call (#1996)
- Activate warnings in pytest. (#1994)
- Add profile option to pytest. (#1991)
- Simplify message tests (#1990)
- Upgrade pylint and ruff (#1989)
- Add first architecture document. (#1988)
- Update CONTRIBUTING.rst.

- Return None for broadcast. (#1987)
- Make ModbusClientMixin Generic to fix type issues for sync and async (#1980)
- remove strange None default (#1984)
- Fix incorrect bytearray type hint in diagnostics query (#1983)
- Fix URL to CHANGELOG (#1979)
- move server_hostname to be local in tls client. (#1978)
- Parameter “strict” is and was only used for serial server/client. (#1975)
- Removed unused parameter close_comm_on_error. (#1974)

11.26 Version 3.6.4

- Update datastore_simulator example with client (#1967)
- Test and correct receiving more than one packet (#1965)
- Remove unused FifoTransactionManager. (#1966)
- Always set exclusive serial port access. (#1964)
- Add server/client network stub, to allow test of network packets. (#1963)
- Combine confest to a central file (#1962)
- Call on_reconnect_callback. (#1959)
- Readd ModbusBaseClient to external API.
- Update README.rst
- minor fix for typo and consistency (#1946)
- More coverage. (#1947)
- Client coverage 100%. (#1943)
- Run coverage in CI with % check of coverage. (#1945)
- transport 100% coverage. (#1941)
- contrib example: TCP drainage simulator with two devices (#1936)
- Remove “pragma no cover”. (#1935)
- transport_serial -> serialtransport. (#1933)
- Fix behavior after Exception response (#1931)
- Correct expected length for udp sync client. (#1930)

11.27 Version 3.6.3

- solve Socket_framer problem with Exception response (#1925)
- Allow socket frames to be split in multiple packets (#1923)
- Reset frame for serial connections.
- Source address None not 0.0.0.0 for IPv6
- Missing Copyright in License file

- Correct wrong url to modbus protocol spec.
- Fix serial port in TestComm.

11.28 Version 3.6.2

- Set documentation to v3.6.2.

11.29 Version 3.6.1

- Solve pypi upload error.

11.30 Version 3.6.0

- doc: Fix a code mismatch in client.rst
- Update README.
- truncated duration to milliseconds
- Update examples for current dev.
- Ignore all remaining implicit optional (#1888)
- docstring
- Remove unnecessary abort() call
- Enable RUF013 (implicit optional) (#1882)
- Support aiohttp 3.9.0b1 (#1886)
- Actually perform aiohttp runner teardown
- Pin to working aiohttp (#1884)
- Docstring typo cleanup (#1879)
- Clean client API imports. (#1819)
- Update issue template.
- Eliminate implicit optional in reconnect_delay* (#1874)
- Split client base in sync/async version (#1878)
- Rework host/port and listener setup (#1866)
- use baudrate directly (#1872)
- Eliminate more implicit optional (#1871)
- Fix serial server args order (#1870)
- Relax test task/thread checker. (#1867)
- Make doc link references version dependent. (#1864)
- Remove pre-commit (#1860)
- Ruff reduce ignores. (#1862)
- Bump ruff to 0.1.3 and remove ruff.toml (#1861)
- More elegant noop. (#1859)

- Cache (#1829)
- Eliminate more implicit optional (#1858)
- Ignore files downloaded by pytest (#1857)
- Avoid malicious user path input (#1855)
- Add more return types to transport (#1852)
- Do not attempt to close an already-closed serial connection (#1853)
- Fix stopbits docstring typo (#1850)
- Convert type hints to PEP585 (#1846)
- Eliminate even more implicit optional (#1845)
- Eliminate more implicit optionals in client (#1844)
- Eliminate implicit optional in transport_serial (#1843)
- Make client type annotations compatible with async client usage (#1842)
- Merge pull request #1838 from pymodbus-dev/ruff
- Eliminate implicit optional in simulator (#1841)
- eliminate implicit optional for callback_disconnected (#1840)
- pre-commit run --all-files
- Update exclude paths
- Replace black with ruff
- Use other dependency groups for 'all' (#1834)
- Cleanup author/maintainer fields (#1833)
- Consistent messages if imports fail (#1831)
- Client/Server framer as enum. (#1822)
- Solve relative path in examples. (#1828)
- Eliminate implicit optional for CommParams types (#1825)
- Add 3.12 classifier (#1826)
- Bump actions/stale to 8.0.0 (#1824)
- Cleanup paths included in mypy/pylint (#1823)
- Client documentation amended and updated. (#1820)
- Import aiohttp in way pleasing mypy. (#1818)
- Update doc, remove md files. (#1814)
- Bump dependencies. (#1816)
- Solve pylint / pytest.
- fix pylint.
- Examples are without parent module.
- Wrong zip of examples.
- Serial delay (#1810)

- Add python 3.12. (#1800)
- Release errors (pyproject.toml changes). (#1811)

11.31 Version 3.5.4

- Release errors (pyproject.toml changes). (#1811)

11.32 Version 3.5.3

- Simplify transport_serial (modbus use) (#1808)
- Reduce transport_serial (#1807)
- Change to pyproject.toml. (#1805)
- fixes access to asyncio loop via loop property of SerialTransport (#1804)
- Bump aiohttp to support python 3.12. (#1802)
- README wrong links. (#1801)
- CI caching. (#1796)
- Solve pylint unhappy. (#1799)
- Clean except last 7 days. (#1798)
- Reconnect_delay == 0, do not reconnect. (#1795)
- Update simulator.py method docstring (#1793)
- add type to isError. (#1781)
- Allow repr(ModbusException) to return complete information (#1779)
- Update docs. (#1777)

11.33 Version 3.5.2

- server tracer example. (#1773)
- sync connect missing. (#1772)
- simulator future problem. (#1771)

11.34 Version 3.5.1

- Always close socket on error (reset_sock). (#1767)
- Revert reset_socket change.
- add close_comm_on_error to example.
- Test long term (HomeAssistant problem). (#1765)
- Update ruff to 0.0.287 (#1764)
- Remove references to ModbusSerialServer.start (#1759) (#1762)
- Readd test to get 100% coverage.
- transport: Don't raise a RuntimeError in ModbusProtocol.error_received() (#1758)

11.35 Version 3.5.0

- Async retry (#1752)
- test_client: Fix test_client_protocol_execute() (#1751)
- Use enums for constants (#1743)
- Local Echo Broadcast with Async Clients (#1744)
- Fix #1746 . Return missing result (#1748)
- Document nullmodem. (#1739)
- Add system health check to all tests. (#1736)
- Handle partial message in ReadDeviceInformationResponse (#1738)
- Broadcast with Handle Local Echo (#1737)
- transport_emulator, part II. (#1710)
- Added file AUTHORS, to list all Volunteers. (#1734)
- Fix #1702 and #1728 (#1733)
- Clear retry count when success. (#1732)
- RFC: Reduce parameters for REPL server classes (#1714)
- retries=1, solved. (#1731)
- Improved the example “server Updating.py” (#1720)
- pylint 3.11 (#1730)
- Correct retry loop. (#1729)
- Fix faulty not check (#1725)
- bugfix local echo handling on sync clients (#1723)
- Updated copyright in LICENSE.
- Correct README pre-commit.
- Fix custom message parsing in RTU framer (#1716)
- Request tracer (#1715)
- pymodbus.server: allow strings for “-p” paramter (#1713)
- New nullmodem and transport. (#1696)
- xdist loadscope (test is not split). (#1708)
- Add client performance example. (#1707)

11.36 Version 3.4.1

- Fix serial startup problems. (#1701)
- pass source_address in tcp client. (#1700)
- serial server use source_address[0]. (#1699)
- Examples coverage nearly 100%. (#1694)
- new async serial (#1681)

- Docker is not supported (lack of maintainer). (#1693)
- Forwarder `write_coil` -> `write_coil`. (#1691)
- Change default `source_address` to (0.0.0.0, 502) (#1690)
- Update `ruff` to 0.0.277 (#1689)
- Fix dict comprehension (#1687)
- Removed `requests` dependency from `contrib/explain.py` (#1688)
- Fix broken test (#1685)
- Fix readme badges (#1682)
- Bump `aiohttp` from 3.8.3 to 3.8.5 (#1680)
- `pygments` from 2.14.0 to 2.15.0 (#1677)

11.37 Version 3.4.0

- Handle partial local echo. (#1675)
- clarify `handle_local_echo`. (#1674)
- `async_client`: add retries/reconnect. (#1672)
- Fix 3.11 problem. (#1673)
- Add new example simulator server/client. (#1671)
- `examples/contrib/explain.py` leveraging Rapid SCADA (#1665)
- `_logger` missed `basicConfig`. (#1670)
- Bug fix for #1662 (#1663)
- Bug fix for #1661 (#1664)
- Fix typo in `config.rst` (#1660)
- test `action_increment`. (#1659)
- test `codeql` (#1655)
- `mypy` complaints. (#1656)
- Remove `self.params` from `async client` (#1640)
- Drop test of `pypy` with `python 3.8`.
- repair `server_async.py` (#1644)
- move common `framer` to `base`. (#1639)
- Restrict `Return diag` call to `bytes`. (#1638)
- use `slave=` in `diag` requests. (#1636)
- `transport listen` in `server`. (#1628)
- CI test.
- Integrate `transport` in `server`. (#1617)
- fix `getFrameStart` for `ExceptionResponse` (#1627)
- Add `min/min` to simulator actions.

- Change to “sync client” in forwarder example (#1625)
- Remove docker (lack of maintenance). (#1623)
- Clean defaults (#1618)
- Reduce CI log with no debug. (#1616)
- prepare server to use transport. (#1607)
- Fix RemoteSlaveContext (#1599)
- Combine stale and lock. (#1608)
- update pytest + extensions. (#1610)
- Change version follow PEP 440. (#1609)
- Fix regression with REPL server not listening (#1604)
- Remove handler= for server classes. (#1602)
- Fix write function codes (#1598)
- transport nullmodem (#1591)
- move test of examples to subdirectory. (#1592)
- transport as object, not base class. (#1572)
- Simple examples. (#1590)
- transport_connect as bool. (#1587)
- Prepare dev (#1588)
- Release corrections. (#1586)

11.38 Version 3.3.2

- Fix RemoteSlaveContext (#1599)
- Change version follow PEP 440. (#1609)
- Fix regression with REPL server not listening (#1604)
- Fix write function codes (#1598)
- Release corrections. (#1586)

11.39 Version 3.3.1

- transport fixes and 100% test coverage. (#1580)
- Delay self.loop until connect(). (#1579)
- Added mechanism to determine if server did not start cleanly (#1539)
- Proof transport reconnect works. (#1577)
- Fix non-shared block doc in config.rst. (#1573)

11.40 Version 3.3.0

- Stabilize windows tests. (#1567)
- Bump mypy 1.3.0 (#1568)
- Transport integrated in async clients. (#1541)
- Client async corrections (due to 3.1.2) (#1565)
- Server_async[udp], solve 3.1.1 problem. (#1564)
- Remove ModbusTcpDiagClient. (#1560)
- Remove old method from Python2/3 transition (#1559)
- Switch to ruff's version of bandit (#1557)
- Allow reading/writing address 0 in the simulator (#1552)
- Remove references to "defer_start". (#1548)
- Client more robust against faulty response. (#1547)
- Fix missing package_data directives for simulator web (#1544)
- Fix installation instructions (#1543)
- Solve pytest timeout problem. (#1540)
- DiagnosticStatus encode missing tuple check. (#1533)
- test SparseDataStore. (#1532)
- BinaryPayloadBuilder.to_string to BinaryPayloadBuilder.encode (#1526)
- Adding flake8-pytest-style` to ruff (#1520)
- Simplify version management. (#1522)
- pylint and pre-commit autoupdate (#1519)
- Add type hint (#1512)
- Add action to lock issues/PR. (#1508)
- New common transport layer. (#1492)
- Solve serial close raise problem.
- Remove old config values (#1503)
- Document pymodbus.simulator. (#1502)
- Refactor REPL server to reduce complexity (#1499)
- Don't catch KeyboardInterrupt twice for REPL server (#1498)
- Refactor REPL client to reduce complexity (#1489)
- pymodbus.server: listen on ID 1 by default (#1496)
- Clean framer/___init___py (#1494)
- Duplicate transactions in UDP. (#1486)
- clean ProcessIncommingPacket. (#1491)
- Enable pyupgrade (U) rules in ruff (#1484)
- clean_workflow.yaml solve parameter problem.

- Correct wrong import in test. (#1483)
- Implement pyflakes-simplify (#1480)
- Test case for UDP duplicate msg issue (#1470)
- Test of write_coil. (#1479)
- Test reuse of client object. (#1475)
- Comment about addressing when shared=false (#1474)
- Remove old aliases to OSError (#1473)
- pymodbus.simulator fixes (#1463)
- Fix wrong error message with pymodbus console (#1456)
- update modbusrtuframer (#1435)
- Server multidrop test.: (#1451)
- mypy problem ModbusResponse.

11.41 Version 3.2.2

- Add forgotten await

11.42 Version 3.2.1

- add missing server.start(). (#1443)
- Don't publish univerval (Python2 / Python 3) wheels (#1423)
- Remove unnecessary custom LOG_LEVEL check (#1424)
- Include py.typed in package (#1422)

11.43 Version 3.2.0

- Add value <-> registers converter helpers. (#1413)
- Add pre-commit config (#1406)
- Make baud rate configurable for examples (#1410)
- Clean __init__ and update log module. (#1411)
- Simulator add calls functionality. (#1390)
- Add note about not being thread safe. (#1404)
- Update docker-publish.yml
- Forward retry_on_empty and retries by calling transaction (#1401)
- serial sync recv interval (#1389)
- Add tests for writing multiple writes with a single value (#1402)
- Enable mypy in CI (#1388)
- Limit use of Singleton. (#1397)
- Cleanup interfaces (#1396)

- Add request names. (#1391)
- Simulator, register look and feel. (#1387)
- Fix enum for REPL server (#1384)
- Remove unneeded attribute (#1383)
- Fix mypy errors in reactive server (#1381)
- remove nose (#1379)
- Fix type hints for http_server (#1369)
- Merge pull request #1380 from pymodbus-dev/requirements
- remove second client instance in async mode. (#1367)
- Pin setuptools to prevent breakage with Version including “X” (#1373)
- Lint and type hints for REPL (#1364)
- Clean mixin execute (#1366)
- Remove unused setup_commands.py. (#1362)
- Run black on top-level files and /doc (#1361)
- repl config path (#1359)
- Fix NoReponse -> NoResponse (#1358)
- Make whole main async. (#1355)
- Fix more typing issues (#1351)
- Test sync task (#1341)
- Fixed text in ModbusClientMixin’s writes (#1352)
- lint /doc (#1345)
- Remove unused linters (#1344)
- Allow log level as string or integer. (#1343)
- Sync serial, clean recv. (#1340)
- Test server task, async completed (#1318)
- main() should be sync (#1339)
- Bug: Fixed caused by passing wrong arg (#1336)

11.44 Version 3.1.3

- Solve log problem in payload.
- Fix register type check for size bigger than 3 registers (6 bytes) (#1323)
- Re-add SQL tests. (#1329)
- Central logging. (#1324)
- Skip sqlalchemy test. (#1325)
- Solve 1319 (#1320)

11.45 Version 3.1.2

- Update README.rst
- Correct README link. (#1316)
- More direct readme links for REPL (#1314)
- Add classifier for 3.11 (#1312)
- Update README.rst (#1313)
- Delete ModbusCommonBlock.png (#1311)
- Add modbus standard to README. (#1308)
- fix no auto reconnect after close/connect in TCPclient (#1298)
- Update examples.rst (#1307)
- var name clarification (#1304)
- Bump external libraries. (#1302)
- Reorganize documentation to make it easier accessible (#1299)
- Simulator documentation (first version). (#1296)
- Updated datastore Simulator. (#1255)
- Update links to pydmodbus-dev (#1291)
- Change riptideio to pymodbus-dev. (#1292)
- #1258 Avoid showing unit as a seperate command line argument (#1288)
- Solve docker cache problem. (#1287)

11.46 Version 3.1.1

- add missing server.start() (#1282)
- small performance improvement on debug log (#1279)
- Fix Unix sockets parsing (#1281)
- client: Allow unix domain socket. (#1274)
- transfer timeout to protocol object. (#1275)
- Add ModbusUnixServer / StartAsyncUnixServer. (#1273)
- Added return in AsyncModbusSerialClient.connect (#1271)
- add connect() to the very first example (#1270)
- Solve docker problem. (#1268)
- Test stop of server task. (#1256)

11.47 Version 3.1.0

- Add xdist pr default. (#1253)
- Create docker-publish.yml (#1250)

- Parallelize pytest with pytest-xdist (#1247)
- Support Python3.11 (#1246)
- Fix reconnectDelay to be within (100ms, 5min) (#1244)
- Fix typos in comments (#1233)
- WEB simulator, first version. (#1226)
- Clean async serial problem. (#1235)
- terminate when using 'randomize' and 'change_rate' at the same time (#1231)
- Used tooled python and OS (#1232)
- add 'change_rate' randomization option (#1229)
- add check_ci.sh (#1225)
- Simplify CI and use cache. (#1217)
- Solve issue 1210, update simulator (#1211)
- Add missing client calls in mixin.py. (#1206)
- Advanced simulator with cross memory. (#1195)
- AsyncModbusTcp/UdpClient honors delay_ms == 0 (#1203) (#1205)
- Fix #1188 and some pylint issues (#1189)
- Serial receive incomplete bytes.issue #1183 (#1185)
- Handle echo (#1186)
- Add updating server example. (#1176)

11.48 Version 3.0.2

- Add pygments as requirement for repl
- Update datastore remote to handle write requests (#1166)
- Allow multiple servers. (#1164)
- Fix typo. (#1162)
- Transfer parms. to connected client. (#1161)
- Repl enhancements 2 (#1141)
- Server simulator with datastore with json data. (#1157)
- Avoid unwanted reconnects (#1154)
- Do not initialize framer twice. (#1153)
- Allow timeout as float. (#1152)
- Improve Docker Support (#1145)
- Fix unreachable code in AsyncModbusTcpClient (#1151)
- Fix type hints for port and timeout (#1147)
- Start/stop multiple servers. (#1138)
- Server/asyncio.py correct logging when disconnecting the socket (#1135)

- Add Docker and container registry support (#1132)
- Removes undue reported error when forwarding (#1134)
- Obey timeout parameter on connection (#1131)
- Readme typos (#1129)
- Clean noqa directive. (#1125)
- Add isort and activate CI fail for black/isort. (#1124)
- Update examples. (#1117)
- Move logging configuration behind function call (#1120)
- serial2TCP forwarding example (#1116)
- Make serial import dynamic. (#1114)
- Bugfix ModbusSerialServer setup so handler is called correctly. (#1113)
- Clean configurations. (#1111)

11.49 Version 3.0.1

- Faulty release!

11.50 Version 3.0.0

- Solve multiple incoming frames. (#1107)
- Up coverage, tests are 100%. (#1098)
- Prepare for rc1. (#1097)
- Prepare 3.0.0dev5 (#1095)
- Adapt serial tests. (#1094)
- Allow windows. (#1093)
- Remove server sync code and combine with async code. (#1092)
- Solve test of tls by adding certificates and remove bugs (#1080)
- Simplify server implementation. (#1071)
- Do not filter using unit id in the received response (#1076)
- Hex values for repl arguments (#1075)
- All parameters in class parameter. (#1070)
- Add len parameter to decode_bits. (#1062)
- New combined test for all types of clients. (#1061)
- Dev mixin client (#1056)
- Add/update client documentation, including docstrings etc. (#1055)
- Add unit to arguments (#1041)
- Add timeout to all pytest. (#1037)
- Simplify client parent classes. (#1018)

- Clean copyright statements, to ensure we follow FOSS rules. (#1014)
- Rectify sync/async client parameters. (#1013)
- Clean client directory structure for async. (#1010)
- Remove async_io, simplify AsyncModbus<x>Client. (#1009)
- remove init_<something>_client(). (#1008)
- Remove async factory. (#1001)
- Remove loop parameter from client/server (#999)
- add example async client. (#997)
- Change async ModbusSerialClient to framer= from method=. (#994)
- Add forwarder example with multiple slaves. (#992)
- Remove async get_factory. (#990)
- Remove unused ModbusAccessControl. (#989)
- Solve problem with remote datastore. (#988)
- Remove unused schedulers. (#976)
- Remove twisted (#972)
- Remove/Update tornado/twister tests. (#971)
- remove easy_install and ez_setup (#964)
- Fix mask write register (#961)
- Activate pytest-asyncio. (#949)
- Changed default framer for serial to be ModbusRtuFramer. (#948)
- Remove tornado. (#935)
- Pylint, check method parameter documentation. (#909)
- Add get_response_pdu_size to mask read/write. (#922)
- Minimum python version is 3.8. (#921)
- Ensure make doc fails on warnings and/or errors. (#920)
- Remove central makefile. (#916)
- Re-organize examples (#914)
- Documentation cleanup and clarification (#689)
- Update doc for repl. (#910)
- Include package and tests in coverage measurement (#912)
- Use response byte length if available (#880)
- better fix for rtu incomplete frames (#511)
- Remove twisted/tornado from doc. (#904)
- Update classifiers for pypi. (#907)
- Documentation updates
- PEP8 compatible code

- More tooling and CI updates
- Remove python2 compatibility code (#564)
- Remove Python2 checks and Python2 code snippets
- Misc co-routines related fixes
- Fix CI for python3 and remove PyPI from CI
- Fix mask_write_register call. (#685)
- Add support for byte strings in the device information fields (#693)
- Catch socket going away. (#722)
- Misc typo errors (#718)
- Support python3.10
- Implement asyncio ModbusSerialServer
- ModbusTLS updates (tls handshake, default framer)
- Support broadcast messages with asyncio client
- Fix for lazy loading serial module with asyncio clients.
- Updated examples and tests
- Support python3.7 and above
- Support creating asyncio clients from within coroutines.

11.51 Version 2.5.3

- Fix retries on tcp client failing randomly.
- Fix Asyncio client timeout arg not being used.
- Treat exception codes as valid responses
- Fix examples (modbus_payload)
- Add missing identity argument to async ModbusSerialServer

11.52 Version 2.5.2

- Add kwarg *reset_socket* to control closing of the socket on read failures (set to *True* by default).
- Add *-reset-socket/-no-reset-socket* to REPL client.

11.53 Version 2.5.1

- Bug fix TCP Repl server.
- Support multiple UID's with REPL server.
- Support serial for URL (sync serial client)
- Bug fix/enhancements, close socket connections only on empty or invalid response

11.54 Version 2.5.0

- Support response types *stray* and *empty* in repl server.
- Minor updates in asyncio server.
- Update reactive server to send stray response of given length.
- Transaction manager updates on retries for empty and invalid packets.
- Test fixes for asyncio client and transaction manager.
- Fix sync client and processing of incomplete frames with rtu framers
- Support synchronous diagnostic client (TCP)
- Server updates (REPL and async)
- Handle Memory leak in sync servers due to socketserver memory leak
- Minor fix in documentations
- Travis fix for Mac OSX
- Disable unnecessary deprecation warning while using async clients.
- Use Github actions for builds in favor of travis.
- Documentation updates
- Disable *strict* mode by default.
- Fix *ReportSlaveIdRequest* request
- Sparse datablock initialization updates.
- Support REPL for modbus server (only python3 and asyncio)
- Fix REPL client for write requests
- Fix examples
- Asyncio server
- Asynchronous server (with custom datablock)
- Fix version info for servers
- Fix and enhancements to Tornado clients (serial and tcp)
- Fix and enhancements to Asyncio client and server
- Update Install instructions
- Synchronous client retry on empty and error enhancements
- Add new modbus state *RETRYING*
- Support runtime response manipulations for Servers
- Bug fixes with logging module in servers
- Asyncio modbus serial server support

11.55 Version 2.4.0

- Support async modbus tls server/client
- Add local echo option
- Add exponential backoffs on retries.
- REPL - Support broadcasts.
- Fix framers using wrong unit address.
- Update documentation for serial_forwarder example
- Fix error with rtu client for *local_echo*
- Fix asyncio client not working with already running loop
- Fix passing serial arguments to async clients
- Support timeouts to break out of response await when server goes offline
- Misc updates and bugfixes.

11.56 Version 2.3.0

- Support Modbus TLS (client / server)
- Distribute license with source
- BinaryPayloadDecoder/Encoder now supports float16 on python3.6 and above
- Fix asyncio UDP client/server
- Minor cosmetic updates
- Asyncio Server implementation (Python 3.7 and above only)
- Bug fix for DiagnosticStatusResponse when odd sized response is received
- Remove Pycrypto from dependencies and include cryptodome instead
- Remove *SIX* requirement pinned to exact version.
- Minor bug-fixes in documentations.

11.57 Version 2.2.0

- Support Python 3.7
- Fix to task cancellations and CRC errors for async serial clients.
- Fix passing serial settings to asynchronous serial server.
- Fix *AttributeError* when setting *interCharTimeout* for serial clients.
- Provide an option to disable inter char timeouts with Modbus RTU.
- Add support to register custom requests in clients and server instances.
- Fix read timeout calculation in ModbusTCP.
- Fix *SQLDbcontext* always returning *InvalidAddress* error.
- Fix *SQLDbcontext* update failure

- Fix Binary payload example for endianness.
- Fix BinaryPayloadDecoder.to_coils and BinaryPayloadBuilder.fromCoils methods.
- Fix tornado async serial client *TypeError* while processing incoming packet.
- Fix erroneous CRC handling in Modbus RTU framer.
- Support broadcasting in Modbus Client and Servers (sync).
- Fix asyncio examples.
- Improved logging in Modbus Server .
- ReportSlaveIdRequest would fetch information from Device identity instead of hardcoded *Pymodbus*.
- Fix regression introduced in 2.2.0rc2 (Modbus sync client transaction failing)
- Minor update in factory.py, now server logs prints received request instead of only function code

11.58 Version 2.1.0

- Fix Issues with Serial client where in partial data was read when the response size is unknown.
- Fix Infinite sleep loop in RTU Framer.
- Add pygments as extra requirement for repl.
- Add support to modify modbus client attributes via repl.
- Update modbus repl documentation.
- More verbose logs for repl.

11.59 Version 2.0.1

- Fix unicode decoder error with BinaryPayloadDecoder in some platforms
- Avoid unnecessary import of deprecated modules with dependencies on twisted

11.60 Version 2.0.0

- Async client implementation based on Tornado, Twisted and asyncio with backward compatibility support for twisted client.
- Allow reusing existing[running] asyncio loop when creating async client based on asyncio.
- Allow reusing address for Modbus TCP sync server.
- Add support to install tornado as extra requirement while installing pymodbus.
- Support Pymodbus REPL
- Add support to python 3.7.
- Bug fix and enhancements in examples.
- Async client implementation based on Tornado, Twisted and asyncio

11.61 Version 1.5.2

- Fix serial client *is_socket_open* method

11.62 Version 1.5.1

- Fix device information selectors
- Fixed behaviour of the MEI device information command as a server when an invalid object_id is provided by an external client.
- Add support for repeated MEI device information Object IDs (client/server)
- Added support for encoding device information when it requires more than one PDU to pack.
- Added REPR statements for all synchronous clients
- Added *isError* method to exceptions, Any response received can be tested for success before proceeding.
- Add examples for MEI read device information request

11.63 Version 1.5.0

- Improve transaction speeds for sync clients (RTU/ASCII), now retry on empty happens only when *retry_on_empty* kwarg is passed to client during initialization
- Fix tcp servers (sync/async) not processing requests with transaction id > 255
- Introduce new api to check if the received response is an error or not (*response.isError()*)
- Move timing logic to framers so that irrespective of client, correct timing logics are followed.
- Move framers from *transaction.py* to respective modules
- Fix modbus payload builder and decoder
- Async servers can now have an option to defer *reactor.run()* when using *Start<Tcp/Serial/Udo>Server(...,defer_reactor_run=True)*
- Fix UDP client issue while handling MEI messages (*ReadDeviceInformationRequest*)
- Add expected response lengths for *WriteMultipleCoilRequest* and *WriteMultipleRegisterRequest*
- Fix *_rtu_byte_count_pos* for *GetCommEventLogResponse*
- Add support for repeated MEI device information Object IDs
- Fix struct errors while decoding stray response
- Modbus read retries works only when empty/no message is received
- Change test runner from *nosetest* to *pytest*
- Fix Misc examples

11.64 Version 1.4.0

- Bug fix Modbus TCP client reading incomplete data
- Check for slave unit id before processing the request for serial clients
- Bug fix serial servers with Modbus Binary Framer

- Bug fix header size for ModbusBinaryFramer
- Bug fix payload decoder with endian Little
- Payload builder and decoder can now deal with the wordorder as well of 32/64 bit data.
- Support Database slave contexts (SqlStore and RedisStore)
- Custom handlers could be passed to Modbus TCP servers
- Asynchronous Server could now be stopped when running on a seperate thread (StopServer)
- Signal handlers on Asynchronous servers are now handled based on current thread
- Registers in Database datastore could now be read from remote clients
- Fix examples in contrib (message_parser.py/message_generator.py/remote_server_context)
- Add new example for SqlStore and RedisStore (db store slave context)
- Fix minor comaptibility issues with utilities.
- Update test requirements
- Update/Add new unit tests
- Move twisted requirements to extra so that it is not installed by default on pymodbus installtion

11.65 Version 1.3.2

- ModbusSerialServer could now be stopped when running on a seperate thread.
- Fix issue with server and client where in the frame buffer had values from previous unsuccesful transaction
- Fix response length calculation for ModbusASCII protocol
- Fix response length calculation ReportSlaveIdResponse, DiagnosticStatusResponse
- Fix never ending transaction case when response is received without header and CRC
- Fix tests

11.66 Version 1.3.1

- Recall socket recv until get a complete response
- Register_write_message.py: Observe skip_encode option when encoding a single register request
- Fix wrong expected response length for coils and discrete inputs
- Fix decode errors with ReadDeviceInformationRequest and ReportSlaveIdRequest on Python3
- Move MaskWriteRegisterRequest/MaskWriteRegisterResponse to register_write_message.py from file_message.py
- Python3 compatible examples [WIP]
- Misc updates with examples
- Fix encoding problem for ReadDeviceInformationRequest method on python3
- Fix problem with the usage of ord in python3 while cleaning up receive buffer
- Fix struct unpack errors with BinaryPayloadDecoder on python3 - string vs bytestring error
- Calculate expected response size for ReadWriteMultipleRegistersRequest

- Enhancement for ModbusTcpClient, ModbusTcpClient can now accept connection timeout as one of the parameter
- Misc updates
- Timing improvements over MODBUS Serial interface
- Modbus RTU use 3.5 char silence before and after transactions
- Bug fix on FifoTransactionManager , flush stray data before transaction
- Update repository information
- Added ability to ignore missing slaves
- Added ability to revert to ZeroMode
- Passed a number of extra options through the stack
- Fixed documentation and added a number of examples

11.67 Version 1.2.0

- Reworking the transaction managers to be more explicit and to handle modbus RTU over TCP.
- Adding examples for a number of unique requested use cases
- Allow RTU framers to fail fast instead of staying at fault
- Working on datastore saving and loading

11.68 Version 1.1.0

- Fixing memory leak in clients and servers (removed `__del__`)
- Adding the ability to override the client framers
- Working on web page api and GUI
- Moving examples and extra code to contrib sections
- Adding more documentation

11.69 Version 1.0.0

- Adding support for payload builders to form complex encoding and decoding of messages.
- Adding BCD and binary payload builders
- Adding support for pydev
- Cleaning up the build tools
- Adding a message encoding generator for testing.
- Now passing kwargs to base of PDU so arguments can be used correctly at all levels of the protocol.
- A number of bug fixes (see bug tracker and commit messages)

PYMODBUS INTERNALS

12.1 NullModem

Pymodbus offers a special NullModem transport to help end-to-end test without network.

The NullModem is activated by setting host= (port= for serial) to NULLMODEM_HOST (import pymodbus.transport)

The NullModem works with the normal transport types, and simply substitutes the physical connection: - *Serial* (RS-485) typically using a dongle - *TCP* - *TLS* - *UDP*

The NullModem is currently integrated in - *Modbus<x>Client* - *AsyncModbus<x>Client* - *Modbus<x>Server* - *AsyncModbus<x>Server*

Of course the NullModem requires that server and client(s) run in the same python instance.

12.2 Datastore

Datastore is responsible for managing registers for a server.

12.2.1 Datastore classes

class pymodbus.datastore.ModbusSparseDataBlock(values=None, mutable=True)

A sparse modbus datastore.

E.g Usage. sparse = ModbusSparseDataBlock({10: [3, 5, 6, 8], 30: 1, 40: [0]*20})

This would create a datablock with 3 blocks One starts at offset 10 with length 4, one at 30 with length 1, and one at 40 with length 20

sparse = ModbusSparseDataBlock([10]*100) Creates a sparse datablock of length 100 starting at offset 0 and default value of 10

sparse = ModbusSparseDataBlock() -> Create empty datablock sparse.setValues(0, [10]*10) -> Add block 1 at offset 0 with length 10 (default value 10) sparse.setValues(30, [20]*5) -> Add block 2 at offset 30 with length 5 (default value 20)

Unless 'mutable' is set to True during initialization, the datablock cannot be altered with setValues (new datablocks cannot be added)

classmethod create(values=None)

Create sparse datastore.

Use setValues to initialize registers.

Parameters

values – Either a list or a dictionary of values

Returns

An initialized datastore

reset()

Reset the store to the initially provided defaults.

getValues(*address*, *count=1*) → list[int] | list[bool] | ExcCodes

Return the requested values of the datastore.

Parameters

- **address** – The starting address
- **count** – The number of values to retrieve

Returns

The requested values from a:a+c

setValues(*address*, *values*, *use_as_default=False*) → None | ExcCodes

Set the requested values of the datastore.

Parameters

- **address** – The starting address
- **values** – The new values to be set
- **use_as_default** – Use the values as default

Raises

ParameterException –

```
class pymodbus.datastore.ModbusDeviceContext(*_args, di: BaseModbusDataBlock | None = None, co:
                                             BaseModbusDataBlock | None = None, ir:
                                             BaseModbusDataBlock | None = None, hr:
                                             BaseModbusDataBlock | None = None)
```

Create a modbus data model with data stored in a block.

Parameters

- **di** – discrete inputs initializer ModbusDataBlock
- **co** – coils initializer ModbusDataBlock
- **hr** – holding register initializer ModbusDataBlock
- **ir** – input registers initializer ModbusDataBlock

reset()

Reset all the datastores to their default values.

getValues(*func_code*, *address*, *count=1*) → list[int] | list[bool] | ExcCodes

Get *count* values from datastore.

Parameters

- **func_code** – The function we are working with
- **address** – The starting address
- **count** – The number of values to retrieve

Returns

The requested values from a:a+c

setValues(*func_code, address, values*) → None | ExcCodes

Set the datastore with the supplied values.

Parameters

- **func_code** – The function we are working with
- **address** – The starting address
- **values** – The new values to be set

register(*function_code, func_code, datablock=None*)

Register a datablock with the device context.

Parameters

- **function_code** – function code (int)
- **func_code** – string representation of function code (e.g “cf”)
- **datablock** – datablock to associate with this function code

class pymodbus.datastore.**ModbusServerContext**(*devices=None, single=True*)

This represents a master collection of device contexts.

If single is set to true, it will be treated as a single context so every device id returns the same context. If single is set to false, it will be interpreted as a collection of device contexts.

device_ids()

Define device_ids.

class pymodbus.datastore.**ModbusSimulatorContext**(*config: dict[str, Any], custom_actions: dict[str, Callable] | None*)

Modbus simulator.

Parameters

- **config** – A dict with structure as shown below.
- **actions** – A dict with “<name>”: <function> structure.

Raises

RuntimeError – if json contains errors (msg explains what)

It builds and maintains a virtual copy of a device, with simulation of device specific functions.

The device is described in a dict, user supplied actions will be added to the builtin actions.

It is used in conjunction with a pymodbus server.

Example:

```
store = ModbusSimulatorContext(<config dict>, <actions dict>)
StartAsyncTcpServer(<host>, context=store)
```

Now the server will simulate the defined device **with** features like:

- invalid addresses
- write protected addresses
- optional control of access **for** string, uint32, bit/bits
- builtin actions **for** e.g. reset/datetime, value increment by read
- custom actions

Description of the json file or dict to be supplied:

```
{
  "setup": {
    "di size": 0, --> Size of discrete input block (8 bit)
    "co size": 0, --> Size of coils block (8 bit)
    "ir size": 0, --> Size of input registers block (16 bit)
    "hr size": 0, --> Size of holding registers block (16 bit)
    "shared blocks": True, --> share memory for all blocks (largest size wins)
    "defaults": {
      "value": { --> Initial values (can be overwritten)
        "bits": 0x01,
        "uint16": 122,
        "uint32": 67000,
        "float32": 127.4,
        "string": " ",
      },
      "action": { --> default action (can be overwritten)
        "bits": None,
        "uint16": None,
        "uint32": None,
        "float32": None,
        "string": None,
      },
    },
    "type exception": False, --> return IO exception if read/write on non_
    ↪boundary
  },
  "invalid": [ --> List of invalid addresses, IO exception returned
    51, --> single register
    [78, 99], --> start, end registers, repeated as needed
  ],
  "write": [ --> allow write, default is ReadOnly
    [5, 5] --> start, end bytes, repeated as needed
  ],
  "bits": [ --> Define bits (1 register == 2 bytes)
    [30, 31], --> start, end registers, repeated as needed
    {"addr": [32, 34], "value": 0xF1}, --> with value
    {"addr": [35, 36], "action": "increment"}, --> with action
    {"addr": [37, 38], "action": "increment", "value": 0xF1} --> with action_
    ↪and value
    {"addr": [37, 38], "action": "increment", "parameters": {"min": 0, "max":_
    ↪100}} --> with action with arguments
  ],
  "uint16": [ --> Define uint16 (1 register == 2 bytes)
    --> same as type_bits
  ],
  "uint32": [ --> Define 32 bit integers (2 registers == 4 bytes)
    --> same as type_bits
  ],
  "float32": [ --> Define 32 bit floats (2 registers == 4 bytes)
    --> same as type_bits
  ],
}
```

(continues on next page)

(continued from previous page)

```

"string": [ --> Define strings (variable number of registers (each 2 bytes))
    [21, 22], --> start, end registers, define 1 string
    {"addr": 23, 25], "value": "ups"}, --> with value
    {"addr": 26, 27], "action": "user"}, --> with action
    {"addr": 28, 29], "action": "", "value": "user"} --> with action and value
],
"repeat": [ --> allows to repeat section e.g. for n devices
    {"addr": [100, 200], "to": [50, 275]} --> Repeat registers 100-200 to 50+
    until 275
]
}

```

get_text_register(*register*)

Get raw register.

classmethod build_registers_from_value(*value, is_int*)

Build registers from int32 or float32.

classmethod build_value_from_registers(*registers, is_int*)

Build int32 or float32 value from registers.

12.3 Framer

12.3.1 pymodbus.framer.FramerAscii module

Modbus ASCII Frame Controller.

Layout::

```

[ Start ][ Dev id ][ Function ][ Data ][ LRC ][ End ]
  1c 2c 2c N*2c 1c 2c

```

- data can be 1 - 2x252 chars
- end is “\r\n” (Carriage return line feed), however the line feed character can be changed via a special command
- start is “:”

This framer is used for serial transmission. Unlike the RTU protocol, the data in this framer is transferred in plain text ascii.

12.3.2 pymodbus.framer.FramerRTU module

Modbus RTU frame type.

Layout:

```

[ Start Wait ] [Address ] [ Function Code] [ Data ] [ CRC ]
  3.5 chars    1b         1b              Nb      2b

```

Note

due to the USB converter and the OS drivers, timing cannot be guaranteed neither when receiving nor when sending.

Decoding is a complicated process because the RTU frame does not have a fixed prefix only suffix, therefore it is necessary to decode the content of the frame to get length etc. There are some protocol restrictions that help with the detection.

For client:

- a request causes 1 response !
- Multiple requests are NOT allowed (master controlled protocol)
- the server will not retransmit responses

this means decoding is always exactly 1 frame (response)

For server (Single device)

- only 1 request allowed (master controlled protocol)
- the client (master) may retransmit but in larger time intervals

this means decoding is always exactly 1 frame (request)

For server (Multidrop line → devices in parallel)

- only 1 request allowed (master controlled protocol)
- other devices will send responses
- the client (master) may retransmit but in larger time intervals

this means decoding is always exactly 1 frame request, however some requests will be for unknown devices, which must be ignored together with the response from the unknown device.

Recovery from bad cabling and unstable USB etc is important, the following scenarios is possible:

- garble data before frame
- garble data in frame
- garble data after frame
- data in frame garbled (wrong CRC)

decoding assumes the frame is sound, and if not enters a hunting mode.

The 3.5 byte transmission time at the slowest speed 1.200Bps is 31ms. Device drivers will typically flush buffer after 10ms of silence. If no data is received for 50ms the transmission / frame can be considered complete.

The following table is a listing of the baud wait times for the specified baud rates:

Baud	1.5c (18 bits)	3.5c (38 bits)
1200	13333.3 us	31666.7 us
4800	3333.3 us	7916.7 us
9600	1666.7 us	3958.3 us
19200	833.3 us	1979.2 us
38400	416.7 us	989.6 us

1 Byte = start + 8 bits + parity + stop = 11 bits
(1/Baud)(bits) = delay seconds

⚠ Danger

Current framerRTU does not support running the server on a multipoint rs485 line.

12.3.3 pymodbus.framer.FramerSocket module

Modbus Socket frame type.

Layout:

[MBAP Header]				[Function Code]	[Data]
[tid]	[pid]	[length]	[uid]		
2b	2b	2b	1b	1b	Nb

length = uid + function code + data

12.3.4 pymodbus.framer.FramerTLS module

Modbus TLS frame type.

Layout:

[MBAP Header]				[Function Code]	[Data]
[tid]	[pid]	[length]	[uid]		
2b	2b	2b	1b	1b	Nb

length = uid + function code + data

12.4 Constants

Constants For Modbus Server/Client.

This is the single location for storing default values for the servers and clients.

class pymodbus.constants.DataType(value)

Bases: IntEnum

Register types, used to define of a group of registers.

This is the types pymodbus recognizes, actually the modbus standard do NOT define e.g. INT32, but since nearly every device contain e.g. values of type INT32, it is available in pymodbus, with automatic conversions to/from registers.

BITS = 10

16 bits == 1 register

FLOAT32 = 7

1 float == 2 registers

FLOAT64 = 8

1 float == 4 registers

INT16 = 1

1 integer == 1 register

INT32 = 3

1 integer == 2 registers

INT64 = 5

1 integer == 4 registers

REGISTERS = 11

Registers == 2 bytes (identical to UINT16)

STRING = 9

1 string == (len(string) / 2) registers

UINT16 = 2

1 positive integer == 1 register

UINT32 = 4

1 positive integer == 2 registers

UINT64 = 6

1 positive integer == 4 register

class pymodbus.constants.DeviceInformation(value)

Bases: IntEnum

Represents what type of device information to read.

BASIC

This is the basic (required) device information to be returned. This includes VendorName, ProductCode, and MajorMinorRevision code.

REGULAR

In addition to basic data objects, the device provides additional and optional identification and description data objects. All of the objects of this category are defined in the standard but their implementation is optional.

EXTENDED

In addition to regular data objects, the device provides additional and optional identification and description private data about the physical device itself. All of these data are device dependent.

SPECIFIC

Request to return a single data object.

BASIC = 1

EXTENDED = 3

REGULAR = 2

SPECIFIC = 4

class pymodbus.constants.ExcCodes(value)

Bases: IntEnum

Represents the allowed exception codes.

ACKNOWLEDGE = 5

DEVICE_BUSY = 6

DEVICE_FAILURE = 4

GATEWAY_NO_RESPONSE = 11

GATEWAY_PATH_UNAVIABLE = 10

ILLEGAL_ADDRESS = 2

ILLEGAL_FUNCTION = 1

ILLEGAL_VALUE = 3

MEMORY_PARITY_ERROR = 8

NEGATIVE_ACKNOWLEDGE = 7

class pymodbus.constants.ModbusPlusOperation(value)

Bases: IntEnum

Represents the type of modbus plus request.

GET_STATISTICS

Operation requesting that the current modbus plus statistics be returned in the response.

CLEAR_STATISTICS

Operation requesting that the current modbus plus statistics be cleared and not returned in the response.

CLEAR_STATISTICS = 4

GET_STATISTICS = 3

class pymodbus.constants.ModbusStatus(value)

Bases: IntEnum

These represent various status codes in the modbus protocol.

WAITING

This indicates that a modbus device is currently waiting for a given request to finish some running task.

READY

This indicates that a modbus device is currently free to perform the next request task.

ON

This indicates that the given modbus entity is on

OFF

This indicates that the given modbus entity is off

OFF = 0

ON = 65280

READY = 0

WAITING = 65535

class pymodbus.constants.MoreData(value)

Bases: IntEnum

Represents the more follows condition.

NOTHING

This indicates that no more objects are going to be returned.

KEEP_READING

This indicates that there are more objects to be returned.

KEEP_READING = 255

NOTHING = 0

12.5 Extra functions

Pymodbus: Modbus Protocol Implementation.

Released under the BSD license

```
class pymodbus.ExceptionResponse(function_code: int, exception_code: int = 0, device_id: int = 1,
                                transaction: int = 0)
```

Bases: ModbusPDU

Base class for a modbus exception PDU.

decode(data: bytes) → None

Decode a modbus exception response.

encode() → bytes

Encode a modbus exception response.

rtu_frame_size: int = 5

```
class pymodbus.FramerType(value)
```

Bases: str, Enum

Type of Modbus frame.

ASCII = 'ascii'

RTU = 'rtu'

SOCKET = 'socket'

TLS = 'tls'

```
class pymodbus.ModbusDeviceIdentification(info=None, info_name=None)
```

Bases: object

This is used to supply the device identification.

For the readDeviceIdentification function

For more information read section 6.21 of the modbus application protocol.

property MajorMinorRevision

property ModelName

property ProductCode

property ProductName

property `UserApplicationName`

property `VendorName`

property `VendorUrl`

summary()

Return a summary of the main items.

Returns

An dictionary of the main items

update(value)

Update the values of this identity.

using another identify as the value

Parameters

value – The value to copy values from

exception `pymodbus.ModbusException(string)`

Bases: `Exception`

Base modbus exception.

isError()

Error

`pymodbus.pymodbus_apply_logging_config(level: str | int = 10, log_file_name: str | None = None)`

Apply basic logging configuration used by default by Pymodbus maintainers.

Parameters

- **level** – (optional) set log level, if not set it is inherited.
- **log_file_name** – (optional) log additional to file

Please call this function to format logging appropriately when opening issues.

Pymodbus Exceptions.

Custom exceptions to be used in the Modbus code.

exception `pymodbus.exceptions.ConnectionException(string="")`

Bases: `ModbusException`

Error resulting from a bad connection.

exception `pymodbus.exceptions.MessageRegisterException(string="")`

Bases: `ModbusException`

Error resulting from failing to register a custom message request/response.

exception `pymodbus.exceptions.ModbusIOException(string="", function_code=None)`

Bases: `ModbusException`

Error resulting from data i/o.

exception `pymodbus.exceptions.NoSuchIdException(string="")`

Bases: `ModbusException`

Error resulting from making a request to a id that does not exist.

exception pymodbus.exceptions.**NotImplementedException**(string="")

Bases: [ModbusException](#)

Error resulting from not implemented function.

exception pymodbus.exceptions.**ParameterException**(string="")

Bases: [ModbusException](#)

Error resulting from invalid parameter.

Transaction.

class pymodbus.transaction.**TransactionManager**(params: *CommParams*, framer: *FramerBase*, retries: *int*, is_server: *bool*, trace_packet: *Callable[[bool, bytes], bytes] | None*, trace_pdu: *Callable[[bool, ModbusPDU], ModbusPDU] | None*, trace_connect: *Callable[[bool], None] | None*, sync_client=None)

Bases: [ModbusProtocol](#)

Transaction manager.

This is the central class of the library, providing a separation between API and communication: - clients/servers calls the manager to execute requests/responses - transport/framer/pdu is by the manager to communicate with the devices

Transaction manager handles: - Execution of requests (client), with retries and locking - Sending of responses (server), with retries - Connection management (on top of what transport offers) - No response (temporarily) from a device

Transaction manager offers: - a simple execute interface for requests (client) - a simple send interface for responses (server) - external trace methods tracing outgoing/incoming packets/PDUs (byte stream)

callback_connected() → None

Call when connection is successful.

callback_data(data: *bytes*, addr: *tuple | None = None*) → int

Handle received data.

callback_disconnected(exc: *Exception | None*) → None

Call when connection is lost.

callback_new_connection()

Call when listener receive new connection request.

dummy_trace_connect(connect: *bool*) → None

Do dummy trace.

dummy_trace_packet(sending: *bool*, data: *bytes*) → bytes

Do dummy trace.

dummy_trace_pdu(sending: *bool*, pdu: *ModbusPDU*) → *ModbusPDU*

Do dummy trace.

async execute(no_response_expected: *bool*, request: *ModbusPDU*) → *ModbusPDU*

Execute requests asynchronously.

REMARK: this method is identical to sync_execute, apart from the lock and try/except.
any changes in either method MUST be mirrored !!!

getNextTID() → int

Retrieve the next transaction identifier.

pdu_send(*pdu: ModbusPDU*, *addr: tuple | None = None*) → None

Build byte stream and send.

sync_execute(*no_response_expected: bool*, *request: ModbusPDU*) → ModbusPDU

Execute requests asynchronously.

REMARK: this method is identical to execute, apart from the lock and sync_receive.
any changes in either method MUST be mirrored !!!

sync_get_response(*dev_id, tid*) → ModbusPDU

Receive until PDU is correct or timeout.

Modbus Utilities.

A collection of utilities for packing data, unpacking data computing checksums, and decode checksums.

pymodbus.utilities.dict_property(*store, index*)

Create class properties from a dictionary.

Basically this allows you to remove a lot of possible boilerplate code.

Parameters

- **store** – The store store to pull from
- **index** – The index into the store to close over

Returns

An initialized property set

pymodbus.utilities.hexlify_packets(*packet*)

Return hex representation of bytearray received.

Parameters

packet

Returns

12.6 PDU classes

Modbus Request/Response Decoders.

class pymodbus.pdu.decoders.**DecodePDU**(*is_server: bool*)

Bases: object

Decode pdu requests/responses (server/client).

classmethod **add_pdu**(*req: type[ModbusPDU]*, *resp: type[ModbusPDU]*)

Register request/response.

classmethod **add_sub_pdu**(*req: type[ModbusPDU]*, *resp: type[ModbusPDU]*)

Register request/response.

decode(*frame: bytes*) → ModbusPDU | None

Decode a frame.

list_function_codes()

Return list of function codes.

`lookupPduClass(data: bytes) → type[ModbusPDU] | None`

Use `function_code` to determine the class of the PDU.

```
pdu_sub_table: dict[int, dict[int, tuple[type[ModbusPDU], type[ModbusPDU]]]] = {8:
{0: (<class 'pymodbus.pdu.diag_message.ReturnQueryDataRequest'>, <class
'pymodbus.pdu.diag_message.ReturnQueryDataResponse'>), 1: (<class
'pymodbus.pdu.diag_message.RestartCommunicationsOptionRequest'>, <class
'pymodbus.pdu.diag_message.RestartCommunicationsOptionResponse'>), 2: (<class
'pymodbus.pdu.diag_message.ReturnDiagnosticRegisterRequest'>, <class
'pymodbus.pdu.diag_message.ReturnDiagnosticRegisterResponse'>), 3: (<class
'pymodbus.pdu.diag_message.ChangeAsciiInputDelimiterRequest'>, <class
'pymodbus.pdu.diag_message.ChangeAsciiInputDelimiterResponse'>), 4: (<class
'pymodbus.pdu.diag_message.ForceListenOnlyModeRequest'>, <class
'pymodbus.pdu.diag_message.ForceListenOnlyModeResponse'>), 10: (<class
'pymodbus.pdu.diag_message.ClearCountersRequest'>, <class
'pymodbus.pdu.diag_message.ClearCountersResponse'>), 11: (<class
'pymodbus.pdu.diag_message.ReturnBusMessageCountRequest'>, <class
'pymodbus.pdu.diag_message.ReturnBusMessageCountResponse'>), 12: (<class
'pymodbus.pdu.diag_message.ReturnBusCommunicationErrorCountRequest'>, <class
'pymodbus.pdu.diag_message.ReturnBusCommunicationErrorCountResponse'>), 13: (<class
'pymodbus.pdu.diag_message.ReturnBusExceptionErrorCountRequest'>, <class
'pymodbus.pdu.diag_message.ReturnBusExceptionErrorCountResponse'>), 14: (<class
'pymodbus.pdu.diag_message.ReturnDeviceMessageCountRequest'>, <class
'pymodbus.pdu.diag_message.ReturnDeviceMessageCountResponse'>), 15: (<class
'pymodbus.pdu.diag_message.ReturnDeviceNoResponseCountRequest'>, <class
'pymodbus.pdu.diag_message.ReturnDeviceNoResponseCountResponse'>), 16: (<class
'pymodbus.pdu.diag_message.ReturnDeviceNAKCountRequest'>, <class
'pymodbus.pdu.diag_message.ReturnDeviceNAKCountResponse'>), 17: (<class
'pymodbus.pdu.diag_message.ReturnDeviceBusyCountRequest'>, <class
'pymodbus.pdu.diag_message.ReturnDeviceBusyCountResponse'>), 18: (<class
'pymodbus.pdu.diag_message.ReturnDeviceBusCharacterOverrunCountRequest'>, <class
'pymodbus.pdu.diag_message.ReturnDeviceBusCharacterOverrunCountResponse'>), 19:
(<class 'pymodbus.pdu.diag_message.ReturnIopOverrunCountRequest'>, <class
'pymodbus.pdu.diag_message.ReturnIopOverrunCountResponse'>), 20: (<class
'pymodbus.pdu.diag_message.ClearOverrunCountRequest'>, <class
'pymodbus.pdu.diag_message.ClearOverrunCountResponse'>), 21: (<class
'pymodbus.pdu.diag_message.GetClearModbusPlusRequest'>, <class
'pymodbus.pdu.diag_message.GetClearModbusPlusResponse'>)}, 43: {14: (<class
'pymodbus.pdu.mei_message.ReadDeviceInformationRequest'>, <class
'pymodbus.pdu.mei_message.ReadDeviceInformationResponse'>)}}
```

```
pdu_table: dict[int, tuple[type[ModbusPDU], type[ModbusPDU]]] = {1: (<class
'pymodbus.pdu.bit_message.ReadCoilsRequest'>, <class
'pymodbus.pdu.bit_message.ReadCoilsResponse'>), 2: (<class
'pymodbus.pdu.bit_message.ReadDiscreteInputsRequest'>, <class
'pymodbus.pdu.bit_message.ReadDiscreteInputsResponse'>), 3: (<class
'pymodbus.pdu.register_message.ReadHoldingRegistersRequest'>, <class
'pymodbus.pdu.register_message.ReadHoldingRegistersResponse'>), 4: (<class
'pymodbus.pdu.register_message.ReadInputRegistersRequest'>, <class
'pymodbus.pdu.register_message.ReadInputRegistersResponse'>), 5: (<class
'pymodbus.pdu.bit_message.WriteSingleCoilRequest'>, <class
'pymodbus.pdu.bit_message.WriteSingleCoilResponse'>), 6: (<class
'pymodbus.pdu.register_message.WriteSingleRegisterRequest'>, <class
'pymodbus.pdu.register_message.WriteSingleRegisterResponse'>), 7: (<class
'pymodbus.pdu.other_message.ReadExceptionStatusRequest'>, <class
'pymodbus.pdu.other_message.ReadExceptionStatusResponse'>), 8: (<class
'pymodbus.pdu.diag_message.DiagnosticBase'>, <class
'pymodbus.pdu.diag_message.DiagnosticBase'>), 11: (<class
'pymodbus.pdu.other_message.GetCommEventCounterRequest'>, <class
'pymodbus.pdu.other_message.GetCommEventCounterResponse'>), 12: (<class
'pymodbus.pdu.other_message.GetCommEventLogRequest'>, <class
'pymodbus.pdu.other_message.GetCommEventLogResponse'>), 15: (<class
'pymodbus.pdu.bit_message.WriteMultipleCoilsRequest'>, <class
'pymodbus.pdu.bit_message.WriteMultipleCoilsResponse'>), 16: (<class
'pymodbus.pdu.register_message.WriteMultipleRegistersRequest'>, <class
'pymodbus.pdu.register_message.WriteMultipleRegistersResponse'>), 17: (<class
'pymodbus.pdu.other_message.ReportDeviceIdRequest'>, <class
'pymodbus.pdu.other_message.ReportDeviceIdResponse'>), 20: (<class
'pymodbus.pdu.file_message.ReadFileRecordRequest'>, <class
'pymodbus.pdu.file_message.ReadFileRecordResponse'>), 21: (<class
'pymodbus.pdu.file_message.WriteFileRecordRequest'>, <class
'pymodbus.pdu.file_message.WriteFileRecordResponse'>), 22: (<class
'pymodbus.pdu.register_message.MaskWriteRegisterRequest'>, <class
'pymodbus.pdu.register_message.MaskWriteRegisterResponse'>), 23: (<class
'pymodbus.pdu.register_message.ReadWriteMultipleRegistersRequest'>, <class
'pymodbus.pdu.register_message.ReadWriteMultipleRegistersResponse'>), 24: (<class
'pymodbus.pdu.file_message.ReadFifoQueueRequest'>, <class
'pymodbus.pdu.file_message.ReadFifoQueueResponse'>), 43: (<class
'pymodbus.pdu.mei_message.ReadDeviceInformationRequest'>, <class
'pymodbus.pdu.mei_message.ReadDeviceInformationResponse'>)}

```

register(*custom_class: type[ModbusPDU]*) → None

Register a function and sub function class with the decoder.

Bit Reading Request/Response messages.

```
class pymodbus.pdu.bit_message.ReadCoilsRequest(dev_id: int = 0, transaction_id: int = 0, address: int
= 0, count: int = 0, bits: list[bool] | None = None,
registers: list[int] | None = None, status: int = 1)
```

Bases: ModbusPDU

ReadCoilsRequest.

decode(*data: bytes*) → None

Decode a request pdu.

encode() → bytes

Encode a request pdu.

function_code: int = 1

get_response_pdu_size() → int

Get response pdu size.

Func_code (1 byte) + Byte Count(1 byte) + Quantity of Coils (n Bytes)/8, if the remainder is different of 0 then N = N+1

rtu_frame_size: int = 8

async_update_datastore(context: [ModbusDeviceContext](#)) → ModbusPDU

Run request against a datastore.

class pymodbus.pdu.bit_message.**ReadCoilsResponse**(dev_id: int = 0, transaction_id: int = 0, address: int = 0, count: int = 0, bits: list[bool] | None = None, registers: list[int] | None = None, status: int = 1)

Bases: [ModbusPDU](#)

[ReadCoilsResponse](#).

decode(data: bytes) → None

Decode response pdu.

encode() → bytes

Encode response pdu.

function_code: int = 1

rtu_byte_count_pos: int = 2

class pymodbus.pdu.bit_message.**ReadDiscreteInputsRequest**(dev_id: int = 0, transaction_id: int = 0, address: int = 0, count: int = 0, bits: list[bool] | None = None, registers: list[int] | None = None, status: int = 1)

Bases: [ReadCoilsRequest](#)

[ReadDiscreteInputsRequest](#).

function_code: int = 2

class pymodbus.pdu.bit_message.**ReadDiscreteInputsResponse**(dev_id: int = 0, transaction_id: int = 0, address: int = 0, count: int = 0, bits: list[bool] | None = None, registers: list[int] | None = None, status: int = 1)

Bases: [ReadCoilsResponse](#)

[ReadDiscreteInputsResponse](#).

function_code: int = 2

class pymodbus.pdu.bit_message.**WriteMultipleCoilsRequest**(dev_id: int = 0, transaction_id: int = 0, address: int = 0, count: int = 0, bits: list[bool] | None = None, registers: list[int] | None = None, status: int = 1)

Bases: [ModbusPDU](#)

[WriteMultipleCoilsRequest](#).

decode(*data: bytes*) → None

Decode a write coils request.

encode() → bytes

Encode write coils request.

function_code: int = 15

get_response_pdu_size() → int

Get response pdu size.

Func_code (1 byte) + Output Address (2 byte) + Quantity of Outputs (2 Bytes) :return:

rtu_byte_count_pos: int = 6

async update_datastore(*context: ModbusDeviceContext*) → ModbusPDU

Run a request against a datastore.

```
class pymodbus.pdu.bit_message.WriteMultipleCoilsResponse(dev_id: int = 0, transaction_id: int = 0,
                                                         address: int = 0, count: int = 0, bits:
                                                         list[bool] | None = None, registers:
                                                         list[int] | None = None, status: int = 1)
```

Bases: ModbusPDU

WriteMultipleCoilsResponse.

decode(*data: bytes*) → None

Decode a write coils response.

encode() → bytes

Encode write coils response.

function_code: int = 15

rtu_frame_size: int = 8

```
class pymodbus.pdu.bit_message.WriteSingleCoilRequest(dev_id: int = 0, transaction_id: int = 0,
                                                       address: int = 0, count: int = 0, bits:
                                                       list[bool] | None = None, registers: list[int] |
                                                       None = None, status: int = 1)
```

Bases: *WriteSingleCoilResponse*

WriteSingleCoilRequest.

get_response_pdu_size() → int

Get response pdu size.

Func_code (1 byte) + Output Address (2 byte) + Output Value (2 Bytes)

async update_datastore(*context: ModbusDeviceContext*) → ModbusPDU

Run a request against a datastore.

```
class pymodbus.pdu.bit_message.WriteSingleCoilResponse(dev_id: int = 0, transaction_id: int = 0,
                                                         address: int = 0, count: int = 0, bits:
                                                         list[bool] | None = None, registers: list[int] |
                                                         None = None, status: int = 1)
```

Bases: ModbusPDU

WriteSingleCoilResponse.

decode(*data: bytes*) → None
Decode a write coil request.

encode() → bytes
Encode write coil request.

function_code: int = 5

rtu_frame_size: int = 8

Diagnostic Record Read/Write.

class pymodbus.pdu.diag_message.**ChangeAsciiInputDelimiterRequest**(*message: bytes | int | list | tuple | None = 0, dev_id: int = 1, transaction_id: int = 0*)

Bases: *DiagnosticBase*

ChangeAsciiInputDelimiterRequest.

sub_function_code: int = 3

async update_datastore(*_context: ModbusDeviceContext*) → ModbusPDU
update_datastore the diagnostic request on the given device.

class pymodbus.pdu.diag_message.**ChangeAsciiInputDelimiterResponse**(*message: bytes | int | list | tuple | None = 0, dev_id: int = 1, transaction_id: int = 0*)

Bases: *DiagnosticBase*

ChangeAsciiInputDelimiterResponse.

sub_function_code: int = 3

class pymodbus.pdu.diag_message.**ClearCountersRequest**(*message: bytes | int | list | tuple | None = 0, dev_id: int = 1, transaction_id: int = 0*)

Bases: *DiagnosticBase*

ClearCountersRequest.

sub_function_code: int = 10

async update_datastore(*_context: ModbusDeviceContext*) → ModbusPDU
update_datastore the diagnostic request on the given device.

class pymodbus.pdu.diag_message.**ClearCountersResponse**(*message: bytes | int | list | tuple | None = 0, dev_id: int = 1, transaction_id: int = 0*)

Bases: *DiagnosticBase*

ClearCountersResponse.

sub_function_code: int = 10

class pymodbus.pdu.diag_message.**ClearOverrunCountRequest**(*message: bytes | int | list | tuple | None = 0, dev_id: int = 1, transaction_id: int = 0*)

Bases: *DiagnosticBase*

ClearOverrunCountRequest.

sub_function_code: int = 20

```

async update_datastore(_context: ModbusDeviceContext) → ModbusPDU
    update_datastore the diagnostic request on the given device.

class pymodbus.pdu.diag_message.ClearOverrunCountResponse(message: bytes | int | list | tuple | None =
    0, dev_id: int = 1, transaction_id: int =
    0)

    Bases: DiagnosticBase
    ClearOverrunCountResponse.
    sub_function_code: int = 20

class pymodbus.pdu.diag_message.DiagnosticBase(message: bytes | int | list | tuple | None = 0, dev_id: int
    = 1, transaction_id: int = 0)

    Bases: ModbusPDU
    DiagnosticBase.
    decode(data: bytes) → None
        Decode a diagnostic request.
    classmethod decode_sub_function_code(data: bytes) → int
        Decode sub function code (2 bytes).
    encode() → bytes
        Encode a diagnostic response.
    function_code: int = 8
    get_response_pdu_size() → int
        Get response pdu size.
        Func_code (1 byte) + Sub function code (2 byte) + Data (2 * N bytes)
    rtu_frame_size: int = 8
    sub_function_code: int = 9999

    async update_datastore(_context: ModbusDeviceContext) → ModbusPDU
        Implement dummy.

class pymodbus.pdu.diag_message.ForceListenOnlyModeRequest(message: bytes | int | list | tuple | None
    = 0, dev_id: int = 1, transaction_id: int
    = 0)

    Bases: DiagnosticBase
    ForceListenOnlyModeRequest.
    sub_function_code: int = 4

    async update_datastore(_context: ModbusDeviceContext) → ModbusPDU
        update_datastore the diagnostic request on the given device.

class pymodbus.pdu.diag_message.ForceListenOnlyModeResponse(dev_id=1, transaction_id=0)
    Bases: DiagnosticBase
    ForceListenOnlyModeResponse.
    This does not send a response

```

sub_function_code: int = 4

class pymodbus.pdu.diag_message.**GetClearModbusPlusRequest**(*message: bytes | int | list | tuple | None = 0, dev_id: int = 1, transaction_id: int = 0*)

Bases: *DiagnosticBase*

GetClearModbusPlusRequest.

encode()

Encode a diagnostic response.

get_response_pdu_size()

Return size of the response.

Func_code (1 byte) + Sub function code (2 byte) + Operation (2 byte) + Data (108 bytes)

sub_function_code: int = 21

async update_datastore(*_context: ModbusDeviceContext*) → ModbusPDU

update_datastore the diagnostic request on the given device.

class pymodbus.pdu.diag_message.**GetClearModbusPlusResponse**(*message: bytes | int | list | tuple | None = 0, dev_id: int = 1, transaction_id: int = 0*)

Bases: *DiagnosticBase*

GetClearModbusPlusResponse.

sub_function_code: int = 21

class pymodbus.pdu.diag_message.**RestartCommunicationsOptionRequest**(*message: bytes | int | list | tuple | None = 0, dev_id: int = 1, transaction_id: int = 0*)

Bases: *DiagnosticBase*

RestartCommunicationsOptionRequest.

sub_function_code: int = 1

class pymodbus.pdu.diag_message.**RestartCommunicationsOptionResponse**(*message: bytes | int | list | tuple | None = 0, dev_id: int = 1, transaction_id: int = 0*)

Bases: *DiagnosticBase*

RestartCommunicationsOptionResponse.

sub_function_code: int = 1

class pymodbus.pdu.diag_message.**ReturnBusCommunicationErrorCountRequest**(*message: bytes | int | list | tuple | None = 0, dev_id: int = 1, transaction_id: int = 0*)

Bases: *DiagnosticBase*

ReturnBusCommunicationErrorCountRequest.

sub_function_code: int = 12

async update_datastore(*_context*: [ModbusDeviceContext](#)) → ModbusPDU

update_datastore the diagnostic request on the given device.

```
class pymodbus.pdu.diag_message.ReturnBusCommunicationErrorCountResponse(message: bytes | int | list | tuple | None = 0, dev_id: int = 1, transaction_id: int = 0)
```

Bases: [DiagnosticBase](#)

ReturnBusCommunicationErrorCountResponse.

sub_function_code: int = 12

```
class pymodbus.pdu.diag_message.ReturnBusExceptionErrorCountRequest(message: bytes | int | list | tuple | None = 0, dev_id: int = 1, transaction_id: int = 0)
```

Bases: [DiagnosticBase](#)

ReturnBusExceptionErrorCountRequest.

sub_function_code: int = 13

async update_datastore(*_context*: [ModbusDeviceContext](#)) → ModbusPDU

update_datastore the diagnostic request on the given device.

```
class pymodbus.pdu.diag_message.ReturnBusExceptionErrorCountResponse(message: bytes | int | list | tuple | None = 0, dev_id: int = 1, transaction_id: int = 0)
```

Bases: [DiagnosticBase](#)

ReturnBusExceptionErrorCountResponse.

sub_function_code: int = 13

```
class pymodbus.pdu.diag_message.ReturnBusMessageCountRequest(message: bytes | int | list | tuple | None = 0, dev_id: int = 1, transaction_id: int = 0)
```

Bases: [DiagnosticBase](#)

ReturnBusMessageCountRequest.

sub_function_code: int = 11

async update_datastore(*_context*: [ModbusDeviceContext](#)) → ModbusPDU

update_datastore the diagnostic request on the given device.

```
class pymodbus.pdu.diag_message.ReturnBusMessageCountResponse(message: bytes | int | list | tuple | None = 0, dev_id: int = 1, transaction_id: int = 0)
```

Bases: [DiagnosticBase](#)

ReturnBusMessageCountResponse.

sub_function_code: int = 11

```
class pymodbus.pdu.diag_message.ReturnDeviceBusCharacterOverrunCountRequest(message: bytes |  
                                                                              int | list | tuple |  
                                                                              None = 0,  
                                                                              dev_id: int = 1,  
                                                                              transaction_id:  
                                                                              int = 0)
```

Bases: *DiagnosticBase*

ReturnDeviceBusCharacterOverrunCountRequest.

sub_function_code: int = 18

async update_datastore(*_context*: ModbusDeviceContext) → ModbusPDU

update_datastore the diagnostic request on the given device.

```
class pymodbus.pdu.diag_message.ReturnDeviceBusCharacterOverrunCountResponse(message: bytes  
                                                                              | int | list | tuple  
                                                                              | None = 0,  
                                                                              dev_id: int = 1,  
                                                                              transaction_id:  
                                                                              int = 0)
```

Bases: *DiagnosticBase*

ReturnDeviceBusCharacterOverrunCountResponse.

sub_function_code: int = 18

```
class pymodbus.pdu.diag_message.ReturnDeviceBusyCountRequest(message: bytes | int | list | tuple |  
                                                                None = 0, dev_id: int = 1,  
                                                                transaction_id: int = 0)
```

Bases: *DiagnosticBase*

ReturnDeviceBusyCountRequest.

sub_function_code: int = 17

async update_datastore(*_context*: ModbusDeviceContext) → ModbusPDU

update_datastore the diagnostic request on the given device.

```
class pymodbus.pdu.diag_message.ReturnDeviceBusyCountResponse(message: bytes | int | list | tuple |  
                                                                None = 0, dev_id: int = 1,  
                                                                transaction_id: int = 0)
```

Bases: *DiagnosticBase*

ReturnDeviceBusyCountResponse.

sub_function_code: int = 17

```
class pymodbus.pdu.diag_message.ReturnDeviceMessageCountRequest(message: bytes | int | list | tuple |  
                                                                None = 0, dev_id: int = 1,  
                                                                transaction_id: int = 0)
```

Bases: *DiagnosticBase*

ReturnDeviceMessageCountRequest.

sub_function_code: int = 14

async update_datastore(*_context*: ModbusDeviceContext) → ModbusPDU

update_datastore the diagnostic request on the given device.

```
class pymodbus.pdu.diag_message.ReturnDeviceMessageCountResponse(message: bytes | int | list | tuple
                                                                    | None = 0, dev_id: int = 1,
                                                                    transaction_id: int = 0)
```

Bases: *DiagnosticBase*

ReturnDeviceMessageCountResponse.

sub_function_code: int = 14

```
class pymodbus.pdu.diag_message.ReturnDeviceNAKCountRequest(message: bytes | int | list | tuple | None
                                                                = 0, dev_id: int = 1, transaction_id:
                                                                int = 0)
```

Bases: *DiagnosticBase*

ReturnDeviceNAKCountRequest.

sub_function_code: int = 16

async update_datastore(_context: *ModbusDeviceContext*) → *ModbusPDU*

update_datastore the diagnostic request on the given device.

```
class pymodbus.pdu.diag_message.ReturnDeviceNAKCountResponse(message: bytes | int | list | tuple |
                                                                None = 0, dev_id: int = 1,
                                                                transaction_id: int = 0)
```

Bases: *DiagnosticBase*

ReturnDeviceNAKCountResponse.

sub_function_code: int = 16

```
class pymodbus.pdu.diag_message.ReturnDeviceNoResponseCountRequest(message: bytes | int | list |
                                                                      tuple | None = 0, dev_id: int
                                                                      = 1, transaction_id: int = 0)
```

Bases: *DiagnosticBase*

ReturnDeviceNoResponseCountRequest.

sub_function_code: int = 15

async update_datastore(_context: *ModbusDeviceContext*) → *ModbusPDU*

update_datastore the diagnostic request on the given device.

```
class pymodbus.pdu.diag_message.ReturnDeviceNoResponseCountResponse(message: bytes | int | list |
                                                                      tuple | None = 0, dev_id:
                                                                      int = 1, transaction_id: int
                                                                      = 0)
```

Bases: *DiagnosticBase*

ReturnDeviceNoResponseCountResponse.

sub_function_code: int = 15

```
class pymodbus.pdu.diag_message.ReturnDiagnosticRegisterRequest(message: bytes | int | list | tuple |
                                                                    None = 0, dev_id: int = 1,
                                                                    transaction_id: int = 0)
```

Bases: *DiagnosticBase*

ReturnDiagnosticRegisterRequest.

sub_function_code: int = 2

async update_datastore(*_context*: [ModbusDeviceContext](#)) → ModbusPDU

update_datastore the diagnostic request on the given device.

class pymodbus.pdu.diag_message.**ReturnDiagnosticRegisterResponse**(*message*: bytes | int | list | tuple | None = 0, *dev_id*: int = 1, *transaction_id*: int = 0)

Bases: [DiagnosticBase](#)

ReturnDiagnosticRegisterResponse.

sub_function_code: int = 2

class pymodbus.pdu.diag_message.**ReturnIopOverrunCountRequest**(*message*: bytes | int | list | tuple | None = 0, *dev_id*: int = 1, *transaction_id*: int = 0)

Bases: [DiagnosticBase](#)

ReturnIopOverrunCountRequest.

sub_function_code: int = 19

async update_datastore(*_context*: [ModbusDeviceContext](#)) → ModbusPDU

update_datastore the diagnostic request on the given device.

class pymodbus.pdu.diag_message.**ReturnIopOverrunCountResponse**(*message*: bytes | int | list | tuple | None = 0, *dev_id*: int = 1, *transaction_id*: int = 0)

Bases: [DiagnosticBase](#)

ReturnIopOverrunCountResponse.

sub_function_code: int = 19

class pymodbus.pdu.diag_message.**ReturnQueryDataRequest**(*message*: bytes | int | list | tuple | None = 0, *dev_id*: int = 1, *transaction_id*: int = 0)

Bases: [DiagnosticBase](#)

ReturnQueryDataRequest.

sub_function_code: int = 0

class pymodbus.pdu.diag_message.**ReturnQueryDataResponse**(*message*: bytes | int | list | tuple | None = 0, *dev_id*: int = 1, *transaction_id*: int = 0)

Bases: [DiagnosticBase](#)

ReturnQueryDataResponse.

sub_function_code: int = 0

File Record Read/Write Messages.

class pymodbus.pdu.file_message.**FileRecord**(*file_number*: int = 0, *record_number*: int = 0, *record_data*: bytes = b'', *record_length*: int = 0)

Bases: object

Represents a file record and its relevant data.

file_number: int = 0


```

    record_data: bytes = b''

    record_length: int = 0

    record_number: int = 0

class pymodbus.pdu.file_message.ReadFifoQueueRequest(address: int = 0, dev_id: int = 1,
                                                    transaction_id: int = 0)

    Bases: ModbusPDU
    ReadFifoQueueRequest.
    decode(data: bytes) → None
        Decode the incoming request.
    encode() → bytes
        Encode the request packet.
    function_code: int = 24
    rtu_frame_size: int = 6

    async update_datastore(_context: ModbusDeviceContext) → ModbusPDU
        Run a read exception status request against the store.

class pymodbus.pdu.file_message.ReadFifoQueueResponse(values: list[int] | None = None, dev_id: int =
                                                    1, transaction_id: int = 0)

    Bases: ModbusPDU
    ReadFifoQueueResponse.
    classmethod calculateRtuFrameSize(data: bytes) → int
        Calculate the size of the message.
    decode(data: bytes) → None
        Decode a the response.
    encode() → bytes
        Encode the response.
    function_code: int = 24

class pymodbus.pdu.file_message.ReadFileRecordRequest(records: list[FileRecord] | None = None,
                                                    dev_id: int = 1, transaction_id: int = 0)

    Bases: ModbusPDU
    ReadFileRecordRequest.
    decode(data: bytes) → None
        Decode the incoming request.
    encode() → bytes
        Encode the request packet.
    function_code: int = 20

    get_response_pdu_size() → int
        Get response pdu size.
        Func_code (1 byte) + Quantity of record (each 7 bytes),

```

rtu_byte_count_pos: int = 2

async_update_datastore(*_context*: [ModbusDeviceContext](#)) → ModbusPDU

Run a read exception status request against the store.

class pymodbus.pdu.file_message.**ReadFileRecordResponse**(*records*: list[[FileRecord](#)] | None = None,
dev_id: int = 1, transaction_id: int = 0)

Bases: ModbusPDU

ReadFileRecordResponse.

decode(*data*: bytes) → None

Decode the response.

encode() → bytes

Encode the response.

function_code: int = 20

rtu_byte_count_pos: int = 2

class pymodbus.pdu.file_message.**WriteFileRecordRequest**(*records*: list[[FileRecord](#)] | None = None,
dev_id: int = 1, transaction_id: int = 0)

Bases: ModbusPDU

WriteFileRecordRequest.

decode(*data*: bytes) → None

Decode the incoming request.

encode() → bytes

Encode the request packet.

function_code: int = 21

get_response_pdu_size() → int

Get response pdu size.

Func_code (1 byte) + Quantity of record (each 7 bytes),

rtu_byte_count_pos: int = 2

async_update_datastore(*_context*: [ModbusDeviceContext](#)) → ModbusPDU

Run the write file record request against the context.

class pymodbus.pdu.file_message.**WriteFileRecordResponse**(*records*: list[[FileRecord](#)] | None = None,
dev_id: int = 1, transaction_id: int = 0)

Bases: ModbusPDU

The normal response is an echo of the request.

decode(*data*: bytes) → None

Decode the incoming request.

encode() → bytes

Encode the response.

function_code: int = 21

rtu_byte_count_pos: `int = 2`

Encapsulated Interface (MEI) Transport Messages.

```
class pymodbus.pdu.mei_message.ReadDeviceInformationRequest(read_code: int | None = None,  
object_id: int = 0, dev_id: int = 1,  
transaction_id: int = 0)
```

Bases: `ModbusPDU`

`ReadDeviceInformationRequest`.

decode(*data: bytes*) → `None`

Decode data part of the message.

classmethod decode_sub_function_code(*data: bytes*) → `int`

Decode sub function code (1 byte).

encode() → `bytes`

Encode the request packet.

function_code: `int = 43`

rtu_frame_size: `int = 7`

sub_function_code: `int = 14`

async update_datastore(*_context: [ModbusDeviceContext](#)*) → `ModbusPDU`

Run a read exception status request against the store.

```
class pymodbus.pdu.mei_message.ReadDeviceInformationResponse(read_code: int | None = None,  
information: dict | None = None,  
dev_id: int = 1, transaction_id: int = 0)
```

Bases: `ModbusPDU`

`ReadDeviceInformationResponse`.

classmethod calculateRtuFrameSize(*data: bytes*) → `int`

Calculate the size of the message.

decode(*data: bytes*) → `None`

Decode a the response.

classmethod decode_sub_function_code(*data: bytes*) → `int`

Decode sub function code (1 byte).

encode() → `bytes`

Encode the response.

function_code: `int = 43`

sub_function_code: `int = 14`

Diagnostic record read/write.

```
class pymodbus.pdu.other_message.GetCommEventCounterRequest(dev_id: int = 0, transaction_id: int = 0,  
address: int = 0, count: int = 0,  
bits: list[bool] | None = None,  
registers: list[int] | None = None,  
status: int = 1)
```

Bases: ModbusPDU

GetCommEventCounterRequest.

decode(*_data: bytes*) → None

Decode data part of the message.

encode() → bytes

Encode the message.

function_code: int = 11

rtu_frame_size: int = 4

async update_datastore(*_context*) → ModbusPDU

Run a read exception status request against the store.

```
class pymodbus.pdu.other_message.GetCommEventCounterResponse(dev_id: int = 0, transaction_id: int  
                                                             = 0, address: int = 0, count: int = 0,  
                                                             bits: list[bool] | None = None,  
                                                             registers: list[int] | None = None,  
                                                             status: int = 1)
```

Bases: ModbusPDU

GetCommEventCounterRequest.

decode(*data: bytes*) → None

Decode a the response.

encode() → bytes

Encode the response.

function_code: int = 11

rtu_frame_size: int = 8

```
class pymodbus.pdu.other_message.GetCommEventLogRequest(dev_id: int = 0, transaction_id: int = 0,  
                                                         address: int = 0, count: int = 0, bits:  
                                                         list[bool] | None = None, registers: list[int]  
                                                         | None = None, status: int = 1)
```

Bases: ModbusPDU

GetCommEventLogRequest.

decode(*_data: bytes*) → None

Decode data part of the message.

encode() → bytes

Encode the message.

function_code: int = 12

rtu_frame_size: int = 4

async update_datastore(*_context: [ModbusDeviceContext](#)*) → ModbusPDU

Run a read exception status request against the store.

```
class pymodbus.pdu.other_message.GetCommEventLogResponse(status: bool = True, message_count: int = 0, event_count: int = 0, events: list[int] | None = None, dev_id: int = 1, transaction_id: int = 0)
```

Bases: ModbusPDU

GetCommEventLogRequest.

decode(*data: bytes*) → None

Decode a the response.

encode() → bytes

Encode the response.

function_code: int = 12

rtu_byte_count_pos: int = 2

```
class pymodbus.pdu.other_message.ReadExceptionStatusRequest(dev_id: int = 0, transaction_id: int = 0, address: int = 0, count: int = 0, bits: list[bool] | None = None, registers: list[int] | None = None, status: int = 1)
```

Bases: ModbusPDU

ReadExceptionStatusRequest.

decode(*_data: bytes*) → None

Decode data part of the message.

encode() → bytes

Encode the message.

function_code: int = 7

rtu_frame_size: int = 4

async update_datastore(*_context: ModbusDeviceContext*) → ModbusPDU

Run a read exception status request against the store.

```
class pymodbus.pdu.other_message.ReadExceptionStatusResponse(dev_id: int = 0, transaction_id: int = 0, address: int = 0, count: int = 0, bits: list[bool] | None = None, registers: list[int] | None = None, status: int = 1)
```

Bases: ModbusPDU

ReadExceptionStatusResponse.

decode(*data: bytes*) → None

Decode a the response.

encode() → bytes

Encode the response.

function_code: int = 7

rtu_frame_size: int = 5

```
class pymodbus.pdu.other_message.ReportDeviceIdRequest(dev_id: int = 0, transaction_id: int = 0,  
                                                    address: int = 0, count: int = 0, bits:  
                                                    list[bool] | None = None, registers: list[int] |  
                                                    None = None, status: int = 1)
```

Bases: ModbusPDU

ReportDeviceIdRequest.

decode(*_data: bytes*) → None

Decode data part of the message.

encode() → bytes

Encode the message.

function_code: int = 17

rtu_frame_size: int = 4

async update_datastore(*_context: ModbusDeviceContext*) → ModbusPDU

Run a report device id request against the store.

```
class pymodbus.pdu.other_message.ReportDeviceIdResponse(identifier: bytes = b'\x00', status: bool =  
                                                    True, dev_id: int = 1, transaction_id: int =  
                                                    0)
```

Bases: ModbusPDU

ReportDeviceIdRequeste.

decode(*data: bytes*) → None

Decode a the response.

Since the identifier is device dependent, we just return the raw value that a user can decode to whatever it should be.

encode() → bytes

Encode the response.

function_code: int = 17

rtu_byte_count_pos: int = 2

Contains base classes for modbus request/response/error packets.

```
class pymodbus.pdu.pdu.ModbusPDU(dev_id: int = 0, transaction_id: int = 0, address: int = 0, count: int = 0,  
                                bits: list[bool] | None = None, registers: list[int] | None = None, status:  
                                int = 1)
```

Bases: object

Base class for all Modbus messages.

classmethod calculateRtuFrameSize(*data: bytes*) → int

Calculate the size of a PDU.

abstractmethod decode(*data: bytes*) → None

Decode data part of the message.

classmethod decode_sub_function_code(*data: bytes*) → int

Decode sub function code.

abstractmethod encode() → bytes

Encode the message.

function_code: int = 0

get_response_pdu_size() → int

Calculate response pdu size.

isError() → bool

Check if the error is a success or failure.

rtu_byte_count_pos: int = 0

rtu_frame_size: int = 0

sub_function_code: int = -1

async update_datastore(context: [ModbusDeviceContext](#)) → ModbusPDU

Run request against a datastore.

verifyAddress(address: int = -1) → None

Validate API supplied address.

verifyCount(max_count: int, count: int = -1) → None

Validate API supplied count.

pymodbus.pdu.pdu.pack_bitstring(bits: list[bool], align_byte=True) → bytes

Create a bytestring out of a list of bits.

example:

```
bits = [True, False, False, False] +
        [False, False, False, True] +
        [True, False, True, False] +
        [False, False, False, False]
result = pack_bitstring(bits)
bytes 0x05 0x81
```

pymodbus.pdu.pdu.unpack_bitstring(data: bytes) → list[bool]

Create bit list out of a bytestring.

example:

```
bytes 0x05 0x81
result = unpack_bitstring(bytes)

[True, False, True, False] + [False, False, False, False]
[True, False, False, False] + [False, False, False, True]
```

Register Reading Request/Response.

class pymodbus.pdu.register_message.**MaskWriteRegisterRequest**(address=0, and_mask=65535,
or_mask=0, dev_id=1,
transaction_id=0)

Bases: ModbusPDU

MaskWriteRegisterRequest.

decode(*data: bytes*) → None

Decode the incoming request.

encode() → bytes

Encode the request packet.

function_code: int = 22

rtu_frame_size: int = 10

async update_datastore(*context: ModbusDeviceContext*) → ModbusPDU

Run a mask write register request against the store.

class pymodbus.pdu.register_message.**MaskWriteRegisterResponse**(*address=0, and_mask=65535, or_mask=0, dev_id=1, transaction_id=0*)

Bases: ModbusPDU

MaskWriteRegisterResponse.

decode(*data: bytes*) → None

Decode a the response.

encode() → bytes

Encode the response.

function_code: int = 22

rtu_frame_size: int = 10

class pymodbus.pdu.register_message.**ReadHoldingRegistersRequest**(*dev_id: int = 0, transaction_id: int = 0, address: int = 0, count: int = 0, bits: list[bool] | None = None, registers: list[int] | None = None, status: int = 1*)

Bases: ModbusPDU

ReadHoldingRegistersRequest.

decode(*data: bytes*) → None

Decode a register request packet.

encode() → bytes

Encode the request packet.

function_code: int = 3

get_response_pdu_size() → int

Get response pdu size.

Func_code (1 byte) + Byte Count(1 byte) + 2 * Quantity of registers (== byte count).

rtu_frame_size: int = 8

async update_datastore(*context: ModbusDeviceContext*) → ModbusPDU

Run a read holding request against a datastore.


```

class pymodbus.pdu.register_message.ReadHoldingRegistersResponse(dev_id: int = 0, transaction_id: int = 0, address: int = 0, count: int = 0, bits: list[bool] | None = None, registers: list[int] | None = None, status: int = 1)

    Bases: ModbusPDU
    ReadHoldingRegistersResponse.
    decode(data: bytes) → None
        Decode a register response packet.
    encode() → bytes
        Encode the response packet.
    function_code: int = 3
    rtu_byte_count_pos: int = 2

class pymodbus.pdu.register_message.ReadInputRegistersRequest(dev_id: int = 0, transaction_id: int = 0, address: int = 0, count: int = 0, bits: list[bool] | None = None, registers: list[int] | None = None, status: int = 1)

    Bases: ReadHoldingRegistersRequest
    ReadInputRegistersRequest.
    function_code: int = 4

class pymodbus.pdu.register_message.ReadInputRegistersResponse(dev_id: int = 0, transaction_id: int = 0, address: int = 0, count: int = 0, bits: list[bool] | None = None, registers: list[int] | None = None, status: int = 1)

    Bases: ReadHoldingRegistersResponse
    ReadInputRegistersResponse.
    function_code: int = 4

class pymodbus.pdu.register_message.ReadWriteMultipleRegistersRequest(read_address: int = 0, read_count: int = 0, write_address: int = 0, write_registers: list[int] | None = None, dev_id: int = 1, transaction_id: int = 0)

    Bases: ModbusPDU
    ReadWriteMultipleRegistersRequest.
    decode(data: bytes) → None
        Decode the register request packet.
    encode() → bytes
        Encode the request packet.

```

function_code: `int = 23`

get_response_pdu_size() `→ int`

Get response pdu size.

Func_code (1 byte) + Byte Count(1 byte) + 2 * Quantity of Coils (n Bytes)

rtu_byte_count_pos: `int = 10`

async update_datastore(*context*: [ModbusDeviceContext](#)) `→ ModbusPDU`

Run a write single register request against a datastore.

```
class pymodbus.pdu.register_message.ReadWriteMultipleRegistersResponse(dev_id: int = 0,  
                                                                    transaction_id: int = 0,  
                                                                    address: int = 0, count:  
                                                                    int = 0, bits: list[bool] |  
                                                                    None = None, registers:  
                                                                    list[int] | None = None,  
                                                                    status: int = 1)
```

Bases: [ReadHoldingRegistersResponse](#)

ReadWriteMultipleRegistersResponse.

function_code: `int = 23`

```
class pymodbus.pdu.register_message.WriteMultipleRegistersRequest(dev_id: int = 0,  
                                                                    transaction_id: int = 0,  
                                                                    address: int = 0, count: int =  
                                                                    0, bits: list[bool] | None =  
                                                                    None, registers: list[int] |  
                                                                    None = None, status: int = 1)
```

Bases: ModbusPDU

WriteMultipleRegistersRequest.

decode(*data*: bytes) `→ None`

Decode a write single register packet packet request.

encode() `→ bytes`

Encode a write single register packet packet request.

function_code: `int = 16`

get_response_pdu_size() `→ int`

Get response pdu size.

Func_code (1 byte) + Starting Address (2 byte) + Quantity of Registers (2 Bytes)

rtu_byte_count_pos: `int = 6`

async update_datastore(*context*: [ModbusDeviceContext](#)) `→ ModbusPDU`

Run a write single register request against a datastore.

```
class pymodbus.pdu.register_message.WriteMultipleRegistersResponse(dev_id: int = 0,  
                                                                    transaction_id: int = 0,  
                                                                    address: int = 0, count: int =  
                                                                    0, bits: list[bool] | None =  
                                                                    None, registers: list[int] |  
                                                                    None = None, status: int =  
                                                                    1)
```

Bases: ModbusPDU

WriteMultipleRegistersResponse.

decode(data: bytes) → None

Decode a write single register packet request.

encode() → bytes

Encode a write single register packet request.

function_code: int = 16

rtu_frame_size: int = 8

```
class pymodbus.pdu.register_message.WriteSingleRegisterRequest(dev_id: int = 0, transaction_id:
                                                                int = 0, address: int = 0, count:
                                                                int = 0, bits: list[bool] | None =
                                                                None, registers: list[int] | None =
                                                                None, status: int = 1)
```

Bases: [WriteSingleRegisterResponse](#)

WriteSingleRegisterRequest.

get_response_pdu_size() → int

Get response pdu size.

Func_code (1 byte) + Register Address(2 byte) + Register Value (2 bytes)

async update_datastore(context: [ModbusDeviceContext](#)) → ModbusPDU

Run a write single register request against a datastore.

```
class pymodbus.pdu.register_message.WriteSingleRegisterResponse(dev_id: int = 0, transaction_id:
                                                                int = 0, address: int = 0, count:
                                                                int = 0, bits: list[bool] | None =
                                                                None, registers: list[int] | None
                                                                = None, status: int = 1)
```

Bases: ModbusPDU

WriteSingleRegisterResponse.

decode(data: bytes) → None

Decode a write single register packet request.

encode() → bytes

Encode a write single register packet request.

function_code: int = 6

rtu_frame_size: int = 8

Modbus Device Controller.

These are the device management handlers. They should be maintained in the server context and the various methods should be inserted in the correct locations.

```
class pymodbus.pdu.device.DeviceInformationFactory
```

Bases: object

This is a helper.

That really just hides some of the complexity of processing the device information requests (function code 0x2b 0x0e).

classmethod **get**(*control*, *read_code*=*DeviceInformation.BASIC*, *object_id*=0)

Get the requested device data from the system.

Parameters

- **control** – The control block to pull data from
- **read_code** – The read code to process
- **object_id** – The specific object_id to read

Returns

The requested data (id, length, value)

class `pymodbus.pdu.device.ModbusDeviceIdentification`(*info*=None, *info_name*=None)

Bases: object

This is used to supply the device identification.

For the readDeviceIdentification function

For more information read section 6.21 of the modbus application protocol.

property **MajorMinorRevision**

property **ModelName**

property **ProductCode**

property **ProductName**

property **UserApplicationName**

property **VendorName**

property **VendorUrl**

summary()

Return a summary of the main items.

Returns

An dictionary of the main items

update(*value*)

Update the values of this identity.

using another identify as the value

Parameters

value – The value to copy values from

class `pymodbus.pdu.device.ModbusPlusStatistics`

Bases: object

This is used to maintain the current modbus plus statistics count.

As of right now this is simply a stub to complete the modbus implementation. For more information, see the modbus implementation guide page 87.

encode()

Return a summary of the modbus plus statistics.

Returns

54 16-bit words representing the status

reset()

Clear all of the modbus plus statistics.

summary()

Return a summary of the modbus plus statistics.

Returns

54 16-bit words representing the status

Modbus Remote Events.

An event byte returned by the Get Communications Event Log function can be any one of four types. The type is defined by bit 7 (the high-order bit) in each byte. It may be further defined by bit 6.

class pymodbus.pdu.events.CommunicationRestartEvent

Bases: *ModbusEvent*

Restart remote device Initiated Communication.

The remote device stores this type of event byte when its communications port is restarted. The remote device can be restarted by the Diagnostics function (code 08), with sub-function Restart Communications Option (code 00 01).

That function also places the remote device into a “Continue on Error” or “Stop on Error” mode. If the remote device is placed into “Continue on Error” mode, the event byte is added to the existing event log. If the remote device is placed into “Stop on Error” mode, the byte is added to the log and the rest of the log is cleared to zeros.

The event is defined by a content of zero.

decode(event)

Decode the event message to its status bits.

Parameters

event – The event to decode

Raises

ParameterException –

encode()

Encode the status bits to an event message.

Returns

The encoded event message

value = 0

class pymodbus.pdu.events.EnteredListenModeEvent

Bases: *ModbusEvent*

Enter Remote device Listen Only Mode.

The remote device stores this type of event byte when it enters the Listen Only Mode. The event is defined by a content of 04 hex.

decode(*event*)

Decode the event message to its status bits.

Parameters**event** – The event to decode**Raises***ParameterException* –**encode**()

Encode the status bits to an event message.

Returns

The encoded event message

value = 4**class** pymodbus.pdu.events.**ModbusEvent**

Bases: ABC

Define modbus events.

abstractmethod **decode**(*event*)

Decode the event message to its status bits.

Parameters**event** – The event to decode**abstractmethod** **encode**() → bytes

Encode the status bits to an event message.

class pymodbus.pdu.events.**RemoteReceiveEvent**(*overrun=False, listen=False, broadcast=False*)Bases: *ModbusEvent*

Remote device MODBUS Receive Event.

The remote device stores this type of event byte when a query message is received. It is stored before the remote device processes the message. This event is defined by bit 7 set to logic “1”. The other bits will be set to a logic “1” if the corresponding condition is TRUE. The bit layout is:

Bit Contents

0	Not Used
2	Not Used
3	Not Used
4	Character Overrun
5	Currently in Listen Only Mode
6	Broadcast Receive
7	1

decode(*event: bytes*) → None

Decode the event message to its status bits.

Parameters**event** – The event to decode**encode**() → bytes

Encode the status bits to an event message.

Returns

The encoded event message

```
class pymodbus.pdu.events.RemoteSendEvent(read=False, device_abort=False, device_busy=False,
                                             device_nak=False, write_timeout=False, listen=False)
```

Bases: [ModbusEvent](#)

Remote device MODBUS Send Event.

The remote device stores this type of event byte when it finishes processing a request message. It is stored if the remote device returned a normal or exception response, or no response.

This event is defined by bit 7 set to a logic “0”, with bit 6 set to a “1”. The other bits will be set to a logic “1” if the corresponding condition is TRUE. The bit layout is:

Bit Contents

Bit	Contents
0	Read Exception Sent (Exception Codes 1-3)
1	Device Abort Exception Sent (Exception Code 4)
2	Device Busy Exception Sent (Exception Codes 5-6)
3	Device Program NAK Exception Sent (Exception Code 7)
4	Write Timeout Error Occurred
5	Currently in Listen Only Mode
6	1
7	0

decode(event)

Decode the event message to its status bits.

Parameters

event – The event to decode

encode()

Encode the status bits to an event message.

Returns

The encoded event message

12.7 Architecture

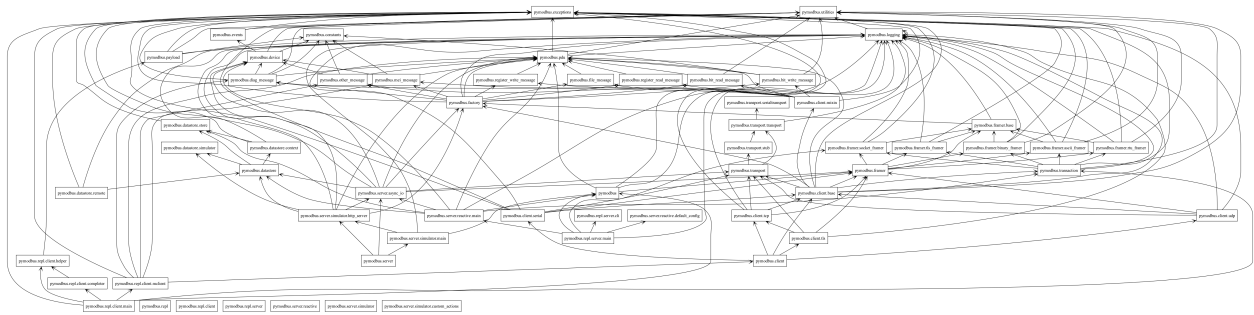
The internal structure of pymodbus is a bit complicated, mostly due to the mixture of sync and async.

The overall architecture can be viewed as:

Client classes (interface to applications) mixin (interface with all requests defined as methods) transaction (handles transactions and allow concurrent calls) framers (add pre/post headers to make a valid package) transport (handles actual transportation)

Server classes (interface to applications) datastores (handles registers/values to be returned) transaction (handles transactions and allow concurrent calls) framers (add pre/post headers to make a valid package) transport (handles actual transportation)

In detail the packages can viewed as:



In detail the classes can be viewed as:

PYMODBUS - A PYTHON MODBUS STACK

Pymodbus is a full Modbus protocol implementation offering a client and server with synchronous/asynchronous API and simulators.

Our releases follow the pattern *X.Y.Z*. We have strict rules for what different version number updates mean:

- **Z**, No API changes! bug fixes and smaller enhancements.
- **Y**, API changes, bug fixes and bigger enhancements.
- **X**, Major changes in API and/or method to use pymodbus

Upgrade examples:

- **3.9.0 -> 3.9.2: just plugin the new version, no changes needed.**
Remark fixing bugs, can lead to a different behaviors/returns
- 3.8.0 -> 3.9.0: Smaller changes to the pymodbus calls might be needed
- 2.5.4 -> 3.0.0: Major changes in the application might be needed

REMARK: As can be seen from the above Pymodbus do NOT follow the semver.org standard.

It is always recommended to read the [CHANGELOG](#) as well as the [API_changes](#) files.

Current release is [3.11.3](#).

Bleeding edge (not released) is [dev](#).

All changes are described in [release notes](#) and all API changes are [documented](#)

A big thanks to all the [volunteers](#) that helps make pymodbus a great project.

Source code on [github](#)

Full documentation for newest releases as well as the bleeding edge (dev) [readthedocs](#)

13.1 Pymodbus in a nutshell

Pymodbus consist of 5 parts:

- **client**, connect to your favorite device(s)
- **server**, simulate your favorite device(s)

- **simulator**, an html based server simulator
- **examples**, showing both simple and advances usage

13.1.1 Common features

- Full modbus standard protocol implementation
- Support for custom function codes
- Support serial (rs-485), tcp, tls and udp communication
- Support all standard frames: socket, rtu, rtu-over-tcp, tcp and ascii
- Does not have third party dependencies, apart from pyserial (optional)
- Very lightweight project
- Requires Python >= 3.10
- Thorough test suite, that test all corners of the library
- Automatically tested on Windows, Linux and MacOS combined with python 3.10 - 3.14
- Strongly typed API (py.typed present)

The modbus protocol specification: Modbus_Application_Protocol_V1_1b3.pdf can be found on modbus.org

13.1.2 Client Features

- Asynchronous API and synchronous API for applications
- Very simple setup and call sequence (just 6 lines of code)
- Utilities to convert int/float to/from multiple registers
- Encoder/decoder to help with standard python data types

[Client documentation](#)

13.1.3 Server Features

- Asynchronous implementation for high performance
- Synchronous API classes for convenience
- Simulate real life devices
- Full server control context (device information, counters, etc)
- Different backend datastores to manage register values
- Callback to intercept requests/responses
- Work on RS485 in parallel with other devices

[Server documentation](#)

13.1.4 Simulator Features

- Server simulator with WEB interface
- Configure the structure of a real device
- Monitor traffic online
- Allow distributed team members to work on a virtual device using internet

- Simulation of broken requests/responses
- Simulation of error responses (hard to provoke in real devices)

[Simulator documentation](#)

13.2 Use Cases

The client is the most typically used. It is embedded into applications, where it abstract the modbus protocol from the application by providing an easy to use API. The client is integrated into some well known projects like [home-assistant](#).

Although most system administrators will find little need for a Modbus server, the server is handy to verify the functionality of an application.

The simulator and/or server is often used to simulate real life devices testing applications. The server is excellent to perform high volume testing (e.g. hundreds of devices connected to the application). The advantage of the server is that it runs not only on “normal” computers but also on small ones like a Raspberry PI.

Since the library is written in python, it allows for easy scripting and/or integration into existing solutions.

For more information please browse the project documentation:

<https://readthedocs.org/docs/pymodbus/en/latest/index.html>

13.3 Install

The library is available on [pypi.org](#) and [github.com](#) to install with

- `pip` for those who just want to use the library
- `git clone` for those who wants to help or just are curious

Be aware that there are a number of project, who have forked pymodbus and

- Seems just to provide a version frozen in time
- Extended pymodbus with extra functionality

The latter is not because we rejected the extra functionality (we welcome all changes), but because the codeowners made that decision.

In both cases, please understand, we cannot offer support to users of these projects as we do not know what have been changed nor what status the forked code have.

A growing number of Linux distributions include pymodbus in their standard installation.

You need to have python3 installed, preferable 3.11.

13.3.1 Install with pip

You can install using pip by issuing the following commands in a terminal window:

```
pip install pymodbus
```

If you want to use the serial interface:

```
pip install pymodbus[serial]
```

This will install pymodbus with the pyserial dependency.

Pymodbus offers a number of extra options:

- **serial**, needed for serial communication
- **simulator**, needed by pymodbus.simulator
- **documentation**, needed to generate documentation
- **development**, needed for development
- **all**, installs all of the above

which can be installed as:

```
pip install pymodbus[<option>,...]
```

It is possible to install old releases if needed:

```
pip install pymodbus==3.5.4
```

13.3.2 Install with github

On github, fork <https://github.com/pymodbus-dev/pymodbus.git>

Clone the source, and make a virtual environment:

```
git clone git://github.com/<your account>/pymodbus.git
cd pymodbus
python3 -m venv .venv
```

Activate the virtual environment, this command needs repeated in every new terminal:

```
source .venv/bin/activate
```

To get a specific release:

```
git checkout v3.5.2
```

or the bleeding edge:

```
git checkout dev
```

Some distributions have an old pip, which needs to be upgraded:

```
pip install --upgrade pip
```

Install required development tools:

```
pip install ".[development]"
```

Install all (allows creation of documentation etc):

```
pip install ".[all]"
```

Install git hooks, that helps control the commit and avoid errors when submitting a Pull Request:

```
cp githooks/* .git/hooks
```

This installs dependencies in your virtual environment with pointers directly to the pymodbus directory, so any change you make is immediately available as if installed.

The repository contains a number of important branches and tags.

- **dev** is where all development happens, this branch is not always stable.

- **master** is where releases are kept.
- **vX.Y.Z** (e.g. v2.5.3) is a specific release

13.4 Example Code

For those of you who just want to get started quickly, here you go:

```
from pymodbus.client import ModbusTcpClient

client = ModbusTcpClient('MyDevice.lan')
client.connect()
client.write_coil(1, True)
result = client.read_coils(1,1)
print(result.bits[0])
client.close()
```

We provide a couple of simple ready to go clients:

- [async client](#)
- [sync client](#)

For more advanced examples, check out [Examples](#) included in the repository. If you have created any utilities that meet a specific need, feel free to submit them so others can benefit.

Also, if you have a question, please [create a post in discussions q&a topic](#), so that others can benefit from the results.

If you think, that something in the code is broken/not running well, please [open an issue](#), read the Template-text first and then post your issue with your setup information.

[Example documentation](#)

13.5 Contributing

Just fork the repo and raise your Pull Request against dev branch.

We always have more work than time, so feel free to open a discussion / issue on a theme you want to solve.

If your company would like your device tested or have a cloud based device simulation, feel free to contact us. We are happy to help your company solve your modbus challenges.

That said, the current work mainly involves polishing the library and solving issues:

- Fixing bugs/feature requests
- Architecture documentation
- Functional testing against any reference we can find

There are 2 bigger projects ongoing:

- rewriting the internal part of all clients (both sync and async)
- Add features to the simulator, and enhance the web design

13.6 Development instructions

The current code base is compatible with python >= 3.10.

Here are some of the common commands to perform a range of activities:

```
source .venv/bin/activate    <-- Activate the virtual environment
./check_ci.sh                <-- run the same checks as CI runs on a pull request.
```

Make a pull request:

```
git checkout dev              <-- activate development branch
git pull                      <-- update branch with newest changes
git checkout -b feature       <-- make new branch for pull request
... make source changes
git commit                    <-- commit change to git
git push                      <-- push to your account on github

on github open a pull request, check that CI turns green and then wait for review.
↪ comments.
```

Test your changes:

```
cd test
pytest
```

you can also do extended testing:

```
pytest --cov                  <-- Coverage html report in build/html
pytest --profile               <-- Call profile report in prof
```

13.6.1 Internals

There is no documentation of the architecture (help is welcome), but most classes and methods are documented:

[Pymodbus internals](#)

13.6.2 Generate documentation

Remark Assumes that you have installed documentation tools::

```
pip install "[documentation]"
```

to build do:

```
cd doc
./build_html
```

The documentation is available in <root>/build/html

Remark: this generates a new zip/tgz file of examples which are uploaded.

13.7 License Information

Released under the [BSD License](#)

PYTHON MODULE INDEX

p

- `pymodbus`, 146
- `pymodbus.exceptions`, 147
- `pymodbus.framer.FramerAscii`, 141
- `pymodbus.framer.FramerRTU`, 141
- `pymodbus.framer.FramerSocket`, 143
- `pymodbus.framer.FramerTLS`, 143
- `pymodbus.pdu.bit_message`, 151
- `pymodbus.pdu.decoders`, 149
- `pymodbus.pdu.device`, 171
- `pymodbus.pdu.diag_message`, 154
- `pymodbus.pdu.events`, 173
- `pymodbus.pdu.file_message`, 160
- `pymodbus.pdu.mei_message`, 163
- `pymodbus.pdu.other_message`, 163
- `pymodbus.pdu.register_message`, 167
- `pymodbus.server`, 37
- `pymodbus.server.simulator.http_server`, 64
- `pymodbus.transaction`, 148
- `pymodbus.utilities`, 149

INDEX

A

- `action` (*pymodbus.simulator.SimData* attribute), 81
 - `action_add()` (*pymodbus.server.ModbusSimulatorServer* method), 39
 - `action_add()` (*pymodbus.server.simulator.http_server.ModbusSimulatorServer* method), 67
 - `action_clear()` (*pymodbus.server.ModbusSimulatorServer* method), 39
 - `action_clear()` (*pymodbus.server.simulator.http_server.ModbusSimulatorServer* method), 66
 - `action_monitor()` (*pymodbus.server.ModbusSimulatorServer* method), 39
 - `action_monitor()` (*pymodbus.server.simulator.http_server.ModbusSimulatorServer* method), 67
 - `action_reset()` (*pymodbus.server.ModbusSimulatorServer* method), 39
 - `action_reset()` (*pymodbus.server.simulator.http_server.ModbusSimulatorServer* method), 66
 - `action_set()` (*pymodbus.server.ModbusSimulatorServer* method), 39
 - `action_set()` (*pymodbus.server.simulator.http_server.ModbusSimulatorServer* method), 67
 - `action_simulate()` (*pymodbus.server.ModbusSimulatorServer* method), 39
 - `action_simulate()` (*pymodbus.server.simulator.http_server.ModbusSimulatorServer* method), 67
 - `action_stop()` (*pymodbus.server.ModbusSimulatorServer* method), 39
 - `action_stop()` (*pymodbus.server.simulator.http_server.ModbusSimulatorServer* method), 66
 - `active_server` (*pymodbus.server.ModbusBaseServer* attribute), 37
 - `add_pdu()` (*pymodbus.pdu.decoders.DecodePDU* class method), 149
 - `add_sub_pdu()` (*pymodbus.pdu.decoders.DecodePDU* class method), 149
 - `address` (*pymodbus.simulator.SimData* attribute), 81
 - ASCII (*pymodbus.FramerType* attribute), 146
 - `AsyncModbusSerialClient` (class in *pymodbus.client*), 14
 - `AsyncModbusTcpClient` (class in *pymodbus.client*), 17
 - `AsyncModbusTlsClient` (class in *pymodbus.client*), 19
 - `AsyncModbusUdpClient` (class in *pymodbus.client*), 22
- ### B
- BASIC (*pymodbus.constants.DeviceInformation* attribute), 144
 - BITS (*pymodbus.constants.DataType* attribute), 80
 - `build_html_calls()` (*pymodbus.server.ModbusSimulatorServer* method), 39
 - `build_html_calls()` (*pymodbus.server.simulator.http_server.ModbusSimulatorServer* method), 66
 - `build_html_log()` (*pymodbus.server.ModbusSimulatorServer* method), 39
 - `build_html_log()` (*pymodbus.server.simulator.http_server.ModbusSimulatorServer* method), 66
 - `build_html_registers()` (*pymodbus.server.ModbusSimulatorServer* method), 39
 - `build_html_registers()` (*pymodbus.server.simulator.http_server.ModbusSimulatorServer* method), 66
 - `build_html_server()` (*pymodbus.server.ModbusSimulatorServer* method), 39
 - `build_html_server()` (*pymodbus.server.simulator.http_server.ModbusSimulatorServer* method), 66

<i>bus.server.simulator.http_server.ModbusSimulatorServer</i>	37		
<i>method</i>), 66		<i>callback_disconnected()</i>	(<i>pymod-</i>
<i>build_json_calls()</i>	(<i>pymod-</i>	<i>bus.transaction.TransactionManager</i>	<i>method</i>),
<i>bus.server.ModbusSimulatorServer</i>	<i>method</i>),	148	
39		<i>callback_new_connection()</i>	(<i>pymod-</i>
<i>build_json_calls()</i>	(<i>pymod-</i>	<i>bus.server.ModbusBaseServer</i>	<i>method</i>),
<i>bus.server.simulator.http_server.ModbusSimulatorServer</i>	37		
<i>method</i>), 66		<i>callback_new_connection()</i>	(<i>pymod-</i>
<i>build_json_log()</i>	(<i>pymod-</i>	<i>bus.transaction.TransactionManager</i>	<i>method</i>),
<i>bus.server.ModbusSimulatorServer</i>	<i>method</i>),	148	
39		<i>CallTracer</i>	(<i>class</i> in <i>pymod-</i>
<i>build_json_log()</i>	(<i>pymod-</i>	<i>bus.server.simulator.http_server</i>), 64	
<i>bus.server.simulator.http_server.ModbusSimulatorServer</i>	<i>method</i>), 66	<i>CallTypeMonitor</i>	(<i>class</i> in <i>pymod-</i>
<i>method</i>), 66		<i>bus.server.simulator.http_server</i>), 64	
<i>build_json_registers()</i>	(<i>pymod-</i>	<i>CallTypeResponse</i>	(<i>class</i> in <i>pymod-</i>
<i>bus.server.ModbusSimulatorServer</i>	<i>method</i>),	<i>bus.server.simulator.http_server</i>), 65	
39		<i>ChangeAsciiInputDelimiterRequest</i>	(<i>class</i> in <i>py-</i>
<i>build_json_registers()</i>	(<i>pymod-</i>	<i>modbus.pdu.diag_message</i>), 154	
<i>bus.server.simulator.http_server.ModbusSimulatorServer</i>	<i>method</i>), 66	<i>ChangeAsciiInputDelimiterResponse</i>	(<i>class</i> in <i>py-</i>
<i>method</i>), 66		<i>modbus.pdu.diag_message</i>), 154	
<i>build_json_server()</i>	(<i>pymod-</i>	<i>CLEAR_STATISTICS</i>	(<i>pymod-</i>
<i>bus.server.ModbusSimulatorServer</i>	<i>method</i>),	<i>bus.constants.ModbusPlusOperation</i> attribute),	
39		145	
<i>build_json_server()</i>	(<i>pymod-</i>	<i>ClearCountersRequest</i>	(<i>class</i> in <i>pymod-</i>
<i>bus.server.simulator.http_server.ModbusSimulatorServer</i>	<i>method</i>), 66	<i>bus.pdu.diag_message</i>), 154	
<i>method</i>), 66		<i>ClearCountersResponse</i>	(<i>class</i> in <i>pymod-</i>
<i>build_registers_from_value()</i>	(<i>pymod-</i>	<i>bus.pdu.diag_message</i>), 154	
<i>bus.datastore.ModbusSimulatorContext</i>	<i>class</i>	<i>ClearOverrunCountRequest</i>	(<i>class</i> in <i>pymod-</i>
<i>method</i>), 141		<i>bus.pdu.diag_message</i>), 154	
<i>build_value_from_registers()</i>	(<i>pymod-</i>	<i>ClearOverrunCountResponse</i>	(<i>class</i> in <i>pymod-</i>
<i>bus.datastore.ModbusSimulatorContext</i>	<i>class</i>	<i>bus.pdu.diag_message</i>), 155	
<i>method</i>), 141		<i>close()</i>	(<i>pymodbus.client.base.ModbusBaseClient</i>
		<i>method</i>), 13	
C		<i>close()</i>	(<i>pymodbus.client.base.ModbusBaseSyncClient</i>
<i>calculateRtuFrameSize()</i>	(<i>pymod-</i>	<i>method</i>), 14	
<i>bus.pdu.file_message.ReadFifoQueueResponse</i>		<i>close()</i>	(<i>pymodbus.client.ModbusSerialClient</i> <i>method</i>),
<i>class method</i>), 161		16	
<i>calculateRtuFrameSize()</i>	(<i>pymod-</i>	<i>close()</i>	(<i>pymodbus.client.ModbusTcpClient</i> <i>method</i>), 19
<i>bus.pdu.mei_message.ReadDeviceInformationResponse</i>		<i>CommunicationRestartEvent</i>	(<i>class</i> in <i>pymod-</i>
<i>class method</i>), 163		<i>bus.pdu.events</i>), 173	
<i>callback_connected()</i>	(<i>pymod-</i>	<i>connect()</i>	(<i>pymodbus.client.base.ModbusBaseClient</i>
<i>bus.server.ModbusBaseServer</i>	<i>method</i>),	<i>method</i>), 13	
37		<i>connect()</i>	(<i>pymodbus.client.base.ModbusBaseSyncClient</i>
<i>callback_connected()</i>	(<i>pymod-</i>	<i>method</i>), 14	
<i>bus.transaction.TransactionManager</i>	<i>method</i>),	<i>connect()</i>	(<i>pymodbus.client.ModbusSerialClient</i>
148		<i>method</i>), 16	
<i>callback_data()</i>	(<i>pymod-</i>	<i>connect()</i>	(<i>pymodbus.client.ModbusTcpClient</i> <i>method</i>),
<i>bus.server.ModbusBaseServer</i>	<i>method</i>),	19	
37		<i>connect()</i>	(<i>pymodbus.client.ModbusTlsClient</i> <i>method</i>),
<i>callback_data()</i>	(<i>pymod-</i>	22	
<i>bus.transaction.TransactionManager</i>	<i>method</i>),	<i>connected</i>	(<i>pymodbus.client.base.ModbusBaseClient</i>
148		<i>property</i>), 12	
<i>callback_disconnected()</i>	(<i>pymod-</i>	<i>connected</i>	(<i>pymodbus.client.ModbusSerialClient</i> <i>prop-</i>
<i>bus.server.ModbusBaseServer</i>	<i>method</i>),	<i>erty</i>), 16	

connected (*pymodbus.client.ModbusTcpClient* property), 19
 connected (*pymodbus.client.ModbusTlsClient* property), 22
 connected (*pymodbus.client.ModbusUdpClient* property), 24
 ConnectionException, 147
 convert_from_registers() (*pymodbus.client.mixin.ModbusClientMixin* class method), 35
 convert_to_registers() (*pymodbus.client.mixin.ModbusClientMixin* class method), 36
 count (*pymodbus.simulator.SimData* attribute), 81
 create() (*pymodbus.datastore.ModbusSparseDataBlock* class method), 137

D

DataType (class in *pymodbus.constants*), 79
 datatype (*pymodbus.simulator.SimData* attribute), 81
 decode() (*pymodbus.ExceptionResponse* method), 146
 decode() (*pymodbus.pdu.bit_message.ReadCoilsRequest* method), 151
 decode() (*pymodbus.pdu.bit_message.ReadCoilsResponse* method), 152
 decode() (*pymodbus.pdu.bit_message.WriteMultipleCoilsRequest* method), 152
 decode() (*pymodbus.pdu.bit_message.WriteMultipleCoilsResponse* method), 153
 decode() (*pymodbus.pdu.bit_message.WriteSingleCoilResponse* method), 153
 decode() (*pymodbus.pdu.decoders.DecodePDU* method), 149
 decode() (*pymodbus.pdu.diag_message.DiagnosticBase* method), 155
 decode() (*pymodbus.pdu.events.CommunicationRestartEvent* method), 173
 decode() (*pymodbus.pdu.events.EnteredListenModeEvent* method), 173
 decode() (*pymodbus.pdu.events.ModbusEvent* method), 174
 decode() (*pymodbus.pdu.events.RemoteReceiveEvent* method), 174
 decode() (*pymodbus.pdu.events.RemoteSendEvent* method), 175
 decode() (*pymodbus.pdu.file_message.ReadFifoQueueRequest* method), 161
 decode() (*pymodbus.pdu.file_message.ReadFifoQueueResponse* method), 161
 decode() (*pymodbus.pdu.file_message.ReadFileRecordRequest* method), 161
 decode() (*pymodbus.pdu.file_message.ReadFileRecordResponse* method), 162
 decode() (*pymodbus.pdu.file_message.WriteFileRecordRequest* method), 162
 decode() (*pymodbus.pdu.file_message.WriteFileRecordResponse* method), 162
 decode() (*pymodbus.pdu.mei_message.ReadDeviceInformationRequest* method), 163
 decode() (*pymodbus.pdu.mei_message.ReadDeviceInformationResponse* method), 163
 decode() (*pymodbus.pdu.other_message.GetCommEventCounterRequest* method), 164
 decode() (*pymodbus.pdu.other_message.GetCommEventCounterResponse* method), 164
 decode() (*pymodbus.pdu.other_message.GetCommEventLogRequest* method), 164
 decode() (*pymodbus.pdu.other_message.GetCommEventLogResponse* method), 165
 decode() (*pymodbus.pdu.other_message.ReadExceptionStatusRequest* method), 165
 decode() (*pymodbus.pdu.other_message.ReadExceptionStatusResponse* method), 165
 decode() (*pymodbus.pdu.other_message.ReportDeviceIdRequest* method), 166
 decode() (*pymodbus.pdu.other_message.ReportDeviceIdResponse* method), 166
 decode() (*pymodbus.pdu.register_message.MaskWriteRegisterRequest* method), 167
 decode() (*pymodbus.pdu.register_message.MaskWriteRegisterResponse* method), 168
 decode() (*pymodbus.pdu.register_message.ReadHoldingRegistersRequest* method), 168
 decode() (*pymodbus.pdu.register_message.ReadHoldingRegistersResponse* method), 169
 decode() (*pymodbus.pdu.register_message.ReadWriteMultipleRegistersRequest* method), 169
 decode() (*pymodbus.pdu.register_message.WriteMultipleRegistersRequest* method), 170
 decode() (*pymodbus.pdu.register_message.WriteMultipleRegistersResponse* method), 171
 decode() (*pymodbus.pdu.register_message.WriteSingleRegisterResponse* method), 171
 decode_sub_function_code() (*pymodbus.pdu.diag_message.DiagnosticBase* class method), 155
 decode_sub_function_code() (*pymodbus.pdu.mei_message.ReadDeviceInformationRequest* class method), 163
 decode_sub_function_code() (*pymodbus.pdu.mei_message.ReadDeviceInformationResponse* class method), 163
 DecodePDU (class in *pymodbus.pdu.decoders*), 149
 default (*pymodbus.simulator.SimData* attribute), 82
 device_ids() (*pymodbus.datastore.ModbusServerContext* method), 139

DeviceInformationFactory	(class in pymodbus.pdu.device), 171	bus.client.mixin.ModbusClientMixin method), 30
diag_change_ascii_input_delimeter()	(pymodbus.client.mixin.ModbusClientMixin method), 28	diag_restart_communication() (pymodbus.client.mixin.ModbusClientMixin method), 27
diag_clear_counters()	(pymodbus.client.mixin.ModbusClientMixin method), 28	DiagnosticBase (class in pymodbus.pdu.diag_message), 155
diag_clear_overrun_counter()	(pymodbus.client.mixin.ModbusClientMixin method), 31	dict_property() (in module pymodbus.utilities), 149
diag_force_listen_only()	(pymodbus.client.mixin.ModbusClientMixin method), 28	dummy_trace_connect() (pymodbus.transaction.TransactionManager method), 148
diag_get_comm_event_counter()	(pymodbus.client.mixin.ModbusClientMixin method), 31	dummy_trace_packet() (pymodbus.transaction.TransactionManager method), 148
diag_get_comm_event_log()	(pymodbus.client.mixin.ModbusClientMixin method), 32	dummy_trace_pdu() (pymodbus.transaction.TransactionManager method), 148
diag_getclear_modbus_response()	(pymodbus.client.mixin.ModbusClientMixin method), 31	E
diag_query_data()	(pymodbus.client.mixin.ModbusClientMixin method), 27	encode() (pymodbus.ExceptionResponse method), 146
diag_read_bus_char_overrun_count()	(pymodbus.client.mixin.ModbusClientMixin method), 30	encode() (pymodbus.pdu.bit_message.ReadCoilsRequest method), 151
diag_read_bus_comm_error_count()	(pymodbus.client.mixin.ModbusClientMixin method), 29	encode() (pymodbus.pdu.bit_message.ReadCoilsResponse method), 152
diag_read_bus_exception_error_count()	(pymodbus.client.mixin.ModbusClientMixin method), 29	encode() (pymodbus.pdu.bit_message.WriteMultipleCoilsRequest method), 153
diag_read_bus_message_count()	(pymodbus.client.mixin.ModbusClientMixin method), 29	encode() (pymodbus.pdu.bit_message.WriteMultipleCoilsResponse method), 153
diag_read_device_busy_count()	(pymodbus.client.mixin.ModbusClientMixin method), 30	encode() (pymodbus.pdu.bit_message.WriteSingleCoilResponse method), 154
diag_read_device_message_count()	(pymodbus.client.mixin.ModbusClientMixin method), 29	encode() (pymodbus.pdu.device.ModbusPlusStatistics method), 172
diag_read_device_nak_count()	(pymodbus.client.mixin.ModbusClientMixin method), 30	encode() (pymodbus.pdu.diag_message.DiagnosticBase method), 155
diag_read_device_no_response_count()	(pymodbus.client.mixin.ModbusClientMixin method), 30	encode() (pymodbus.pdu.diag_message.GetClearModbusPlusRequest method), 156
diag_read_diagnostic_register()	(pymodbus.client.mixin.ModbusClientMixin method), 28	encode() (pymodbus.pdu.events.CommunicationRestartEvent method), 173
diag_read_iop_overrun_count()	(pymod-	encode() (pymodbus.pdu.events.EnteredListenModeEvent method), 174
		encode() (pymodbus.pdu.events.ModbusEvent method), 174
		encode() (pymodbus.pdu.events.RemoteReceiveEvent method), 174
		encode() (pymodbus.pdu.events.RemoteSendEvent method), 175
		encode() (pymodbus.pdu.file_message.ReadFifoQueueRequest method), 161
		encode() (pymodbus.pdu.file_message.ReadFifoQueueResponse method), 161
		encode() (pymodbus.pdu.file_message.ReadFileRecordRequest method), 161
		encode() (pymodbus.pdu.file_message.ReadFileRecordResponse method), 162

`encode()` (`pymodbus.pdu.file_message.WriteFileRecordRequest` attribute), 162
`encode()` (`pymodbus.pdu.file_message.WriteFileRecordResponse` attribute), 162
`encode()` (`pymodbus.pdu.mei_message.ReadDeviceInformationRequest` attribute), 163
`encode()` (`pymodbus.pdu.mei_message.ReadDeviceInformationResponse` attribute), 163
`encode()` (`pymodbus.pdu.other_message.GetCommEventCounterRequest` attribute), 164
`encode()` (`pymodbus.pdu.other_message.GetCommEventCounterResponse` attribute), 164
`encode()` (`pymodbus.pdu.other_message.GetCommEventLogRequest` attribute), 164
`encode()` (`pymodbus.pdu.other_message.GetCommEventLogResponse` attribute), 165
`encode()` (`pymodbus.pdu.other_message.ReadExceptionStatusRequest` attribute), 165
`encode()` (`pymodbus.pdu.other_message.ReadExceptionStatusResponse` attribute), 165
`encode()` (`pymodbus.pdu.other_message.ReportDeviceIdRequest` attribute), 166
`encode()` (`pymodbus.pdu.other_message.ReportDeviceIdResponse` attribute), 166
`encode()` (`pymodbus.pdu.register_message.MaskWriteRegisterRequest` attribute), 168
`encode()` (`pymodbus.pdu.register_message.MaskWriteRegisterResponse` attribute), 168
`encode()` (`pymodbus.pdu.register_message.ReadHoldingRegistersRequest` attribute), 168
`encode()` (`pymodbus.pdu.register_message.ReadHoldingRegistersResponse` attribute), 169
`encode()` (`pymodbus.pdu.register_message.ReadWriteMultipleRegistersRequest` attribute), 169
`encode()` (`pymodbus.pdu.register_message.WriteMultipleRegistersRequest` attribute), 170
`encode()` (`pymodbus.pdu.register_message.WriteMultipleRegistersResponse` attribute), 171
`encode()` (`pymodbus.pdu.register_message.WriteSingleRegisterResponse` attribute), 171
`EnteredListenModeEvent` (class in `pymodbus.pdu.events`), 173
`ExceptionResponse` (class in `pymodbus`), 146
`execute()` (`pymodbus.client.mixin.ModbusClientMixin` method), 25
`execute()` (`pymodbus.transaction.TransactionManager` method), 148
`EXTENDED` (`pymodbus.constants.DeviceInformation` attribute), 144
F
`file_number` (`pymodbus.pdu.file_message.FileRecord` attribute), 160
`FileRecord` (class in `pymodbus.pdu.file_message`), 160
`FileRecordRequest` (`pymodbus.constants.DataType` attribute), 80
`FileRecordResponse` (`pymodbus.constants.DataType` attribute), 80
`ForceListenOnlyModeRequest` (class in `pymodbus.pdu.diag_message`), 155
`ForceListenOnlyModeResponse` (class in `pymodbus.pdu.diag_message`), 155
`FrameType` (class in `pymodbus`), 146
`function_code` (`pymodbus.pdu.bit_message.ReadCoilsRequest` attribute), 152
`function_code` (`pymodbus.pdu.bit_message.ReadCoilsResponse` attribute), 152
`function_code` (`pymodbus.pdu.bit_message.ReadDiscreteInputsRequest` attribute), 152
`function_code` (`pymodbus.pdu.bit_message.ReadDiscreteInputsResponse` attribute), 152
`function_code` (`pymodbus.pdu.bit_message.WriteMultipleCoilsRequest` attribute), 153
`function_code` (`pymodbus.pdu.bit_message.WriteMultipleCoilsResponse` attribute), 153
`function_code` (`pymodbus.pdu.bit_message.WriteSingleCoilResponse` attribute), 154
`function_code` (`pymodbus.pdu.diag_message.DiagnosticBase` attribute), 155
`function_code` (`pymodbus.pdu.file_message.ReadFifoQueueRequest` attribute), 161
`function_code` (`pymodbus.pdu.file_message.ReadFifoQueueResponse` attribute), 161
`function_code` (`pymodbus.pdu.file_message.ReadFileRecordRequest` attribute), 161
`function_code` (`pymodbus.pdu.file_message.ReadFileRecordResponse` attribute), 162
`function_code` (`pymodbus.pdu.file_message.WriteFileRecordRequest` attribute), 162
`function_code` (`pymodbus.pdu.file_message.WriteFileRecordResponse` attribute), 162
`function_code` (`pymodbus.pdu.mei_message.ReadDeviceInformationRequest` attribute), 163
`function_code` (`pymodbus.pdu.mei_message.ReadDeviceInformationResponse` attribute), 163

attribute), 163	attribute), 171
function_code (pymod- bus.pdu.other_message.GetCommEventCounterRequest attribute), 164	function_code (pymod- bus.pdu.register_message.WriteSingleRegisterResponse attribute), 171
function_code (pymod- bus.pdu.other_message.GetCommEventCounterResponse attribute), 164	G
function_code (pymod- bus.pdu.other_message.GetCommEventLogRequest attribute), 164	generate_ssl() (pymod- bus.client.AsyncModbusTlsClient class method), 20
function_code (pymod- bus.pdu.other_message.GetCommEventLogResponse attribute), 165	generate_ssl() (pymodbus.client.ModbusTlsClient class method), 22
function_code (pymod- bus.pdu.other_message.ReadExceptionStatusRequest attribute), 165	get() (pymodbus.pdu.device.DeviceInformationFactory class method), 172
function_code (pymod- bus.pdu.other_message.ReadExceptionStatusResponse attribute), 165	get_response_pdu_size() (pymod- bus.pdu.bit_message.ReadCoilsRequest method), 152
function_code (pymod- bus.pdu.other_message.ReadExceptionStatusResponse attribute), 165	get_response_pdu_size() (pymod- bus.pdu.bit_message.WriteMultipleCoilsRequest method), 153
function_code (pymod- bus.pdu.other_message.ReportDeviceIdRequest attribute), 166	get_response_pdu_size() (pymod- bus.pdu.bit_message.WriteSingleCoilRequest method), 153
function_code (pymod- bus.pdu.other_message.ReportDeviceIdResponse attribute), 166	get_response_pdu_size() (pymod- bus.pdu.diag_message.DiagnosticBase method), 155
function_code (pymod- bus.pdu.register_message.MaskWriteRegisterRequest attribute), 168	get_response_pdu_size() (pymod- bus.pdu.diag_message.GetClearModbusPlusRequest method), 156
function_code (pymod- bus.pdu.register_message.MaskWriteRegisterResponse attribute), 168	get_response_pdu_size() (pymod- bus.pdu.file_message.ReadFileRecordRequest method), 161
function_code (pymod- bus.pdu.register_message.ReadHoldingRegistersRequest attribute), 168	get_response_pdu_size() (pymod- bus.pdu.file_message.WriteFileRecordRequest method), 162
function_code (pymod- bus.pdu.register_message.ReadHoldingRegistersResponse attribute), 169	get_response_pdu_size() (pymod- bus.pdu.register_message.ReadHoldingRegistersRequest method), 168
function_code (pymod- bus.pdu.register_message.ReadInputRegistersRequest attribute), 169	get_response_pdu_size() (pymod- bus.pdu.register_message.ReadWriteMultipleRegistersRequest method), 170
function_code (pymod- bus.pdu.register_message.ReadInputRegistersResponse attribute), 169	get_response_pdu_size() (pymod- bus.pdu.register_message.WriteMultipleRegistersRequest method), 170
function_code (pymod- bus.pdu.register_message.ReadWriteMultipleRegistersRequest attribute), 169	get_response_pdu_size() (pymod- bus.pdu.register_message.WriteSingleRegisterRequest method), 171
function_code (pymod- bus.pdu.register_message.ReadWriteMultipleRegistersResponse attribute), 170	get_simulator_commandline() (in module pymodbus.server), 43
function_code (pymod- bus.pdu.register_message.WriteMultipleRegistersRequest attribute), 170	GET_STATISTICS (pymod- bus.constants.ModbusPlusOperation attribute), 145
function_code (pymod- bus.pdu.register_message.WriteMultipleRegistersResponse attribute), 170	get_text_register() (pymod- bus.datastore.ModbusSimulatorContext method), 141

- GetClearModbusPlusRequest (class in pymodbus.pdu.diag_message), 156
- GetClearModbusPlusResponse (class in pymodbus.pdu.diag_message), 156
- GetCommEventCounterRequest (class in pymodbus.pdu.other_message), 163
- GetCommEventCounterResponse (class in pymodbus.pdu.other_message), 164
- GetCommEventLogRequest (class in pymodbus.pdu.other_message), 164
- GetCommEventLogResponse (class in pymodbus.pdu.other_message), 164
- getNextTID() (pymodbus.transaction.TransactionManager method), 148
- getValues() (pymodbus.datastore.ModbusDeviceContext method), 138
- getValues() (pymodbus.datastore.ModbusSparseDataBlock method), 138
- ## H
- handle_html() (pymodbus.server.ModbusSimulatorServer method), 39
- handle_html() (pymodbus.server.simulator.http_server.ModbusSimulatorServer method), 66
- handle_html_static() (pymodbus.server.ModbusSimulatorServer method), 40
- handle_html_static() (pymodbus.server.simulator.http_server.ModbusSimulatorServer method), 66
- handle_json() (pymodbus.server.ModbusSimulatorServer method), 40
- handle_json() (pymodbus.server.simulator.http_server.ModbusSimulatorServer method), 66
- helper_handle_submit() (pymodbus.server.ModbusSimulatorServer method), 40
- helper_handle_submit() (pymodbus.server.simulator.http_server.ModbusSimulatorServer method), 66
- hexlify_packets() (in module pymodbus.utilities), 149
- ## I
- id (pymodbus.simulator.SimDevice attribute), 83
- idle_time() (pymodbus.client.base.ModbusBaseSyncClient method), 13
- INT16 (pymodbus.constants.DataType attribute), 80
- INT32 (pymodbus.constants.DataType attribute), 80
- INT64 (pymodbus.constants.DataType attribute), 80
- invalid (pymodbus.simulator.SimData attribute), 82
- is_socket_open() (pymodbus.client.ModbusSerialClient method), 17
- is_socket_open() (pymodbus.client.ModbusTcpClient method), 19
- isError() (pymodbus.ModbusException method), 147
- ## K
- KEEP_READING (pymodbus.constants.MoreData attribute), 146
- ## L
- list_function_codes() (pymodbus.pdu.decoders.DecodePDU method), 149
- lookupPduClass() (pymodbus.pdu.decoders.DecodePDU method), 149
- ## M
- MajorMinorRevision (pymodbus.ModbusDeviceIdentification property), 146
- MajorMinorRevision (pymodbus.pdu.device.ModbusDeviceIdentification property), 172
- mask_write_register() (pymodbus.client.mixin.ModbusClientMixin method), 34
- MaskWriteRegisterRequest (class in pymodbus.pdu.register_message), 167
- MaskWriteRegisterResponse (class in pymodbus.pdu.register_message), 168
- MessageRegisterException, 147
- ModbusBaseClient (class in pymodbus.client.base), 12
- ModbusBaseServer (class in pymodbus.server), 37
- ModbusBaseSyncClient (class in pymodbus.client.base), 13
- ModbusClientMixin (class in pymodbus.client.mixin), 24
- ModbusClientMixin.DATATYPE (class in pymodbus.client.mixin), 35
- ModbusDeviceContext (class in pymodbus.datastore), 138
- ModbusDeviceIdentification (class in pymodbus), 146
- ModbusDeviceIdentification (class in pymodbus.pdu.device), 172
- ModbusEvent (class in pymodbus.pdu.events), 174
- ModbusException, 147
- ModbusIOException, 147

ModbusPlusStatistics (class in pymodbus.pdu.device), 172
 ModbusSerialClient (class in pymodbus.client), 15
 ModbusSerialServer (class in pymodbus.server), 38
 ModbusServerContext (class in pymodbus.datastore), 139
 ModbusSimulatorContext (class in pymodbus.datastore), 139
 ModbusSimulatorServer (class in pymodbus.server), 38
 ModbusSimulatorServer (class in pymodbus.server.simulator.http_server), 65
 ModbusSparseDataBlock (class in pymodbus.datastore), 137
 ModbusTcpClient (class in pymodbus.client), 18
 ModbusTcpServer (class in pymodbus.server), 40
 ModbusTlsClient (class in pymodbus.client), 21
 ModbusTlsServer (class in pymodbus.server), 40
 ModbusUdpClient (class in pymodbus.client), 23
 ModbusUdpServer (class in pymodbus.server), 41
 ModelName (pymodbus.ModbusDeviceIdentification property), 146
 ModelName (pymodbus.pdu.device.ModbusDeviceIdentification property), 172
 module
 pymodbus, 146
 pymodbus.exceptions, 147
 pymodbus.framer.FramerAscii, 141
 pymodbus.framer.FramerRTU, 141
 pymodbus.framer.FramerSocket, 143
 pymodbus.framer.FramerTLS, 143
 pymodbus.pdu.bit_message, 151
 pymodbus.pdu.decoders, 149
 pymodbus.pdu.device, 171
 pymodbus.pdu.diag_message, 154
 pymodbus.pdu.events, 173
 pymodbus.pdu.file_message, 160
 pymodbus.pdu.mei_message, 163
 pymodbus.pdu.other_message, 163
 pymodbus.pdu.register_message, 167
 pymodbus.server, 37
 pymodbus.server.simulator.http_server, 64
 pymodbus.transaction, 148
 pymodbus.utilities, 149

N

NoSuchIdException, 147
 NOTHING (pymodbus.constants.MoreData attribute), 145
 NotImplementedException, 147

O

OFF (pymodbus.constants.ModbusStatus attribute), 145
 offset_address (pymodbus.simulator.SimDevice attribute), 83

ON (pymodbus.constants.ModbusStatus attribute), 145

P

ParameterException, 148
 pdu_send() (pymodbus.transaction.TransactionManager method), 149
 pdu_sub_table (pymodbus.pdu.decoders.DecodePDU attribute), 150
 pdu_table (pymodbus.pdu.decoders.DecodePDU attribute), 150
 ProductCode (pymodbus.ModbusDeviceIdentification property), 146
 ProductCode (pymodbus.pdu.device.ModbusDeviceIdentification property), 172
 ProductName (pymodbus.ModbusDeviceIdentification property), 146
 ProductName (pymodbus.pdu.device.ModbusDeviceIdentification property), 172
 pymodbus
 module, 146
 pymodbus.exceptions
 module, 147
 pymodbus.framer.FramerAscii
 module, 141
 pymodbus.framer.FramerRTU
 module, 141
 pymodbus.framer.FramerSocket
 module, 143
 pymodbus.framer.FramerTLS
 module, 143
 pymodbus.pdu.bit_message
 module, 151
 pymodbus.pdu.decoders
 module, 149
 pymodbus.pdu.device
 module, 171
 pymodbus.pdu.diag_message
 module, 154
 pymodbus.pdu.events
 module, 173
 pymodbus.pdu.file_message
 module, 160
 pymodbus.pdu.mei_message
 module, 163
 pymodbus.pdu.other_message
 module, 163
 pymodbus.pdu.register_message
 module, 167
 pymodbus.server
 module, 37
 pymodbus.server.simulator.http_server
 module, 64
 pymodbus.transaction
 module, 148

pymodbus.utilities
module, 149

pymodbus_apply_logging_config() (in module py-
modbus), 147

R

read_coils() (pymod-
bus.client.mixin.ModbusClientMixin method),
25

read_device_information() (pymod-
bus.client.mixin.ModbusClientMixin method),
35

read_discrete_inputs() (pymod-
bus.client.mixin.ModbusClientMixin method),
25

read_exception_status() (pymod-
bus.client.mixin.ModbusClientMixin method),
27

read_fifo_queue() (pymod-
bus.client.mixin.ModbusClientMixin method),
34

read_file_record() (pymod-
bus.client.mixin.ModbusClientMixin method),
33

read_holding_registers() (pymod-
bus.client.mixin.ModbusClientMixin method),
26

read_input_registers() (pymod-
bus.client.mixin.ModbusClientMixin method),
26

ReadCoilsRequest (class in pymod-
bus.pdu.bit_message), 151

ReadCoilsResponse (class in pymod-
bus.pdu.bit_message), 152

ReadDeviceInformationRequest (class in pymod-
bus.pdu.mei_message), 163

ReadDeviceInformationResponse (class in pymod-
bus.pdu.mei_message), 163

ReadDiscreteInputsRequest (class in pymod-
bus.pdu.bit_message), 152

ReadDiscreteInputsResponse (class in pymod-
bus.pdu.bit_message), 152

ReadExceptionStatusRequest (class in pymod-
bus.pdu.other_message), 165

ReadExceptionStatusResponse (class in pymod-
bus.pdu.other_message), 165

ReadFifoQueueRequest (class in pymod-
bus.pdu.file_message), 161

ReadFifoQueueResponse (class in pymod-
bus.pdu.file_message), 161

ReadFileRecordRequest (class in pymod-
bus.pdu.file_message), 161

ReadFileRecordResponse (class in pymod-
bus.pdu.file_message), 162

ReadHoldingRegistersRequest (class in pymod-
bus.pdu.register_message), 168

ReadHoldingRegistersResponse (class in pymod-
bus.pdu.register_message), 168

ReadInputRegistersRequest (class in pymod-
bus.pdu.register_message), 169

ReadInputRegistersResponse (class in pymod-
bus.pdu.register_message), 169

readonly (pymodbus.simulator.SimData attribute), 81

readwrite_registers() (pymod-
bus.client.mixin.ModbusClientMixin method),
34

ReadWriteMultipleRegistersRequest (class in py-
modbus.pdu.register_message), 169

ReadWriteMultipleRegistersResponse (class in py-
modbus.pdu.register_message), 170

READY (pymodbus.constants.ModbusStatus attribute), 145

record_data (pymodbus.pdu.file_message.FileRecord
attribute), 160

record_length (pymod-
bus.pdu.file_message.FileRecord attribute),
161

record_number (pymod-
bus.pdu.file_message.FileRecord attribute),
161

recv() (pymodbus.client.ModbusSerialClient method),
16

recv() (pymodbus.client.ModbusTcpClient method), 19

register() (pymodbus.client.base.ModbusBaseClient
method), 13

register() (pymodbus.client.base.ModbusBaseSyncClient
method), 13

register() (pymodbus.datastore.ModbusDeviceContext
method), 139

register() (pymodbus.pdu.decoders.DecodePDU
method), 151

register_count (pymodbus.simulator.SimData at-
tribute), 82

REGISTERS (pymodbus.constants.DataType attribute), 80

registers (pymodbus.simulator.SimDevice attribute),
83

REGULAR (pymodbus.constants.DeviceInformation
attribute), 144

RemoteReceiveEvent (class in pymodbus.pdu.events),
174

RemoteSendEvent (class in pymodbus.pdu.events), 175

report_device_id() (pymod-
bus.client.mixin.ModbusClientMixin method),
33

ReportDeviceIdRequest (class in pymod-
bus.pdu.other_message), 165

ReportDeviceIdResponse (class in pymod-
bus.pdu.other_message), 166

reset() (pymodbus.datastore.ModbusDeviceContext

- method*), 138
- `reset()` (*pymodbus.datastore.ModbusSparseDataBlock* *method*), 138
- `reset()` (*pymodbus.pdu.device.ModbusPlusStatistics* *method*), 173
- `RestartCommunicationsOptionRequest` (*class in pymodbus.pdu.diag_message*), 156
- `RestartCommunicationsOptionResponse` (*class in pymodbus.pdu.diag_message*), 156
- `ReturnBusCommunicationErrorCountRequest` (*class in pymodbus.pdu.diag_message*), 156
- `ReturnBusCommunicationErrorCountResponse` (*class in pymodbus.pdu.diag_message*), 157
- `ReturnBusExceptionErrorCountRequest` (*class in pymodbus.pdu.diag_message*), 157
- `ReturnBusExceptionErrorCountResponse` (*class in pymodbus.pdu.diag_message*), 157
- `ReturnBusMessageCountRequest` (*class in pymodbus.pdu.diag_message*), 157
- `ReturnBusMessageCountResponse` (*class in pymodbus.pdu.diag_message*), 157
- `ReturnDeviceBusCharacterOverrunCountRequest` (*class in pymodbus.pdu.diag_message*), 157
- `ReturnDeviceBusCharacterOverrunCountResponse` (*class in pymodbus.pdu.diag_message*), 158
- `ReturnDeviceBusyCountRequest` (*class in pymodbus.pdu.diag_message*), 158
- `ReturnDeviceBusyCountResponse` (*class in pymodbus.pdu.diag_message*), 158
- `ReturnDeviceMessageCountRequest` (*class in pymodbus.pdu.diag_message*), 158
- `ReturnDeviceMessageCountResponse` (*class in pymodbus.pdu.diag_message*), 158
- `ReturnDeviceNAKCountRequest` (*class in pymodbus.pdu.diag_message*), 159
- `ReturnDeviceNAKCountResponse` (*class in pymodbus.pdu.diag_message*), 159
- `ReturnDeviceNoResponseCountRequest` (*class in pymodbus.pdu.diag_message*), 159
- `ReturnDeviceNoResponseCountResponse` (*class in pymodbus.pdu.diag_message*), 159
- `ReturnDiagnosticRegisterRequest` (*class in pymodbus.pdu.diag_message*), 159
- `ReturnDiagnosticRegisterResponse` (*class in pymodbus.pdu.diag_message*), 160
- `ReturnIopOverrunCountRequest` (*class in pymodbus.pdu.diag_message*), 160
- `ReturnIopOverrunCountResponse` (*class in pymodbus.pdu.diag_message*), 160
- `ReturnQueryDataRequest` (*class in pymodbus.pdu.diag_message*), 160
- `ReturnQueryDataResponse` (*class in pymodbus.pdu.diag_message*), 160
- `RTU` (*pymodbus.FramerType* *attribute*), 146
- `rtu_byte_count_pos` (*pymodbus.pdu.bit_message.ReadCoilsResponse* *attribute*), 152
- `rtu_byte_count_pos` (*pymodbus.pdu.bit_message.WriteMultipleCoilsRequest* *attribute*), 153
- `rtu_byte_count_pos` (*pymodbus.pdu.file_message.ReadFileRecordRequest* *attribute*), 161
- `rtu_byte_count_pos` (*pymodbus.pdu.file_message.ReadFileRecordResponse* *attribute*), 162
- `rtu_byte_count_pos` (*pymodbus.pdu.file_message.WriteFileRecordRequest* *attribute*), 162
- `rtu_byte_count_pos` (*pymodbus.pdu.file_message.WriteFileRecordResponse* *attribute*), 162
- `rtu_byte_count_pos` (*pymodbus.pdu.other_message.GetCommEventLogResponse* *attribute*), 165
- `rtu_byte_count_pos` (*pymodbus.pdu.other_message.ReportDeviceIdResponse* *attribute*), 166
- `rtu_byte_count_pos` (*pymodbus.pdu.register_message.ReadHoldingRegistersResponse* *attribute*), 169
- `rtu_byte_count_pos` (*pymodbus.pdu.register_message.ReadWriteMultipleRegistersRequest* *attribute*), 170
- `rtu_byte_count_pos` (*pymodbus.pdu.register_message.WriteMultipleRegistersRequest* *attribute*), 170
- `rtu_frame_size` (*pymodbus.ExceptionResponse* *attribute*), 146
- `rtu_frame_size` (*pymodbus.pdu.bit_message.ReadCoilsRequest* *attribute*), 152
- `rtu_frame_size` (*pymodbus.pdu.bit_message.WriteMultipleCoilsResponse* *attribute*), 153
- `rtu_frame_size` (*pymodbus.pdu.bit_message.WriteSingleCoilResponse* *attribute*), 154
- `rtu_frame_size` (*pymodbus.pdu.diag_message.DiagnosticBase* *attribute*), 155
- `rtu_frame_size` (*pymodbus.pdu.file_message.ReadFifoQueueRequest* *attribute*), 161
- `rtu_frame_size` (*pymodbus.pdu.mei_message.ReadDeviceInformationRequest* *attribute*), 163
- `rtu_frame_size` (*pymod-*

PyModbus, Release 4.0.0dev6

	<i>bus.pdu.diag_message.ClearOverrunCountRequest</i> attribute), 154		<i>bus.pdu.diag_message.ReturnDeviceBusyCountResponse</i> attribute), 158
sub_function_code	(pymod- <i>bus.pdu.diag_message.ClearOverrunCountResponse</i> attribute), 155	sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnDeviceMessageCountRequest</i> attribute), 158
sub_function_code	(pymod- <i>bus.pdu.diag_message.DiagnosticBase</i> at- tribute), 155	sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnDeviceMessageCountResponse</i> attribute), 159
sub_function_code	(pymod- <i>bus.pdu.diag_message.ForceListenOnlyModeRequest</i> attribute), 155	sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnDeviceNAKCountRequest</i> attribute), 159
sub_function_code	(pymod- <i>bus.pdu.diag_message.ForceListenOnlyModeResponse</i> attribute), 155	sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnDeviceNAKCountResponse</i> attribute), 159
sub_function_code	(pymod- <i>bus.pdu.diag_message.GetClearModbusPlusRequest</i> attribute), 156	sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnDeviceNoResponseCountRequest</i> attribute), 159
sub_function_code	(pymod- <i>bus.pdu.diag_message.GetClearModbusPlusResponse</i> attribute), 156	sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnDeviceNoResponseCountResponse</i> attribute), 159
sub_function_code	(pymod- <i>bus.pdu.diag_message.RestartCommunicationsOptionRequest</i> attribute), 156	sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnDiagnosticRegisterRequest</i> attribute), 159
sub_function_code	(pymod- <i>bus.pdu.diag_message.RestartCommunicationsOptionResponse</i> attribute), 156	sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnDiagnosticRegisterResponse</i> attribute), 160
sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnBusCommunicationErrorCountRequest</i> attribute), 156	sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnIopOverrunCountRequest</i> attribute), 160
sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnBusCommunicationErrorCountResponse</i> attribute), 157	sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnIopOverrunCountResponse</i> attribute), 160
sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnBusExceptionErrorCountRequest</i> attribute), 157	sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnQueryDataRequest</i> attribute), 160
sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnBusExceptionErrorCountResponse</i> attribute), 157	sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnQueryDataResponse</i> attribute), 160
sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnBusMessageCountRequest</i> attribute), 157	sub_function_code	(pymod- <i>bus.pdu.mei_message.ReadDeviceInformationRequest</i> attribute), 163
sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnBusMessageCountResponse</i> attribute), 157	sub_function_code	(pymod- <i>bus.pdu.mei_message.ReadDeviceInformationResponse</i> attribute), 163
sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnDeviceBusCharacterOverrunCountRequest</i> attribute), 158	summary()	(pymodbus.ModbusDeviceIdentification method), 172
sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnDeviceBusCharacterOverrunCountResponse</i> attribute), 158	summary()	(pymodbus.pdu.device.ModbusDeviceIdentification method), 173
sub_function_code	(pymod- <i>bus.pdu.diag_message.ReturnDeviceBusyCountRequest</i> attribute), 158	sync_execute()	(pymod- <i>bus.transaction.TransactionManager</i> method), 149
sub_function_code	(pymod-	sync_get_response()	(pymod-

bus.transaction.TransactionManager method), 149

T

TLS (*pymodbus.FramerType* attribute), 146

TransactionManager (class in *pymodbus.transaction*), 148

type_check (*pymodbus.simulator.SimDevice* attribute), 83

type_size (*pymodbus.simulator.SimData* attribute), 82

U

UINT16 (*pymodbus.constants.DataType* attribute), 80

UINT32 (*pymodbus.constants.DataType* attribute), 80

UINT64 (*pymodbus.constants.DataType* attribute), 80

update() (*pymodbus.ModbusDeviceIdentification* method), 147

update() (*pymodbus.pdu.device.ModbusDeviceIdentification* method), 172

update_datastore() (*pymodbus.pdu.bit_message.ReadCoilsRequest* method), 152

update_datastore() (*pymodbus.pdu.bit_message.WriteMultipleCoilsRequest* method), 153

update_datastore() (*pymodbus.pdu.bit_message.WriteSingleCoilRequest* method), 153

update_datastore() (*pymodbus.pdu.diag_message.ChangeAsciiInputDelimiterRequest* method), 154

update_datastore() (*pymodbus.pdu.diag_message.ClearCountersRequest* method), 154

update_datastore() (*pymodbus.pdu.diag_message.ClearOverrunCountRequest* method), 154

update_datastore() (*pymodbus.pdu.diag_message.DiagnosticBase* method), 155

update_datastore() (*pymodbus.pdu.diag_message.ForceListenOnlyModeRequest* method), 155

update_datastore() (*pymodbus.pdu.diag_message.GetClearModbusPlusRequest* method), 156

update_datastore() (*pymodbus.pdu.diag_message.ReturnBusCommunicationErrorCountRequest* method), 157

update_datastore() (*pymodbus.pdu.diag_message.ReturnBusExceptionErrorCountRequest* method), 157

update_datastore() (*pymodbus.pdu.diag_message.ReturnBusMessageCountRequest* method), 157

update_datastore() (*pymodbus.pdu.diag_message.ReturnDeviceBusCharacterOverrunCountRequest* method), 158

update_datastore() (*pymodbus.pdu.diag_message.ReturnDeviceBusyCountRequest* method), 158

update_datastore() (*pymodbus.pdu.diag_message.ReturnDeviceMessageCountRequest* method), 158

update_datastore() (*pymodbus.pdu.diag_message.ReturnDeviceNAKCountRequest* method), 159

update_datastore() (*pymodbus.pdu.diag_message.ReturnDeviceNoResponseCountRequest* method), 159

update_datastore() (*pymodbus.pdu.diag_message.ReturnDiagnosticRegisterRequest* method), 160

update_datastore() (*pymodbus.pdu.diag_message.ReturnIopOverrunCountRequest* method), 160

update_datastore() (*pymodbus.pdu.file_message.ReadFifoQueueRequest* method), 161

update_datastore() (*pymodbus.pdu.file_message.ReadFileRecordRequest* method), 162

update_datastore() (*pymodbus.pdu.file_message.WriteFileRecordRequest* method), 162

update_datastore() (*pymodbus.pdu.mei_message.ReadDeviceInformationRequest* method), 163

update_datastore() (*pymodbus.pdu.other_message.GetCommEventCounterRequest* method), 164

update_datastore() (*pymodbus.pdu.other_message.GetCommEventLogRequest* method), 164

update_datastore() (*pymodbus.pdu.other_message.ReadExceptionStatusRequest* method), 165

update_datastore() (*pymodbus.pdu.other_message.ReportDeviceIdRequest* method), 166

update_datastore() (*pymodbus.pdu.register_message.MaskWriteRegisterRequest* method), 168

update_datastore() (*pymodbus.pdu.register_message.ReadHoldingRegistersRequest* method), 168

update_datastore() (*pymodbus.pdu.register_message.ReadWriteMultipleRegistersRequest* method), 168

method), 170

`update_datastore()` (*pymodbus.pdu.register_message.WriteMultipleRegistersRequest* (*class in pymodbus.pdu.register_message*), 170

`update_datastore()` (*pymodbus.pdu.register_message.WriteSingleRegisterRequest* (*class in pymodbus.pdu.register_message*), 171

`UserApplicationName` (*pymodbus.ModbusDeviceIdentification* *property*), 146

`UserApplicationName` (*pymodbus.pdu.device.ModbusDeviceIdentification* *property*), 172

`WriteMultipleCoilsResponse` (*class in pymodbus.pdu.bit_message*), 153

`WriteMultipleRegistersRequest` (*class in pymodbus.pdu.register_message*), 170

`WriteMultipleRegistersResponse` (*class in pymodbus.pdu.register_message*), 170

`WriteSingleCoilRequest` (*class in pymodbus.pdu.bit_message*), 153

`WriteSingleCoilResponse` (*class in pymodbus.pdu.bit_message*), 153

`WriteSingleRegisterRequest` (*class in pymodbus.pdu.register_message*), 171

`WriteSingleRegisterResponse` (*class in pymodbus.pdu.register_message*), 171

V

`value` (*pymodbus.pdu.events.CommunicationRestartEvent* *attribute*), 173

`value` (*pymodbus.pdu.events.EnteredListenModeEvent* *attribute*), 174

`values` (*pymodbus.simulator.SimData* *attribute*), 81

`VendorName` (*pymodbus.ModbusDeviceIdentification* *property*), 147

`VendorName` (*pymodbus.pdu.device.ModbusDeviceIdentification* *property*), 172

`VendorUrl` (*pymodbus.ModbusDeviceIdentification* *property*), 147

`VendorUrl` (*pymodbus.pdu.device.ModbusDeviceIdentification* *property*), 172

W

`WAITING` (*pymodbus.constants.ModbusStatus* *attribute*), 145

`write_coil()` (*pymodbus.client.mixin.ModbusClientMixin* *method*), 26

`write_coils()` (*pymodbus.client.mixin.ModbusClientMixin* *method*), 32

`write_file_record()` (*pymodbus.client.mixin.ModbusClientMixin* *method*), 33

`write_register()` (*pymodbus.client.mixin.ModbusClientMixin* *method*), 26

`write_registers()` (*pymodbus.client.mixin.ModbusClientMixin* *method*), 32

`WriteFileRecordRequest` (*class in pymodbus.pdu.file_message*), 162

`WriteFileRecordResponse` (*class in pymodbus.pdu.file_message*), 162

`WriteMultipleCoilsRequest` (*class in pymodbus.pdu.bit_message*), 152